SAKARYA ÜNİVERSİTESİ

Bilgisayar ve Bilişim Bilimleri Fakültesi

BSM 211 Veritabanı Yönetim Sistemleri
PROJE ÇALIŞMASI

E-SPOR TURNUVA YÖNETİM SİSTEMİ

PostgreSQL · PL/pgSQL · Python / Flask Web Uygulaması

Öğrenci Numarası	Adı	Soyadı	E-posta
B251210043	Abdurrahman Turan	Ozcan	abdurrahman.ozcan@ogr.sakarya.edu.tr

22.10.2026

İçindekiler

a. Uygulamanın Kısa Tanıtımı ve İş Kuralları

b. Varlık Bağıntı Modeli (Crow's Foot, Kalıtım) ve İlişkisel Şema

c. Veritabanını Verilerle Birlikte Oluşturan SQL İfadeleri

d. Saklı Yordamlar / Fonksiyonlar ve Tetikleyiciler

e. Arama, Ekleme, Silme ve Güncelleme Ekran Görüntüleri

f. Uygulamanın Kaynak Kodları

a) Uygulamanın Kısa Tanıtımı ve İş Kuralları

Uygulamanın Tanıtımı

E-spor Turnuva Yönetim Sistemi, e-spor organizasyonlarının oyuncu, takım, turnuva ve maç süreçlerini yönetmek için geliştirilmiş bir web uygulamasıdır. Sistemde Valorant, Counter-Strike 2, League of Legends, Rocket League gibi farklı oyunlarda yarışan takımlar ve bu takımların kadroları, koçları ve sponsorları tutulur. Turnuvalar *online* (yayın platformu ve sunucu bölgesi ile) veya *salon* (fiziksel bir mekanda, biletli) olarak düzenlenebilir.

Turnuvaya kaydolan takımlar için sistem, **herkesin herkesle oynadığı lig usulü fikstürü** otomatik oluşturur ve hakemleri maçlara sırayla atar. Maç sonuçları girildikçe **puan tablosu tetikleyicilerle anlık güncellenir**; oyuncuların maç başına öldürme / ölüm / asist istatistikleri ve MVP bilgisi tutulur. Turnuva tamamlandığında final sıralaması hesaplanır ve ödül havuzu dereceye giren takımlara otomatik olarak dağıtılır. Kişi, takım ve turnuva tablolarındaki tüm değişiklikler denetim amacıyla işlem günlüğüne yazılır.

Uygulama **Database-First** yaklaşımıyla geliştirilmiştir: veritabanı önce SQL betikleriyle (tablolar, kısıtlar, fonksiyonlar, saklı yordamlar, tetikleyiciler) PostgreSQL üzerinde oluşturulmuş; Python / Flask ile yazılan web uygulaması herhangi bir ORM kullanmadan bu veritabanına *psycopg* sürücüsüyle bağlanarak tablolara SQL ile, iş mantığına ise veritabanındaki fonksiyon ve saklı yordamlar aracılığıyla erişir. Uygulamada oyuncu, takım, turnuva, maç, kadro, ödül ve istatistik kayıtları için **arama, ekleme, güncelleme ve silme** işlemleri bulunmaktadır.

Kullanılan teknolojiler: PostgreSQL (VTYS), PL/pgSQL, Python 3, Flask, Jinja2, HTML/CSS.

21	2	9	14
tablo	kalıtım hiyerarşisi	fonksiyon / saklı yordam	tetikleyici

İş Kuralları

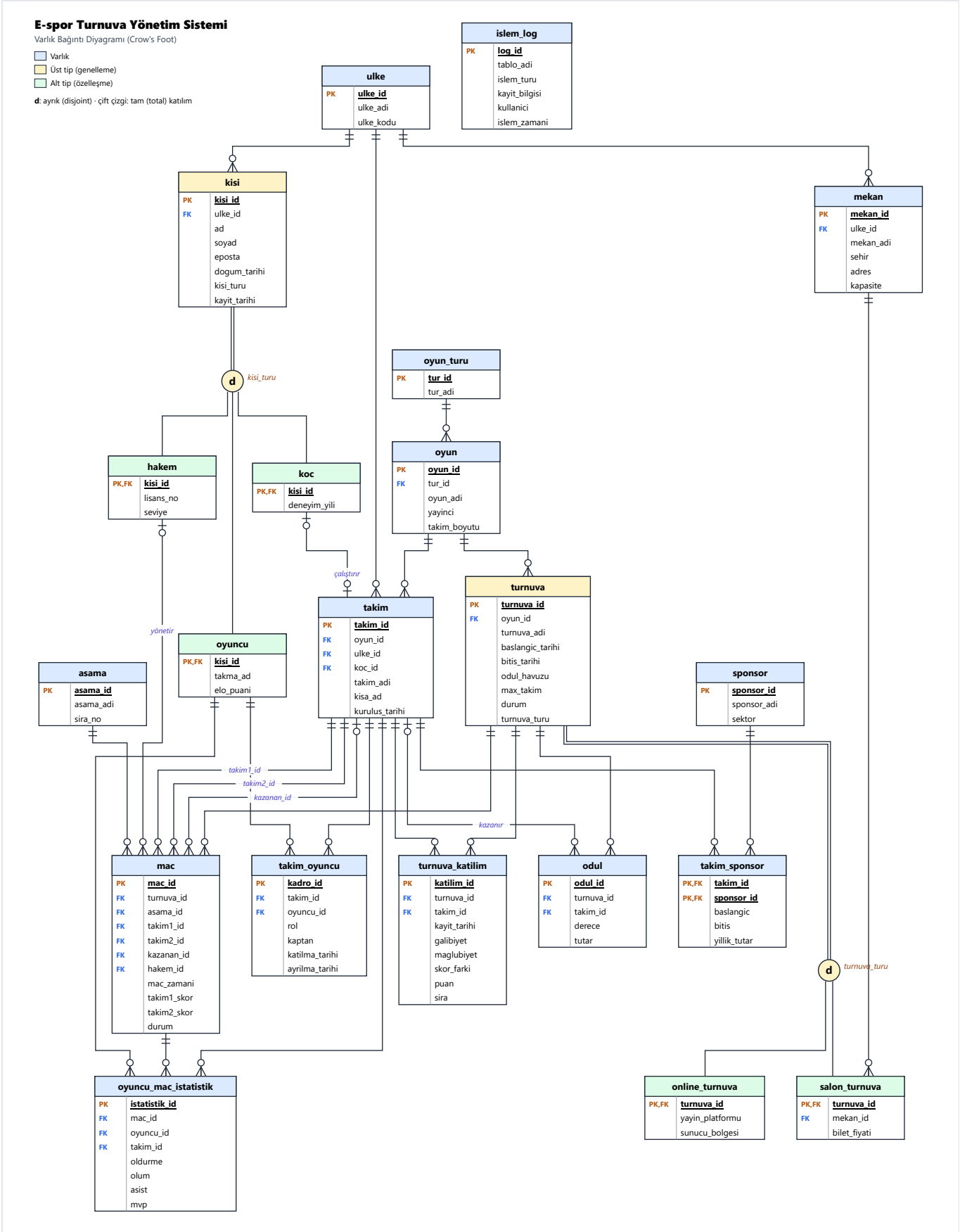
1. Sistemdeki her kişi bir ülkeye aittir; bir ülkede sıfır veya daha fazla kişi bulunabilir.
2. Kişiler **oyuncu**, **koç** veya **hakem** olabilir. Her kişi bu türlerden mutlaka birine ve yalnızca birine aittir (ayrık ve tam genelleme; ayırt edici nitelik *kisi_turu*).
3. Oyuncular en az 16, koçlar ve hakemler en az 18 yaşında olmalıdır.
4. Her oyuncunun benzersiz bir takma adı ve bir ELO puanı vardır; her hakemin benzersiz bir lisans numarası ve seviyesi (Ulusal / Uluslararası) vardır. Kişilerin e-posta adresleri benzersizdir.
5. Her oyun bir oyun türüne aittir; bir oyun türünde birden çok oyun olabilir. Her oyunun sahadaki oyuncu sayısı (takım boyutu) tanımlıdır.
6. Her takım yalnızca bir oyunda yarışır ve bir ülkeye bağlıdır; bir oyunda birden çok takım olabilir.
7. Bir takımı en fazla bir koç çalıştırır, bir koç da en fazla bir takımı çalıştırır (1:1, isteğe bağlı).
8. Bir oyuncu kariyeri boyunca birden çok takımda oynayabilir ve bir takımda birden çok oyuncu bulunur (kadro geçmişi tutulur); ancak bir oyuncu aynı anda yalnızca bir takımın aktif kadrosunda olabilir.
9. Bir takımın aktif kadrosu, oyunun takım boyutunun en fazla 2 fazlası (yedekler) kadar olabilir. Her takımın en fazla bir kaptanı vardır; yeni kaptan atanınca eskisinin kaptanlığı kalkar.
10. Bir takımın birden çok sponsoru, bir sponsorun da birden çok takımı olabilir. Sponsorluk; başlangıç, bitiş tarihi ve yıllık tutar bilgisiyle tutulur.

11. Turnuvalar **online** veya **salon** turnuvası olabilir (ayrık ve tam genelleme; ayırt edici nitelik *turnuva_turu*). Online turnuvanın yayın platformu ve sunucu bölgesi, salon turnuvasının mekanı ve bilet fiyatı vardır. Bir mekanda birden çok salon turnuvası düzenlenebilir.
12. Her turnuva tek bir oyun için düzenlenir; bir takım yalnızca kendi oyununa ait turnuvalara kaydolabilir.
13. Turnuvaya kayıt yalnızca turnuva "Kayıt Açık" durumundayken yapılabilir; kayıtlı takım sayısı turnuva kontenjanını aşamaz ve kaydolan takımın aktif kadrosunda en az takım boyutu kadar oyuncu bulunmalıdır.
14. Fikstür, kayıtlı takımlar arasında her takımın diğer her takımla bir kez karşılaştığı lig usulüyle oluşturulur; fikstür oluşturulunca turnuva "Devam Ediyor" durumuna geçer ve yeni kayıt alınmaz.
15. Her maç bir turnuvaya ve bir aşamaya (Grup, Çeyrek Final, Yarı Final, Final) aittir; iki farklı takım arasında oynanır ve bir hakem tarafından yönetilebilir. Maçtaki iki takım da turnuvaya kayıtlı olmalıdır.
16. E-sporda beraberlik yoktur. Maçı kazanan takım 3 puan alır, kaybeden puan alamaz; skor farkı averaj olarak tutulur. Sonucu girilmiş bir maçın skoru değiştirilemez.
17. Maç istatistikleri yalnızca oynanmış maçlar için ve maç tarihinde maçtaki takımlardan birinin kadrosunda bulunan oyuncular için girilebilir. Bir oyuncunun bir maçta tek istatistik kaydı, her maçın en fazla bir MVP'si olur.
18. Turnuvanın ödülleri dereceye göre tanımlanır; ödüllerin toplamı turnuvanın ödül havuzunu aşamaz.
19. Tüm maçları oynanmadan turnuva tamamlanamaz. Turnuva tamamlandığında takımların final sıraları belirlenir ve ödüller ilgili takımlara atanır. Tamamlanmış bir turnuvanın durumu değiştirilemez.
20. Devam eden bir turnuva silinemez; maç oynamış bir takım silinemez.
21. Kişi, takım ve turnuva kayıtları üzerinde yapılan her ekleme, güncelleme ve silme işlemi; tablo adı, işlem türü, kayıt içeriği, kullanıcı ve zaman bilgisiyle işlem günlüğüne yazılır.

b) Varlık Bağıntı Modeli ve İlişkisel Şema

Varlık Bağıntı Diyagramı (Crow's Foot)

Diyagram draw.io ile düzenlenebilir biçimde *diyagram/er_diyagrami.drawio* dosyasında yer almaktadır.



Genelleme / Kalıtım

Modelde iki genelleme (kalıtım) hiyerarşisi bulunmaktadır:

- **kisi → oyuncu, koc, hakem**: Ortak nitelikler (ad, soyad, e-posta, doğum tarihi, ülke) üst tipte; türe özgü nitelikler (takma ad ve ELO / deneyim yılı / lisans no ve seviye) alt tiplerdedir.
- **turnuva → online_turnuva, salon_turnuva**: Ortak nitelikler (ad, oyun, tarihler, ödül havuzu, kontenjan, durum) üst tipte; online turnuvaya özgü yayın platformu ve sunucu bölgesi ile salon turnuvasına özgü mekan ve bilet fiyatı alt tiplerdedir.

Her iki hiyerarşi de **ayrık (disjoint, "d")** ve **tam (total, çift çizgi)** genellemedir. Gerçekleme, alt tip tablolarının birincil anahtarının aynı zamanda üst tipe yabancı anahtar olduğu (*paylaşılan birincil anahtar*) yöntemle yapılmıştır; ayırt edici nitelikler *kisi_turu* ve *turnuva_turu*dur. Üst tipten bir kayıt silindiğinde alt tip kaydı *ON DELETE CASCADE* ile birlikte silinir. Alt tip ile ayırt edici niteliğin uyumu *trg_kisi_alt_tip_kontrol* tetikleyicisi ve *oyuncu_ekle* / *turnuva_olustur* fonksiyonlarıyla güvence altına alınmıştır. (PostgreSQL'in *INHERITS* özelliği, birincil anahtar / benzersizlik ve yabancı anahtar kısıtlarını alt tablolara aktarmadığı için tercih edilmemiştir.)

İlişkisel Şema (Metinsel Gösterim)

Birincil anahtarlar **altı çizili**, yabancı anahtarlar *eğik* ve FK ile gösterilmiştir.

> **ulke** (**ulke_id**, ulke_adi, ulke_kodu)

> **oyun_turu** (**tur_id**, tur_adi)

> **oyun** (**oyun_id**, oyun_adi, *tur_id*FK, yayinci, takim_boyutu)
> Yabancı anahtar: tur_id → oyun_turu(tur_id)

> **kisi** (**kisi_id**, ad, soyad, eposta, dogum_tarihi, *ulke_id*FK, kisi_turu, kayit_tarihi)
> Yabancı anahtar: ulke_id → ulke(ulke_id)

> **oyuncu** (***kisi_id***FK, takma_ad, elo_puani)
> Yabancı anahtar: kisi_id → kisi(kisi_id)

> **koc** (***kisi_id***FK, deneyim_yili)
> Yabancı anahtar: kisi_id → kisi(kisi_id)

> **hakem** (***kisi_id***FK, lisans_no, seviye)
> Yabancı anahtar: kisi_id → kisi(kisi_id)

> **takim** (**takim_id**, takim_adi, kisa_ad, *oyun_id*FK, *ulke_id*FK, *koc_id*FK, kurulus_tarihi)
> Yabancı anahtar: oyun_id → oyun(oyun_id); ulke_id → ulke(ulke_id); koc_id → koc(kisi_id)

> **takim_oyuncu** (**kadro_id**, *takim_id*FK, *oyuncu_id*FK, rol, kaptan, katilma_tarihi, ayrilma_tarihi)
> Yabancı anahtar: takim_id → takim(takim_id); oyuncu_id → oyuncu(kisi_id)

> **sponsor** (**sponsor_id**, sponsor_adi, sektor)

> **takim_sponsor** (***takim_id***FK, ***sponsor_id***FK, baslangic, bitis, yillik_tutar)
> Yabancı anahtar: takim_id → takim(takim_id); sponsor_id → sponsor(sponsor_id)

mekan (**mekan_id**, mekan_adi, *ulke_id*[FK], sehir, adres, kapasite)

Yabancı anahtar: ulke_id → ulke(ulke_id)

turnuva (**turnuva_id**, turnuva_adi, *oyun_id*[FK], baslangic_tarihi, bitis_tarihi, odul_havuzu, max_takim, durum, turnuva_turu)

Yabancı anahtar: oyun_id → oyun(oyun_id)

online_turnuva (***turnuva_id***[FK], yayin_platformu, sunucu_bolgesi)

Yabancı anahtar: turnuva_id → turnuva(turnuva_id)

salon_turnuva (***turnuva_id***[FK], *mekan_id*[FK], bilet_fiyati)

Yabancı anahtar: turnuva_id → turnuva(turnuva_id); mekan_id → mekan(mekan_id)

turnuva_katilim (**katilim_id**, *turnuva_id*[FK], *takim_id*[FK], kayit_tarihi, galibiyet, maglubiyet, skor_farki, puan, sira)

Yabancı anahtar: turnuva_id → turnuva(turnuva_id); takim_id → takim(takim_id)

asama (**asama_id**, asama_adi, sira_no)

mac (**mac_id**, *turnuva_id*[FK], *asama_id*[FK], *takim1_id*[FK], *takim2_id*[FK], *hakem_id*[FK], mac_zamani, takim1_skor, takim2_skor, *kazanan_id*[FK], durum)

Yabancı anahtar: turnuva_id → turnuva(turnuva_id); asama_id → asama(asama_id); takim1_id → takim(takim_id); takim2_id → takim(takim_id); hakem_id → hakem(kisi_id); kazanan_id → takim(takim_id)

oyuncu_mac_istatistik (**istatistik_id**, *mac_id*[FK], *oyuncu_id*[FK], *takim_id*[FK], oldurme, olum, asist, mvp)

Yabancı anahtar: mac_id → mac(mac_id); oyuncu_id → oyuncu(kisi_id); takim_id → takim(takim_id)

odul (**odul_id**, *turnuva_id*[FK], derece, tutar, *takim_id*[FK])

Yabancı anahtar: turnuva_id → turnuva(turnuva_id); takim_id → takim(takim_id)

islem_log (**log_id**, tablo_adi, islem_turu, kayit_bilgisi, kullanici, islem_zamani)

c) Veritabanını Verilerle Birlikte Oluşturan SQL İfadeleri

Betikler sırasıyla çalıştırıldığında tablolar, kısıtlar, fonksiyonlar, saklı yordamlar, tetikleyiciler ve örnek veriler oluşturulur (*python kurulum.py* veya psql ile). Örnek veri betiği; turnuvaları *turnuva_olustur*, fikstürleri *fikstur_olustur*, maç sonuçlarını *mac_sonucu_gir* ile girerek fonksiyon ve tetikleyicileri de çalıştırır.

01_tablolar.sql

```
-- ======================================================================
--  E-SPOR TURNUVA YÖNETİM SİSTEMİ
--  01 - TABLOLAR
--  Veritabanı: PostgreSQL   Yaklaşım: Database-First
-- ======================================================================

DROP TABLE IF EXISTS islem_log, odul, oyuncu_mac_istatistik, mac, asama,
    turnuva_katilim, salon_turnuva, online_turnuva, turnuva, mekan,
    takim_sponsor, sponsor, takim_oyuncu, takim, hakem, koc, oyuncu, kisi,
    oyun, oyun_turu, ulke
    CASCADE;

-- ----------------------------------------------------------------------
--  TANIM TABLOLARI
-- ----------------------------------------------------------------------
CREATE TABLE ulke (
    ulke_id     SERIAL       PRIMARY KEY,
    ulke_adi    VARCHAR(50)  NOT NULL UNIQUE,
    ulke_kodu   CHAR(2)      NOT NULL UNIQUE
);

CREATE TABLE oyun_turu (
    tur_id      SERIAL       PRIMARY KEY,
    tur_adi     VARCHAR(40)  NOT NULL UNIQUE
);

CREATE TABLE oyun (
    oyun_id       SERIAL       PRIMARY KEY,
    oyun_adi      VARCHAR(60)  NOT NULL UNIQUE,
    tur_id        INT          NOT NULL REFERENCES oyun_turu(tur_id),
    yayinci       VARCHAR(60)  NOT NULL,
    takim_boyutu  SMALLINT     NOT NULL CHECK (takim_boyutu BETWEEN 1 AND 10)  -- sahadaki oyuncu sayısı
);

-- ----------------------------------------------------------------------
--  KİŞİ  (KALITIM: kisi  ->  oyuncu | koc | hakem)
--  Ayrık (disjoint) ve tam (total) genelleme; ayırt edici nitelik: kisi_turu
-- ----------------------------------------------------------------------
CREATE TABLE kisi (
    kisi_id        SERIAL        PRIMARY KEY,
    ad             VARCHAR(40)   NOT NULL,
    soyad          VARCHAR(40)   NOT NULL,
    eposta         VARCHAR(100)  NOT NULL UNIQUE CHECK (eposta LIKE '%_@_%._%'),
    dogum_tarihi   DATE          NOT NULL,
    ulke_id        INT           NOT NULL REFERENCES ulke(ulke_id),
    kisi_turu      CHAR(1)       NOT NULL CHECK (kisi_turu IN ('O', 'K', 'H')),  -- O: oyuncu, K: koç, H: hakem
    kayit_tarihi   TIMESTAMP     NOT NULL DEFAULT now()
);

CREATE TABLE oyuncu (
    kisi_id      INT          PRIMARY KEY REFERENCES kisi(kisi_id) ON DELETE CASCADE,
    takma_ad     VARCHAR(30)  NOT NULL UNIQUE,
    elo_puani    INT          NOT NULL DEFAULT 1000 CHECK (elo_puani >= 0)
);

CREATE TABLE koc (
    kisi_id       INT       PRIMARY KEY REFERENCES kisi(kisi_id) ON DELETE CASCADE,
    deneyim_yili  SMALLINT  NOT NULL DEFAULT 0 CHECK (deneyim_yili >= 0)
);

CREATE TABLE hakem (
    kisi_id    INT          PRIMARY KEY REFERENCES kisi(kisi_id) ON DELETE CASCADE,
    lisans_no  VARCHAR(20)  NOT NULL UNIQUE,
    seviye     VARCHAR(15)  NOT NULL CHECK (seviye IN ('Ulusal', 'Uluslararası'))
```

```
);

-- ---------------------------------------------------------------------
--  TAKIM, KADRO, SPONSOR
-- ---------------------------------------------------------------------
CREATE TABLE takim (
    takim_id        SERIAL       PRIMARY KEY,
    takim_adi       VARCHAR(60)  NOT NULL UNIQUE,
    kisa_ad         VARCHAR(5)   NOT NULL UNIQUE,
    oyun_id         INT          NOT NULL REFERENCES oyun(oyun_id),
    ulke_id         INT          NOT NULL REFERENCES ulke(ulke_id),
    koc_id          INT          UNIQUE REFERENCES koc(kisi_id) ON DELETE SET NULL,  -- bir koç tek takımı çalıştırır
    kurulus_tarihi  DATE         NOT NULL DEFAULT current_date
);

CREATE TABLE takim_oyuncu (
    kadro_id        SERIAL       PRIMARY KEY,
    takim_id        INT          NOT NULL REFERENCES takim(takim_id) ON DELETE CASCADE,
    oyuncu_id       INT          NOT NULL REFERENCES oyuncu(kisi_id) ON DELETE CASCADE,
    rol             VARCHAR(20)  NOT NULL DEFAULT 'Oyuncu',
    kaptan          BOOLEAN      NOT NULL DEFAULT FALSE,
    katilma_tarihi  DATE         NOT NULL DEFAULT current_date,
    ayrilma_tarihi  DATE,
    CONSTRAINT ck_kadro_tarih CHECK (ayrilma_tarihi IS NULL OR ayrilma_tarihi >= katilma_tarihi)
);
-- Bir oyuncu aynı anda yalnızca bir takımın aktif kadrosunda bulunabilir
CREATE UNIQUE INDEX uq_aktif_kadro ON takim_oyuncu (oyuncu_id) WHERE ayrilma_tarihi IS NULL;

CREATE TABLE sponsor (
    sponsor_id   SERIAL       PRIMARY KEY,
    sponsor_adi  VARCHAR(60)  NOT NULL UNIQUE,
    sektor       VARCHAR(40)  NOT NULL
);

CREATE TABLE takim_sponsor (
    takim_id      INT            NOT NULL REFERENCES takim(takim_id) ON DELETE CASCADE,
    sponsor_id    INT            NOT NULL REFERENCES sponsor(sponsor_id) ON DELETE CASCADE,
    baslangic     DATE           NOT NULL,
    bitis         DATE           NOT NULL,
    yillik_tutar  NUMERIC(12,2)  NOT NULL CHECK (yillik_tutar > 0),
    PRIMARY KEY (takim_id, sponsor_id),
    CONSTRAINT ck_sponsor_tarih CHECK (bitis > baslangic)
);

-- ---------------------------------------------------------------------
--  TURNUVA  (KALITIM: turnuva  ->  online_turnuva | salon_turnuva)
--  Ayrık ve tam genelleme; ayırt edici nitelik: turnuva_turu
-- ---------------------------------------------------------------------
CREATE TABLE mekan (
    mekan_id    SERIAL        PRIMARY KEY,
    mekan_adi   VARCHAR(80)   NOT NULL UNIQUE,
    ulke_id     INT           NOT NULL REFERENCES ulke(ulke_id),
    sehir       VARCHAR(50)   NOT NULL,
    adres       VARCHAR(200)  NOT NULL,
    kapasite    INT           NOT NULL CHECK (kapasite > 0)
);

CREATE TABLE turnuva (
    turnuva_id        SERIAL         PRIMARY KEY,
    turnuva_adi       VARCHAR(80)    NOT NULL UNIQUE,
    oyun_id           INT            NOT NULL REFERENCES oyun(oyun_id),
    baslangic_tarihi  DATE           NOT NULL,
    bitis_tarihi      DATE           NOT NULL,
    odul_havuzu       NUMERIC(12,2)  NOT NULL DEFAULT 0 CHECK (odul_havuzu >= 0),
    max_takim         SMALLINT       NOT NULL CHECK (max_takim BETWEEN 2 AND 64),
    durum             VARCHAR(15)    NOT NULL DEFAULT 'Kayıt Açık'
                                     CHECK (durum IN ('Kayıt Açık', 'Devam Ediyor', 'Tamamlandı')),
    turnuva_turu      CHAR(1)        NOT NULL CHECK (turnuva_turu IN ('O', 'S')),  -- O: online, S: salon
    CONSTRAINT ck_turnuva_tarih CHECK (bitis_tarihi >= baslangic_tarihi)
);

CREATE TABLE online_turnuva (
    turnuva_id      INT          PRIMARY KEY REFERENCES turnuva(turnuva_id) ON DELETE CASCADE,
    yayin_platformu VARCHAR(30)  NOT NULL,
    sunucu_bolgesi  VARCHAR(30)  NOT NULL
);
```

```
CREATE TABLE salon_turnuva (
    turnuva_id   INT           PRIMARY KEY REFERENCES turnuva(turnuva_id) ON DELETE CASCADE,
    mekan_id     INT           NOT NULL REFERENCES mekan(mekan_id),
    bilet_fiyati NUMERIC(8,2)  NOT NULL DEFAULT 0 CHECK (bilet_fiyati >= 0)
);

-- ---------------------------------------------------------------------
--  KATILIM (aynı zamanda puan tablosu), AŞAMA, MAÇ, İSTATİSTİK, ÖDÜL
-- ---------------------------------------------------------------------
CREATE TABLE turnuva_katilim (
    katilim_id    SERIAL     PRIMARY KEY,
    turnuva_id    INT        NOT NULL REFERENCES turnuva(turnuva_id) ON DELETE CASCADE,
    takim_id      INT        NOT NULL REFERENCES takim(takim_id) ON DELETE CASCADE,
    kayit_tarihi  TIMESTAMP  NOT NULL DEFAULT now(),
    galibiyet     SMALLINT   NOT NULL DEFAULT 0,
    maglubiyet    SMALLINT   NOT NULL DEFAULT 0,
    skor_farki    SMALLINT   NOT NULL DEFAULT 0,
    puan          SMALLINT   NOT NULL DEFAULT 0,
    sira          SMALLINT,                       -- turnuva bitince tetikleyici ile atanır
    CONSTRAINT uq_katilim UNIQUE (turnuva_id, takim_id)
);

CREATE TABLE asama (
    asama_id   SERIAL       PRIMARY KEY,
    asama_adi  VARCHAR(30)  NOT NULL UNIQUE,
    sira_no    SMALLINT     NOT NULL UNIQUE
);

CREATE TABLE mac (
    mac_id       SERIAL       PRIMARY KEY,
    turnuva_id   INT          NOT NULL REFERENCES turnuva(turnuva_id) ON DELETE CASCADE,
    asama_id     INT          NOT NULL REFERENCES asama(asama_id),
    takim1_id    INT          NOT NULL REFERENCES takim(takim_id),
    takim2_id    INT          NOT NULL REFERENCES takim(takim_id),
    hakem_id     INT          REFERENCES hakem(kisi_id) ON DELETE SET NULL,
    mac_zamani   TIMESTAMP    NOT NULL,
    takim1_skor  SMALLINT     CHECK (takim1_skor >= 0),
    takim2_skor  SMALLINT     CHECK (takim2_skor >= 0),
    kazanan_id   INT          REFERENCES takim(takim_id),   -- tetikleyici ile atanır
    durum        VARCHAR(10)  NOT NULL DEFAULT 'Planlandı'
                              CHECK (durum IN ('Planlandı', 'Oynandı', 'İptal')),
    CONSTRAINT ck_farkli_takim CHECK (takim1_id <> takim2_id),
    -- e-sporda beraberlik yoktur: oynanmış maçın skorları farklı olmalıdır
    CONSTRAINT ck_mac_skor CHECK (durum <> 'Oynandı'
        OR (takim1_skor IS NOT NULL AND takim2_skor IS NOT NULL AND takim1_skor <> takim2_skor))
);

CREATE TABLE oyuncu_mac_istatistik (
    istatistik_id  SERIAL    PRIMARY KEY,
    mac_id         INT       NOT NULL REFERENCES mac(mac_id) ON DELETE CASCADE,
    oyuncu_id      INT       NOT NULL REFERENCES oyuncu(kisi_id) ON DELETE CASCADE,
    takim_id       INT       NOT NULL REFERENCES takim(takim_id) ON DELETE CASCADE,  -- tetikleyici ile atanır
    oldurme        SMALLINT  NOT NULL DEFAULT 0 CHECK (oldurme >= 0),
    olum           SMALLINT  NOT NULL DEFAULT 0 CHECK (olum >= 0),
    asist          SMALLINT  NOT NULL DEFAULT 0 CHECK (asist >= 0),
    mvp            BOOLEAN   NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_istatistik UNIQUE (mac_id, oyuncu_id)
);

CREATE TABLE odul (
    odul_id     SERIAL         PRIMARY KEY,
    turnuva_id  INT            NOT NULL REFERENCES turnuva(turnuva_id) ON DELETE CASCADE,
    derece      SMALLINT       NOT NULL CHECK (derece > 0),
    tutar       NUMERIC(12,2)  NOT NULL CHECK (tutar > 0),
    takim_id    INT            REFERENCES takim(takim_id) ON DELETE SET NULL,  -- turnuva bitince atanır
    CONSTRAINT uq_odul UNIQUE (turnuva_id, derece)
);

-- ---------------------------------------------------------------------
--  DENETİM KAYITLARI
-- ---------------------------------------------------------------------
CREATE TABLE islem_log (
    log_id         SERIAL       PRIMARY KEY,
    tablo_adi      VARCHAR(30)  NOT NULL,
    islem_turu     VARCHAR(10)  NOT NULL,
    kayit_bilgisi  TEXT,
    kullanici      VARCHAR(50)  NOT NULL DEFAULT current_user,
```

```
    islem_zamani   TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE INDEX ix_mac_turnuva        ON mac (turnuva_id);
CREATE INDEX ix_istatistik_oyuncu  ON oyuncu_mac_istatistik (oyuncu_id);
CREATE INDEX ix_kadro_takim        ON takim_oyuncu (takim_id);
```

02_fonksiyonlar.sql

```
-- ======================================================================
--  E-SPOR TURNUVA YÖNETİM SİSTEMİ
--  02 - FONKSİYONLAR VE SAKLI YORDAMLAR
--  (Tetikleyici fonksiyonları 03_tetikleyiciler.sql dosyasındadır.)
-- ======================================================================

-- ----------------------------------------------------------------------
-- 1) oyuncu_ekle: Kalıtım hiyerarşisine uygun şekilde önce üst tipe (kisi),
--    ardından alt tipe (oyuncu) kayıt ekler; yeni kişinin numarasını döndürür.
-- ----------------------------------------------------------------------
CREATE OR REPLACE FUNCTION oyuncu_ekle(
    p_ad         VARCHAR,
    p_soyad      VARCHAR,
    p_eposta     VARCHAR,
    p_dogum      DATE,
    p_ulke_id    INT,
    p_takma_ad   VARCHAR,
    p_elo        INT DEFAULT 1000
) RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_kisi_id INT;
BEGIN
    INSERT INTO kisi (ad, soyad, eposta, dogum_tarihi, ulke_id, kisi_turu)
    VALUES (p_ad, p_soyad, lower(p_eposta), p_dogum, p_ulke_id, 'O')
    RETURNING kisi_id INTO v_kisi_id;

    INSERT INTO oyuncu (kisi_id, takma_ad, elo_puani)
    VALUES (v_kisi_id, p_takma_ad, COALESCE(p_elo, 1000));

    RETURN v_kisi_id;
END;
$$;

-- ----------------------------------------------------------------------
-- 2) oyuncu_guncelle (SAKLI YORDAM): Üst tip (kisi) ve alt tip (oyuncu)
--    tablolarındaki bilgileri tek işlemde günceller.
-- ----------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE oyuncu_guncelle(
    p_kisi_id    INT,
    p_ad         VARCHAR,
    p_soyad      VARCHAR,
    p_eposta     VARCHAR,
    p_dogum      DATE,
    p_ulke_id    INT,
    p_takma_ad   VARCHAR,
    p_elo        INT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE kisi
       SET ad = p_ad, soyad = p_soyad, eposta = lower(p_eposta),
           dogum_tarihi = p_dogum, ulke_id = p_ulke_id
     WHERE kisi_id = p_kisi_id AND kisi_turu = 'O';

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Güncellenecek oyuncu bulunamadı (no: %).', p_kisi_id;
    END IF;

    UPDATE oyuncu
       SET takma_ad = p_takma_ad, elo_puani = p_elo
     WHERE kisi_id = p_kisi_id;
END;
$$;

-- ----------------------------------------------------------------------
```

```
-- 3) oyuncu_ara: Takma ad, ad, soyad veya takım adına göre oyuncu arar.
--    Boş arama metni tüm oyuncuları getirir.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION oyuncu_ara(p_aranan VARCHAR DEFAULT '')
RETURNS TABLE (
    oyuncu_id   INT,
    takma_ad    VARCHAR,
    ad_soyad    TEXT,
    ulke_adi    VARCHAR,
    yas         INT,
    takim_adi   VARCHAR,
    oyun_adi    VARCHAR,
    elo_puani   INT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT o.kisi_id,
           o.takma_ad,
           k.ad || ' ' || k.soyad,
           u.ulke_adi,
           date_part('year', age(k.dogum_tarihi))::INT,
           t.takim_adi,
           oy.oyun_adi,
           o.elo_puani
      FROM oyuncu o
      JOIN kisi k              ON k.kisi_id = o.kisi_id
      JOIN ulke u              ON u.ulke_id = k.ulke_id
      LEFT JOIN takim_oyuncu ko ON ko.oyuncu_id = o.kisi_id AND ko.ayrilma_tarihi IS NULL
      LEFT JOIN takim t        ON t.takim_id = ko.takim_id
      LEFT JOIN oyun oy        ON oy.oyun_id = t.oyun_id
     WHERE COALESCE(p_aranan, '') = ''
        OR o.takma_ad ILIKE '%' || p_aranan || '%'
        OR k.ad       ILIKE '%' || p_aranan || '%'
        OR k.soyad    ILIKE '%' || p_aranan || '%'
        OR t.takim_adi ILIKE '%' || p_aranan || '%'
     ORDER BY o.elo_puani DESC, o.takma_ad;
END;
$$;

-- ---------------------------------------------------------------------
-- 4) turnuva_olustur: Turnuva türüne göre üst tip (turnuva) ile birlikte
--    ilgili alt tipe (online_turnuva veya salon_turnuva) kayıt ekler.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION turnuva_olustur(
    p_adi        VARCHAR,
    p_oyun_id    INT,
    p_baslangic  DATE,
    p_bitis      DATE,
    p_odul       NUMERIC,
    p_max_takim  SMALLINT,
    p_tur        CHAR,                      -- 'O' online, 'S' salon
    p_platform   VARCHAR DEFAULT NULL,      -- online için
    p_bolge      VARCHAR DEFAULT NULL,      -- online için
    p_mekan_id   INT     DEFAULT NULL,      -- salon için
    p_bilet      NUMERIC DEFAULT 0          -- salon için
) RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_id INT;
BEGIN
    IF p_tur = 'O' AND (p_platform IS NULL OR p_bolge IS NULL) THEN
        RAISE EXCEPTION 'Online turnuva için yayın platformu ve sunucu bölgesi zorunludur.';
    ELSIF p_tur = 'S' AND p_mekan_id IS NULL THEN
        RAISE EXCEPTION 'Salon turnuvası için mekan seçilmelidir.';
    END IF;

    INSERT INTO turnuva (turnuva_adi, oyun_id, baslangic_tarihi, bitis_tarihi,
                         odul_havuzu, max_takim, turnuva_turu)
    VALUES (p_adi, p_oyun_id, p_baslangic, p_bitis, p_odul, p_max_takim, p_tur)
    RETURNING turnuva_id INTO v_id;

    IF p_tur = 'O' THEN
        INSERT INTO online_turnuva (turnuva_id, yayin_platformu, sunucu_bolgesi)
        VALUES (v_id, p_platform, p_bolge);
    ELSE
        INSERT INTO salon_turnuva (turnuva_id, mekan_id, bilet_fiyati)
```

```
        VALUES (v_id, p_mekan_id, COALESCE(p_bilet, 0));
    END IF;

    RETURN v_id;
END;
$$;

-- ---------------------------------------------------------------------
-- 5) fikstur_olustur: Turnuvaya kayıtlı takımlar için "herkes herkesle"
--    (lig usulü) grup maçlarını oluşturur, hakemleri sırayla atar ve
--    turnuvayı "Devam Ediyor" durumuna getirir. Oluşan maç sayısını döndürür.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fikstur_olustur(
    p_turnuva_id  INT,
    p_ilk_mac     TIMESTAMP,
    p_aralik      INTERVAL DEFAULT '2 hours'
) RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_durum     VARCHAR;
    v_takimlar  INT[];
    v_hakemler  INT[];
    v_asama_id  INT;
    v_sayac     INT := 0;
    i           INT;
    j           INT;
BEGIN
    SELECT durum INTO v_durum FROM turnuva WHERE turnuva_id = p_turnuva_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Turnuva bulunamadı.';
    ELSIF v_durum <> 'Kayıt Açık' THEN
        RAISE EXCEPTION 'Fikstür yalnızca "Kayıt Açık" durumundaki turnuvalar için oluşturulabilir.';
    ELSIF EXISTS (SELECT 1 FROM mac WHERE turnuva_id = p_turnuva_id) THEN
        RAISE EXCEPTION 'Bu turnuvanın fikstürü zaten oluşturulmuş.';
    END IF;

    SELECT array_agg(takim_id ORDER BY katilim_id) INTO v_takimlar
      FROM turnuva_katilim WHERE turnuva_id = p_turnuva_id;

    IF COALESCE(array_length(v_takimlar, 1), 0) < 2 THEN
        RAISE EXCEPTION 'Fikstür için en az 2 takımın kayıtlı olması gerekir.';
    END IF;

    SELECT array_agg(kisi_id ORDER BY kisi_id) INTO v_hakemler FROM hakem;
    SELECT asama_id INTO v_asama_id FROM asama ORDER BY sira_no LIMIT 1;

    FOR i IN 1 .. array_length(v_takimlar, 1) - 1 LOOP
        FOR j IN i + 1 .. array_length(v_takimlar, 1) LOOP
            INSERT INTO mac (turnuva_id, asama_id, takim1_id, takim2_id, hakem_id, mac_zamani)
            VALUES (p_turnuva_id, v_asama_id, v_takimlar[i], v_takimlar[j],
                    CASE WHEN v_hakemler IS NULL THEN NULL
                         ELSE v_hakemler[(v_sayac % array_length(v_hakemler, 1)) + 1] END,
                    p_ilk_mac + v_sayac * p_aralik);
            v_sayac := v_sayac + 1;
        END LOOP;
    END LOOP;

    UPDATE turnuva SET durum = 'Devam Ediyor' WHERE turnuva_id = p_turnuva_id;
    RETURN v_sayac;
END;
$$;

-- ---------------------------------------------------------------------
-- 6) mac_sonucu_gir (SAKLI YORDAM): Maç skorunu kaydeder ve maçı
--    "Oynandı" yapar. Kazananın belirlenmesi ve puan tablosunun
--    güncellenmesi tetikleyiciler tarafından yapılır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE mac_sonucu_gir(p_mac_id INT, p_skor1 SMALLINT, p_skor2 SMALLINT)
LANGUAGE plpgsql AS $$
BEGIN
    IF p_skor1 = p_skor2 THEN
        RAISE EXCEPTION 'E-spor maçlarında beraberlik olamaz; skorlar farklı olmalıdır.';
    END IF;

    UPDATE mac
       SET takim1_skor = p_skor1, takim2_skor = p_skor2, durum = 'Oynandı'
     WHERE mac_id = p_mac_id;
```

```
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Maç bulunamadı (no: %).', p_mac_id;
    END IF;
END;
$$;

-- ---------------------------------------------------------------------
-- 7) puan_tablosu: Turnuvanın güncel sıralamasını döndürür.
--    Sıralama ölçütü: puan, skor farkı, galibiyet, takım adı.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION puan_tablosu(p_turnuva_id INT)
RETURNS TABLE (
    sira        INT,
    takim_id    INT,
    takim_adi   VARCHAR,
    oynanan     INT,
    galibiyet   SMALLINT,
    maglubiyet  SMALLINT,
    skor_farki  SMALLINT,
    puan        SMALLINT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT (row_number() OVER (ORDER BY tk.puan DESC, tk.skor_farki DESC,
                                        tk.galibiyet DESC, t.takim_adi))::INT,
           t.takim_id,
           t.takim_adi,
           (tk.galibiyet + tk.maglubiyet)::INT,
           tk.galibiyet,
           tk.maglubiyet,
           tk.skor_farki,
           tk.puan
      FROM turnuva_katilim tk
      JOIN takim t ON t.takim_id = tk.takim_id
     WHERE tk.turnuva_id = p_turnuva_id
     ORDER BY 1;
END;
$$;

-- ---------------------------------------------------------------------
-- 8) oyuncu_istatistik: Bir oyuncunun tüm maçlarındaki toplam
--    istatistiklerini ve KDA oranını ((öldürme + asist) / ölüm) hesaplar.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION oyuncu_istatistik(p_oyuncu_id INT)
RETURNS TABLE (
    mac_sayisi      INT,
    toplam_oldurme  INT,
    toplam_olum     INT,
    toplam_asist    INT,
    kda             NUMERIC,
    mvp_sayisi      INT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT count(*)::INT,
           COALESCE(sum(s.oldurme), 0)::INT,
           COALESCE(sum(s.olum), 0)::INT,
           COALESCE(sum(s.asist), 0)::INT,
           round(COALESCE(sum(s.oldurme + s.asist), 0)::NUMERIC
                 / GREATEST(COALESCE(sum(s.olum), 0), 1), 2),
           count(*) FILTER (WHERE s.mvp)::INT
      FROM oyuncu_mac_istatistik s
     WHERE s.oyuncu_id = p_oyuncu_id;
END;
$$;

-- ---------------------------------------------------------------------
-- 9) oyuncu_transfer (SAKLI YORDAM): Oyuncunun aktif kadro kaydını
--    bugünün tarihiyle kapatır ve (verilmişse) yeni takıma ekler.
--    p_yeni_takim_id NULL ise oyuncu serbest kalır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE oyuncu_transfer(
    p_oyuncu_id      INT,
    p_yeni_takim_id  INT,
```

```
    p_rol            VARCHAR DEFAULT 'Oyuncu'
)
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM oyuncu WHERE kisi_id = p_oyuncu_id) THEN
        RAISE EXCEPTION 'Oyuncu bulunamadı.';
    END IF;

    IF EXISTS (SELECT 1 FROM takim_oyuncu
                WHERE oyuncu_id = p_oyuncu_id AND ayrilma_tarihi IS NULL
                  AND takim_id = p_yeni_takim_id) THEN
        RAISE EXCEPTION 'Oyuncu zaten bu takımın kadrosunda.';
    END IF;

    UPDATE takim_oyuncu
       SET ayrilma_tarihi = current_date, kaptan = FALSE
     WHERE oyuncu_id = p_oyuncu_id AND ayrilma_tarihi IS NULL;

    IF p_yeni_takim_id IS NOT NULL THEN
        INSERT INTO takim_oyuncu (takim_id, oyuncu_id, rol)
        VALUES (p_yeni_takim_id, p_oyuncu_id, COALESCE(NULLIF(p_rol, ''), 'Oyuncu'));
    END IF;
END;
$$;
```

03_tetikleyiciler.sql

```
-- ======================================================================
--  E-SPOR TURNUVA YÖNETİM SİSTEMİ
--  03 - TETİKLEYİCİLER (TRIGGER) VE TETİKLEYİCİ FONKSİYONLARI
-- ======================================================================

-- ----------------------------------------------------------------------
-- T1) Alt tip tutarlılığı ve yaş kontrolü
--     oyuncu / koc / hakem tablolarına eklenen kişinin kisi_turu değeri
--     tabloyla uyumlu olmalıdır. Oyuncular en az 16, koç ve hakemler en
--     az 18 yaşında olmalıdır.
-- ----------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_kisi_alt_tip_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_tur      CHAR(1);
    v_dogum    DATE;
    v_beklenen CHAR(1);
    v_min_yas  INT;
BEGIN
    SELECT kisi_turu, dogum_tarihi INTO v_tur, v_dogum
      FROM kisi WHERE kisi_id = NEW.kisi_id;

    IF TG_TABLE_NAME = 'oyuncu' THEN
        v_beklenen := 'O'; v_min_yas := 16;
    ELSIF TG_TABLE_NAME = 'koc' THEN
        v_beklenen := 'K'; v_min_yas := 18;
    ELSE
        v_beklenen := 'H'; v_min_yas := 18;
    END IF;

    IF v_tur IS DISTINCT FROM v_beklenen THEN
        RAISE EXCEPTION 'Kişi türü uyumsuz: % tablosuna yalnızca türü "%" olan kişiler eklenebilir.',
            TG_TABLE_NAME, v_beklenen;
    END IF;

    IF date_part('year', age(v_dogum)) < v_min_yas THEN
        RAISE EXCEPTION '% kaydı için en az % yaşında olmak gerekir.', initcap(TG_TABLE_NAME), v_min_yas;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_oyuncu_alt_tip BEFORE INSERT ON oyuncu
    FOR EACH ROW EXECUTE FUNCTION trg_kisi_alt_tip_kontrol();
CREATE TRIGGER trg_koc_alt_tip BEFORE INSERT ON koc
    FOR EACH ROW EXECUTE FUNCTION trg_kisi_alt_tip_kontrol();
CREATE TRIGGER trg_hakem_alt_tip BEFORE INSERT ON hakem
```

```
    FOR EACH ROW EXECUTE FUNCTION trg_kisi_alt_tip_kontrol();

-- ---------------------------------------------------------------------
-- T2) Kadro kontrolü
--     Bir takımın aktif kadrosu, oyunun takım boyutunun en fazla 2 fazlası
--     (yedekler) kadar olabilir. Bir takımın tek kaptanı olur: yeni kaptan
--     atanınca eski kaptanın kaptanlığı otomatik olarak kaldırılır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_kadro_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_limit  INT;
    v_aktif  INT;
BEGIN
    IF NEW.ayrilma_tarihi IS NOT NULL THEN
        NEW.kaptan := FALSE;
        RETURN NEW;
    END IF;

    SELECT o.takim_boyutu + 2 INTO v_limit
      FROM takim t JOIN oyun o ON o.oyun_id = t.oyun_id
     WHERE t.takim_id = NEW.takim_id;

    SELECT count(*) INTO v_aktif
      FROM takim_oyuncu
     WHERE takim_id = NEW.takim_id AND ayrilma_tarihi IS NULL
       AND kadro_id <> NEW.kadro_id;

    IF v_aktif >= v_limit THEN
        RAISE EXCEPTION 'Takım kadrosu dolu: bu oyun için en fazla % aktif oyuncu olabilir.', v_limit;
    END IF;

    IF NEW.kaptan THEN
        UPDATE takim_oyuncu SET kaptan = FALSE
         WHERE takim_id = NEW.takim_id AND kaptan AND kadro_id <> NEW.kadro_id;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_kadro_kontrol BEFORE INSERT OR UPDATE ON takim_oyuncu
    FOR EACH ROW EXECUTE FUNCTION trg_kadro_kontrol();

-- ---------------------------------------------------------------------
-- T3) Turnuva kayıt kontrolü
--     Takım; yalnızca kendi oyununa ait, kaydı açık ve kontenjanı dolmamış
--     turnuvalara kaydolabilir. Kayıt için takımın aktif kadrosunda en az
--     oyunun takım boyutu kadar oyuncu bulunmalıdır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_katilim_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_turnuva  turnuva%ROWTYPE;
    v_takim    takim%ROWTYPE;
    v_boyut    SMALLINT;
    v_kayitli  INT;
    v_kadro    INT;
BEGIN
    SELECT * INTO v_turnuva FROM turnuva WHERE turnuva_id = NEW.turnuva_id;
    SELECT * INTO v_takim   FROM takim   WHERE takim_id   = NEW.takim_id;

    IF v_turnuva.durum <> 'Kayıt Açık' THEN
        RAISE EXCEPTION 'Bu turnuvanın kaydı kapalı (durum: %).', v_turnuva.durum;
    END IF;

    IF v_turnuva.oyun_id <> v_takim.oyun_id THEN
        RAISE EXCEPTION '% takımı bu turnuvanın oyununda yarışmıyor.', v_takim.takim_adi;
    END IF;

    SELECT count(*) INTO v_kayitli FROM turnuva_katilim WHERE turnuva_id = NEW.turnuva_id;
    IF v_kayitli >= v_turnuva.max_takim THEN
        RAISE EXCEPTION 'Turnuva kontenjanı dolu (en fazla % takım).', v_turnuva.max_takim;
    END IF;

    SELECT takim_boyutu INTO v_boyut FROM oyun WHERE oyun_id = v_takim.oyun_id;
    SELECT count(*) INTO v_kadro
```

```
      FROM takim_oyuncu WHERE takim_id = NEW.takim_id AND ayrilma_tarihi IS NULL;
    IF v_kadro < v_boyut THEN
        RAISE EXCEPTION '% takımının aktif kadrosu yetersiz (% / % oyuncu).',
            v_takim.takim_adi, v_kadro, v_boyut;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_katilim_kontrol BEFORE INSERT ON turnuva_katilim
    FOR EACH ROW EXECUTE FUNCTION trg_katilim_kontrol();

-- ---------------------------------------------------------------------
-- T4) Maç kontrolü ve kazananın belirlenmesi
--     Maçtaki iki takım da turnuvaya kayıtlı olmalıdır. Sonuç yalnızca
--     "Devam Ediyor" durumundaki turnuvalarda girilebilir ve sonucu girilmiş
--     maç değiştirilemez. Skor girildiğinde kazanan otomatik atanır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_mac_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_durum VARCHAR;
BEGIN
    IF TG_OP = 'UPDATE' AND OLD.durum = 'Oynandı' THEN
        IF NEW.takim1_skor IS DISTINCT FROM OLD.takim1_skor
           OR NEW.takim2_skor IS DISTINCT FROM OLD.takim2_skor
           OR NEW.durum <> OLD.durum
           OR NEW.takim1_id <> OLD.takim1_id OR NEW.takim2_id <> OLD.takim2_id THEN
            RAISE EXCEPTION 'Sonucu girilmiş bir maçın skoru, durumu veya takımları değiştirilemez.';
        END IF;
    END IF;

    IF (SELECT count(*) FROM turnuva_katilim
         WHERE turnuva_id = NEW.turnuva_id
           AND takim_id IN (NEW.takim1_id, NEW.takim2_id)) < 2 THEN
        RAISE EXCEPTION 'Maçtaki iki takım da bu turnuvaya kayıtlı olmalıdır.';
    END IF;

    IF NEW.durum = 'Oynandı' THEN
        SELECT durum INTO v_durum FROM turnuva WHERE turnuva_id = NEW.turnuva_id;
        IF v_durum <> 'Devam Ediyor' THEN
            RAISE EXCEPTION 'Maç sonucu yalnızca devam eden turnuvalarda girilebilir.';
        END IF;
        NEW.kazanan_id := CASE WHEN NEW.takim1_skor > NEW.takim2_skor
                               THEN NEW.takim1_id ELSE NEW.takim2_id END;
    ELSE
        NEW.kazanan_id := NULL;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_mac_kontrol BEFORE INSERT OR UPDATE ON mac
    FOR EACH ROW EXECUTE FUNCTION trg_mac_kontrol();

-- ---------------------------------------------------------------------
-- T5) Puan tablosunun güncellenmesi
--     Maç "Oynandı" durumuna geçtiğinde kazanan takıma +3 puan ve galibiyet,
--     kaybeden takıma mağlubiyet yazılır; skor farkları güncellenir.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_mac_puan_guncelle()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_kaybeden INT;
    v_fark     INT;
BEGIN
    v_kaybeden := CASE WHEN NEW.kazanan_id = NEW.takim1_id THEN NEW.takim2_id ELSE NEW.takim1_id END;
    v_fark     := abs(NEW.takim1_skor - NEW.takim2_skor);

    UPDATE turnuva_katilim
       SET galibiyet = galibiyet + 1, puan = puan + 3, skor_farki = skor_farki + v_fark
     WHERE turnuva_id = NEW.turnuva_id AND takim_id = NEW.kazanan_id;

    UPDATE turnuva_katilim
       SET maglubiyet = maglubiyet + 1, skor_farki = skor_farki - v_fark
```

```
     WHERE turnuva_id = NEW.turnuva_id AND takim_id = v_kaybeden;

    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_mac_puan_guncelle AFTER UPDATE OF durum ON mac
    FOR EACH ROW
    WHEN (OLD.durum <> 'Oynandı' AND NEW.durum = 'Oynandı')
    EXECUTE FUNCTION trg_mac_puan_guncelle();

-- ---------------------------------------------------------------------
-- T6) Oyuncu maç istatistiği kontrolü
--     İstatistik yalnızca oynanmış maçlar için ve maç tarihinde maçtaki
--     takımlardan birinin kadrosunda olan oyuncular için girilebilir.
--     Oyuncunun takımı otomatik bulunur. Bir maçta tek MVP olur.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_istatistik_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_mac    mac%ROWTYPE;
    v_takim  INT;
BEGIN
    SELECT * INTO v_mac FROM mac WHERE mac_id = NEW.mac_id;

    IF v_mac.durum <> 'Oynandı' THEN
        RAISE EXCEPTION 'İstatistik yalnızca oynanmış maçlar için girilebilir.';
    END IF;

    SELECT takim_id INTO v_takim
      FROM takim_oyuncu
     WHERE oyuncu_id = NEW.oyuncu_id
       AND takim_id IN (v_mac.takim1_id, v_mac.takim2_id)
       AND katilma_tarihi <= v_mac.mac_zamani::DATE
       AND (ayrilma_tarihi IS NULL OR ayrilma_tarihi >= v_mac.mac_zamani::DATE)
     ORDER BY katilma_tarihi DESC
     LIMIT 1;

    IF v_takim IS NULL THEN
        RAISE EXCEPTION 'Oyuncu, maç tarihinde maçtaki takımlardan hiçbirinin kadrosunda değildi.';
    END IF;

    NEW.takim_id := v_takim;

    IF NEW.mvp THEN
        UPDATE oyuncu_mac_istatistik SET mvp = FALSE
         WHERE mac_id = NEW.mac_id AND mvp AND istatistik_id <> NEW.istatistik_id;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_istatistik_kontrol BEFORE INSERT OR UPDATE ON oyuncu_mac_istatistik
    FOR EACH ROW EXECUTE FUNCTION trg_istatistik_kontrol();

-- ---------------------------------------------------------------------
-- T7) Turnuva durum geçişleri ve ödül dağıtımı
--     Tamamlanan turnuva tekrar açılamaz. Turnuva "Tamamlandı" yapılırken
--     tüm maçlar oynanmış (veya iptal edilmiş) olmalıdır. Turnuva
--     tamamlandığında takımların final sıraları yazılır ve ödüller
--     dereceye göre ilgili takımlara otomatik atanır.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_turnuva_durum()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.durum = 'Tamamlandı' THEN
        RAISE EXCEPTION 'Tamamlanmış bir turnuvanın durumu değiştirilemez.';
    END IF;

    IF NEW.durum = 'Tamamlandı' THEN
        IF NOT EXISTS (SELECT 1 FROM mac WHERE turnuva_id = NEW.turnuva_id) THEN
            RAISE EXCEPTION 'Hiç maç oynanmamış bir turnuva tamamlanamaz.';
        END IF;
        IF EXISTS (SELECT 1 FROM mac WHERE turnuva_id = NEW.turnuva_id AND durum = 'Planlandı') THEN
            RAISE EXCEPTION 'Turnuvada henüz oynanmamış maçlar var.';
        END IF;
```

```
        UPDATE turnuva_katilim tk
           SET sira = pt.sira
          FROM puan_tablosu(NEW.turnuva_id) pt
         WHERE tk.turnuva_id = NEW.turnuva_id AND tk.takim_id = pt.takim_id;

        UPDATE odul o
           SET takim_id = tk.takim_id
          FROM turnuva_katilim tk
         WHERE o.turnuva_id = NEW.turnuva_id
           AND tk.turnuva_id = NEW.turnuva_id
           AND tk.sira = o.derece;
    END IF;

    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_turnuva_durum AFTER UPDATE OF durum ON turnuva
    FOR EACH ROW
    WHEN (OLD.durum IS DISTINCT FROM NEW.durum)
    EXECUTE FUNCTION trg_turnuva_durum();

-- ---------------------------------------------------------------------
-- T8) Devam eden turnuvanın silinmesinin engellenmesi
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_turnuva_silme_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.durum = 'Devam Ediyor' THEN
        RAISE EXCEPTION 'Devam eden bir turnuva silinemez.';
    END IF;
    RETURN OLD;
END;
$$;

CREATE TRIGGER trg_turnuva_silme_kontrol BEFORE DELETE ON turnuva
    FOR EACH ROW EXECUTE FUNCTION trg_turnuva_silme_kontrol();

-- ---------------------------------------------------------------------
-- T9) Ödül havuzu kontrolü
--     Bir turnuvanın ödüllerinin toplamı, ödül havuzunu aşamaz.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_odul_kontrol()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_havuz   NUMERIC;
    v_toplam  NUMERIC;
BEGIN
    SELECT odul_havuzu INTO v_havuz FROM turnuva WHERE turnuva_id = NEW.turnuva_id;

    SELECT COALESCE(sum(tutar), 0) INTO v_toplam
      FROM odul
     WHERE turnuva_id = NEW.turnuva_id AND odul_id <> NEW.odul_id;

    IF v_toplam + NEW.tutar > v_havuz THEN
        RAISE EXCEPTION 'Ödüller toplamı (%) ödül havuzunu (%) aşamaz.', v_toplam + NEW.tutar, v_havuz;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_odul_kontrol BEFORE INSERT OR UPDATE OF tutar ON odul
    FOR EACH ROW EXECUTE FUNCTION trg_odul_kontrol();

-- ---------------------------------------------------------------------
-- T10) İşlem günlüğü (denetim kaydı)
--      kisi, takim ve turnuva tablolarındaki her ekleme, güncelleme ve
--      silme işlemi islem_log tablosuna JSON olarak kaydedilir.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trg_islem_log()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO islem_log (tablo_adi, islem_turu, kayit_bilgisi)
    VALUES (TG_TABLE_NAME, TG_OP,
            CASE WHEN TG_OP = 'DELETE' THEN row_to_json(OLD)::TEXT
```

```
                ELSE row_to_json(NEW)::TEXT END);
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_kisi_log AFTER INSERT OR UPDATE OR DELETE ON kisi
    FOR EACH ROW EXECUTE FUNCTION trg_islem_log();
CREATE TRIGGER trg_takim_log AFTER INSERT OR UPDATE OR DELETE ON takim
    FOR EACH ROW EXECUTE FUNCTION trg_islem_log();
CREATE TRIGGER trg_turnuva_log AFTER INSERT OR UPDATE OR DELETE ON turnuva
    FOR EACH ROW EXECUTE FUNCTION trg_islem_log();
```

04_ornek_veriler.sql

```
-- ======================================================================
--  E-SPOR TURNUVA YÖNETİM SİSTEMİ
--  04 - ÖRNEK VERİLER
--  (01, 02 ve 03 numaralı dosyalar çalıştırıldıktan sonra çalıştırılır.)
-- ======================================================================

-- ----------------------------------------------------------------------
--  Tanım verileri
-- ----------------------------------------------------------------------
INSERT INTO ulke (ulke_adi, ulke_kodu) VALUES
    ('Türkiye', 'TR'), ('Almanya', 'DE'), ('Fransa', 'FR'), ('İsveç', 'SE'),
    ('Güney Kore', 'KR'), ('Danimarka', 'DK'), ('Brezilya', 'BR'), ('Ukrayna', 'UA');

INSERT INTO oyun_turu (tur_adi) VALUES
    ('Taktiksel Nişancı'), ('MOBA'), ('Araç Sporu'), ('Dövüş');

INSERT INTO oyun (oyun_adi, tur_id, yayinci, takim_boyutu) VALUES
    ('Valorant',          1, 'Riot Games',   5),
    ('Counter-Strike 2',  1, 'Valve',        5),
    ('League of Legends', 2, 'Riot Games',   5),
    ('Rocket League',     3, 'Psyonix',      3),
    ('Tekken 8',          4, 'Bandai Namco', 1);

INSERT INTO asama (asama_adi, sira_no) VALUES
    ('Grup Aşaması', 1), ('Çeyrek Final', 2), ('Yarı Final', 3), ('Final', 4);

INSERT INTO sponsor (sponsor_adi, sektor) VALUES
    ('Kuzgun Donanım',  'Bilgisayar Donanımı'),
    ('HızNet Fiber',    'Telekomünikasyon'),
    ('Enerjix',         'İçecek'),
    ('NovaChair',       'Mobilya'),
    ('Atlas Sigorta',   'Sigorta'),
    ('Pixel Kafe',      'Yiyecek');

INSERT INTO mekan (mekan_adi, ulke_id, sehir, adres, kapasite) VALUES
    ('Haliç E-spor Arenası',         1, 'İstanbul', 'Haliç Kongre Merkezi Yanı, Beyoğlu',   2500),
    ('Ankara Dijital Salon',         1, 'Ankara',   'Eskişehir Yolu No: 120, Çankaya',      1200),
    ('İzmir Kordon Gösteri Merkezi', 1, 'İzmir',    'Atatürk Caddesi No: 45, Konak',        1800);

-- ----------------------------------------------------------------------
--  KİŞİLER (üst tip)   1-6: koç,  7-10: hakem,  11-57: oyuncu
-- ----------------------------------------------------------------------
INSERT INTO kisi (kisi_id, ad, soyad, eposta, dogum_tarihi, ulke_id, kisi_turu) VALUES
    ( 1, 'Murat',     'Ersoy',      'murat.ersoy@ornek.com',      '1985-04-12', 1, 'K'),
    ( 2, 'Selim',     'Karaca',     'selim.karaca@ornek.com',     '1988-09-03', 1, 'K'),
    ( 3, 'Gökhan',    'Tekin',      'gokhan.tekin@ornek.com',     '1990-01-27', 1, 'K'),
    ( 4, 'Anders',    'Svensson',   'anders.svensson@ornek.com',  '1983-06-15', 4, 'K'),
    ( 5, 'Stefan',    'Krüger',     'stefan.kruger@ornek.com',    '1986-11-02', 2, 'K'),
    ( 6, 'Jae-won',   'Han',        'jaewon.han@ornek.com',       '1984-03-19', 5, 'K'),

    ( 7, 'Ayşe',      'Demirtaş',   'ayse.demirtas@ornek.com',    '1987-02-08', 1, 'H'),
    ( 8, 'Kemal',     'Uçar',       'kemal.ucar@ornek.com',       '1991-07-30', 1, 'H'),
    ( 9, 'Sofia',     'Andersson',  'sofia.andersson@ornek.com',  '1989-12-11', 4, 'H'),
    (10, 'Pierre',    'Laurent',    'pierre.laurent@ornek.com',   '1982-05-24', 3, 'H'),

    -- Anadolu Kartalları (Valorant)
    (11, 'Emre',      'Yıldız',     'emre.yildiz@ornek.com',      '2001-03-14', 1, 'O'),
    (12, 'Burak',     'Aydın',      'burak.aydin@ornek.com',      '2002-07-22', 1, 'O'),
    (13, 'Mert',      'Kaya',       'mert.kaya@ornek.com',        '2000-11-05', 1, 'O'),
    (14, 'Ozan',      'Çelik',      'ozan.celik@ornek.com',       '2003-01-30', 1, 'O'),
    (15, 'Kaan',      'Demir',      'kaan.demir@ornek.com',       '2001-09-18', 1, 'O'),
```

```
    -- Boğaziçi Kurtları (Valorant)
    (16, 'Can',       'Öztürk',     'can.ozturk@ornek.com',       '2000-02-11', 1, 'O'),
    (17, 'Efe',       'Arslan',     'efe.arslan@ornek.com',       '2002-12-03', 1, 'O'),
    (18, 'Yusuf',     'Koç',        'yusuf.koc@ornek.com',        '2001-06-27', 1, 'O'),
    (19, 'Deniz',     'Kurt',       'deniz.kurt@ornek.com',       '2003-04-15', 1, 'O'),
    (20, 'Barış',     'Aksoy',      'baris.aksoy@ornek.com',      '2002-08-08', 1, 'O'),
    -- Kapadokya Ejderleri (Valorant)
    (21, 'Furkan',    'Yılmaz',     'furkan.yilmaz@ornek.com',    '2001-10-10', 1, 'O'),
    (22, 'Serkan',    'Doğan',      'serkan.dogan@ornek.com',     '2000-05-25', 1, 'O'),
    (23, 'Tolga',     'Erdem',      'tolga.erdem@ornek.com',      '1999-12-01', 1, 'O'),
    (24, 'Hakan',     'Güneş',      'hakan.gunes@ornek.com',      '2002-03-03', 1, 'O'),
    (25, 'Cem',       'Özdemir',    'cem.ozdemir@ornek.com',      '2003-09-09', 1, 'O'),
    -- Nordic Frost (Valorant)
    (26, 'Erik',      'Lindqvist',  'erik.lindqvist@ornek.com',   '2000-01-17', 4, 'O'),
    (27, 'Oskar',     'Nilsson',    'oskar.nilsson@ornek.com',    '2001-04-04', 4, 'O'),
    (28, 'Lukas',     'Berg',       'lukas.berg@ornek.com',       '1999-08-19', 4, 'O'),
    (29, 'Viktor',    'Holm',       'viktor.holm@ornek.com',      '2002-11-11', 4, 'O'),
    (30, 'Axel',      'Strand',     'axel.strand@ornek.com',      '2003-02-28', 4, 'O'),
    -- Berlin Phantoms (Valorant)
    (31, 'Jonas',     'Becker',     'jonas.becker@ornek.com',     '2000-06-06', 2, 'O'),
    (32, 'Felix',     'Wagner',     'felix.wagner@ornek.com',     '2001-12-12', 2, 'O'),
    (33, 'Leon',      'Schulz',     'leon.schulz@ornek.com',      '2002-09-29', 2, 'O'),
    (34, 'Maximilian','Hoffmann',   'max.hoffmann@ornek.com',     '1999-03-21', 2, 'O'),
    (35, 'Paul',      'Richter',    'paul.richter@ornek.com',     '2003-07-07', 2, 'O'),
    -- Seoul Tigers (Valorant)
    (36, 'Min-jun',   'Kim',        'minjun.kim@ornek.com',       '2001-05-05', 5, 'O'),
    (37, 'Ji-ho',     'Park',       'jiho.park@ornek.com',        '2002-02-14', 5, 'O'),
    (38, 'Seo-jun',   'Lee',        'seojun.lee@ornek.com',       '2000-10-20', 5, 'O'),
    (39, 'Do-yun',    'Choi',       'doyun.choi@ornek.com',       '2003-08-31', 5, 'O'),
    (40, 'Ha-jun',    'Jung',       'hajun.jung@ornek.com',       '2002-06-16', 5, 'O'),
    -- Yedek oyuncular
    (41, 'Arda',      'Şahin',      'arda.sahin@ornek.com',       '2004-05-09', 1, 'O'),
    (42, 'Onur',      'Polat',      'onur.polat@ornek.com',       '2005-01-21', 1, 'O'),
    -- Turbo Turkuaz (Rocket League)
    (43, 'Alp',       'Turan',      'alp.turan@ornek.com',        '2002-04-12', 1, 'O'),
    (44, 'Sinan',     'Bulut',      'sinan.bulut@ornek.com',      '2003-06-02', 1, 'O'),
    (45, 'Eren',      'Yavuz',      'eren.yavuz@ornek.com',       '2004-02-19', 1, 'O'),
    -- Ege Rüzgarı (Rocket League)
    (46, 'Umut',      'Keskin',     'umut.keskin@ornek.com',      '2001-08-23', 1, 'O'),
    (47, 'Batuhan',   'Sarı',       'batuhan.sari@ornek.com',     '2003-11-30', 1, 'O'),
    (48, 'Mustafa',   'Ateş',       'mustafa.ates@ornek.com',     '2002-01-08', 1, 'O'),
    -- Paris Boost (Rocket League)
    (49, 'Hugo',      'Martin',     'hugo.martin@ornek.com',      '2001-09-14', 3, 'O'),
    (50, 'Louis',     'Bernard',    'louis.bernard@ornek.com',    '2002-05-27', 3, 'O'),
    (51, 'Théo',      'Dubois',     'theo.dubois@ornek.com',      '2003-03-16', 3, 'O'),
    -- Copenhagen Wings (Rocket League)
    (52, 'Mikkel',    'Jensen',     'mikkel.jensen@ornek.com',    '2000-12-24', 6, 'O'),
    (53, 'Frederik',  'Nielsen',    'frederik.nielsen@ornek.com', '2002-10-05', 6, 'O'),
    (54, 'Rasmus',    'Hansen',     'rasmus.hansen@ornek.com',    '2003-07-19', 6, 'O'),
    -- Serbest oyuncular
    (55, 'Selin',     'Aydemir',    'selin.aydemir@ornek.com',    '2004-09-12', 1, 'O'),
    (56, 'Zeynep',    'Kılıç',      'zeynep.kilic@ornek.com',     '2005-03-08', 1, 'O'),
    (57, 'Lars',      'Eriksson',   'lars.eriksson@ornek.com',    '2001-11-25', 4, 'O');

SELECT setval('kisi_kisi_id_seq', (SELECT max(kisi_id) FROM kisi));

-- ---------------------------------------------------------------------
--  ALT TİPLER: koç, hakem, oyuncu
-- ---------------------------------------------------------------------
INSERT INTO koc (kisi_id, deneyim_yili) VALUES
    (1, 8), (2, 6), (3, 5), (4, 10), (5, 7), (6, 9);

INSERT INTO hakem (kisi_id, lisans_no, seviye) VALUES
    (7,  'TR-H-1001', 'Uluslararası'),
    (8,  'TR-H-1002', 'Ulusal'),
    (9,  'SE-H-2201', 'Uluslararası'),
    (10, 'FR-H-3105', 'Uluslararası');

INSERT INTO oyuncu (kisi_id, takma_ad, elo_puani) VALUES
    (11, 'Kartal', 2150), (12, 'Sis', 2080), (13, 'Kalkan', 2010), (14, 'Radar', 1990), (15, 'Volt', 1970),
    (16, 'Bozkurt', 2120), (17, 'Duman', 2040), (18, 'Tuzak', 1995), (19, 'Pusula', 1960), (20, 'Kıvılcım', 1940),
    (21, 'Ejder', 2060), (22, 'Bulut', 1980), (23, 'Kale', 1950), (24, 'Şimşek', 1930), (25, 'Fırtına', 1900),
    (26, 'Frostbite', 2140), (27, 'Aurora', 2070), (28, 'Glacier', 2020), (29, 'Tundra', 1985), (30, 'Polar', 1955),
    (31, 'Phantom', 2090), (32, 'Specter', 2030), (33, 'Wraith', 1975), (34, 'Haze', 1945), (35, 'Ember', 1915),
    (36, 'Tiger', 2180), (37, 'Lotus', 2100), (38, 'Haneul', 2050), (39, 'Byeol', 2000), (40, 'Raon', 1965),
    (41, 'Gölge', 1850), (42, 'Yankı', 1820),
    (43, 'Nitro', 1720), (44, 'Drift', 1680), (45, 'Roket', 1650),
```

```
    (46, 'Meltem', 1700), (47, 'Dalga', 1660), (48, 'Poyraz', 1640),
    (49, 'Boost', 1750), (50, 'Flip', 1710), (51, 'Aerial', 1690),
    (52, 'Wingman', 1730), (53, 'Kickoff', 1700), (54, 'Dribble', 1670),
    (55, 'Valkür', 1880), (56, 'Nova', 1790), (57, 'Viking', 1760);

-- ---------------------------------------------------------------------
--  TAKIMLAR
-- ---------------------------------------------------------------------
INSERT INTO takim (takim_id, takim_adi, kisa_ad, oyun_id, ulke_id, koc_id, kurulus_tarihi) VALUES
    ( 1, 'Anadolu Kartalları',   'AKR', 1, 1, 1,    '2021-03-01'),
    ( 2, 'Boğaziçi Kurtları',    'BGK', 1, 1, 2,    '2020-09-15'),
    ( 3, 'Kapadokya Ejderleri',  'KPE', 1, 1, 3,    '2022-01-20'),
    ( 4, 'Nordic Frost',         'NFR', 1, 4, 4,    '2019-06-10'),
    ( 5, 'Berlin Phantoms',      'BPH', 1, 2, 5,    '2020-02-02'),
    ( 6, 'Seoul Tigers',         'STG', 1, 5, 6,    '2018-11-11'),
    ( 7, 'Turbo Turkuaz',        'TRB', 4, 1, NULL, '2023-04-04'),
    ( 8, 'Ege Rüzgarı',          'EGR', 4, 1, NULL, '2022-08-17'),
    ( 9, 'Paris Boost',          'PBO', 4, 3, NULL, '2021-05-05'),
    (10, 'Copenhagen Wings',     'CPW', 4, 6, NULL, '2022-12-01');

SELECT setval('takim_takim_id_seq', (SELECT max(takim_id) FROM takim));

-- ---------------------------------------------------------------------
--  KADROLAR (geçmiş kayıtlar dahil)
-- ---------------------------------------------------------------------
-- Geçmiş (sona ermiş) kadro kayıtları
INSERT INTO takim_oyuncu (takim_id, oyuncu_id, rol, kaptan, katilma_tarihi, ayrilma_tarihi) VALUES
    (4, 57, 'Düellocu', FALSE, '2023-01-10', '2025-12-31'),
    (3, 41, 'Nöbetçi',  FALSE, '2024-02-01', '2025-06-30');

-- Aktif kadrolar
INSERT INTO takim_oyuncu (takim_id, oyuncu_id, rol, kaptan, katilma_tarihi) VALUES
    (1, 11, 'Düellocu',   TRUE,  '2024-01-15'), (1, 12, 'Kontrolcü', FALSE, '2024-01-15'),
    (1, 13, 'Nöbetçi',    FALSE, '2024-06-01'), (1, 14, 'Öncü',      FALSE, '2025-01-10'),
    (1, 15, 'Esnek',      FALSE, '2025-01-10'), (1, 41, 'Yedek',     FALSE, '2025-07-15'),

    (2, 16, 'Düellocu',   TRUE,  '2023-09-01'), (2, 17, 'Kontrolcü', FALSE, '2023-09-01'),
    (2, 18, 'Nöbetçi',    FALSE, '2024-03-12'), (2, 19, 'Öncü',      FALSE, '2024-03-12'),
    (2, 20, 'Esnek',      FALSE, '2025-02-20'), (2, 42, 'Yedek',     FALSE, '2026-01-05'),

    (3, 21, 'Düellocu',   TRUE,  '2024-02-01'), (3, 22, 'Kontrolcü', FALSE, '2024-02-01'),
    (3, 23, 'Nöbetçi',    FALSE, '2025-07-01'), (3, 24, 'Öncü',      FALSE, '2024-02-01'),
    (3, 25, 'Esnek',      FALSE, '2025-03-15'),

    (4, 26, 'Düellocu',   TRUE,  '2023-01-10'), (4, 27, 'Kontrolcü', FALSE, '2023-01-10'),
    (4, 28, 'Nöbetçi',    FALSE, '2023-01-10'), (4, 29, 'Öncü',      FALSE, '2024-05-05'),
    (4, 30, 'Esnek',      FALSE, '2026-01-02'),

    (5, 31, 'Düellocu',   TRUE,  '2023-03-03'), (5, 32, 'Kontrolcü', FALSE, '2023-03-03'),
    (5, 33, 'Nöbetçi',    FALSE, '2024-08-08'), (5, 34, 'Öncü',      FALSE, '2023-03-03'),
    (5, 35, 'Esnek',      FALSE, '2025-04-01'),

    (6, 36, 'Düellocu',   TRUE,  '2022-12-01'), (6, 37, 'Kontrolcü', FALSE, '2022-12-01'),
    (6, 38, 'Nöbetçi',    FALSE, '2023-06-15'), (6, 39, 'Öncü',      FALSE, '2024-01-20'),
    (6, 40, 'Esnek',      FALSE, '2024-01-20'),

    (7, 43, 'Hücum',      TRUE,  '2023-04-04'), (7, 44, 'Orta Saha', FALSE, '2023-04-04'),
    (7, 45, 'Defans',     FALSE, '2024-09-01'),
    (8, 46, 'Hücum',      TRUE,  '2022-08-17'), (8, 47, 'Orta Saha', FALSE, '2023-02-01'),
    (8, 48, 'Defans',     FALSE, '2023-02-01'),
    (9, 49, 'Hücum',      TRUE,  '2021-05-05'), (9, 50, 'Orta Saha', FALSE, '2022-01-01'),
    (9, 51, 'Defans',     FALSE, '2023-06-01'),
    (10, 52, 'Hücum',     TRUE,  '2022-12-01'), (10, 53, 'Orta Saha', FALSE, '2022-12-01'),
    (10, 54, 'Defans',    FALSE, '2024-03-03');

-- ---------------------------------------------------------------------
--  SPONSORLUKLAR
-- ---------------------------------------------------------------------
INSERT INTO takim_sponsor (takim_id, sponsor_id, baslangic, bitis, yillik_tutar) VALUES
    (1, 1, '2025-01-01', '2027-12-31', 1200000),
    (1, 3, '2026-01-01', '2026-12-31',  450000),
    (2, 2, '2024-06-01', '2026-12-31',  900000),
    (3, 6, '2025-03-01', '2027-02-28',  300000),
    (4, 4, '2024-01-01', '2026-12-31',  650000),
    (6, 1, '2025-06-01', '2027-05-31',  800000),
    (7, 5, '2025-09-01', '2026-08-31',  150000),
    (8, 3, '2026-02-01', '2027-01-31',  120000);
```

```
-- ---------------------------------------------------------------------
--  TURNUVALAR (kalıtım: turnuva_olustur fonksiyonu ile alt tiplere ekleme)
-- ---------------------------------------------------------------------
SELECT turnuva_olustur('Anadolu Valorant Kupası 2026', 1, '2026-09-05', '2026-09-10', 250000, 4::SMALLINT,
                       'O', 'Twitch', 'Avrupa (İstanbul)');
SELECT turnuva_olustur('İstanbul Valorant Masters', 1, '2026-10-15', '2026-10-30', 500000, 6::SMALLINT,
                       'S', NULL, NULL, 1, 350);
SELECT turnuva_olustur('Rocket League Kış Ligi', 4, '2026-11-10', '2026-11-30', 60000, 8::SMALLINT,
                       'O', 'YouTube', 'Avrupa (Frankfurt)');
SELECT turnuva_olustur('Ankara CS2 Bahar Turnuvası', 2, '2027-03-20', '2027-03-22', 150000, 8::SMALLINT,
                       'S', NULL, NULL, 2, 200);

-- Ödüller
INSERT INTO odul (turnuva_id, derece, tutar) VALUES
    (1, 1, 125000), (1, 2, 75000),  (1, 3, 50000),
    (2, 1, 250000), (2, 2, 150000), (2, 3, 100000),
    (3, 1, 35000),  (3, 2, 25000),
    (4, 1, 90000),  (4, 2, 60000);

-- Turnuva kayıtları (tetikleyici: oyun, kontenjan ve kadro kontrolü)
INSERT INTO turnuva_katilim (turnuva_id, takim_id, kayit_tarihi) VALUES
    (1, 1, '2026-08-01'), (1, 2, '2026-08-02'), (1, 3, '2026-08-03'), (1, 4, '2026-08-05'),
    (2, 1, '2026-09-20'), (2, 2, '2026-09-21'), (2, 3, '2026-09-21'),
    (2, 4, '2026-09-25'), (2, 5, '2026-09-26'), (2, 6, '2026-09-28'),
    (3, 7, '2026-10-01'), (3, 8, '2026-10-03'), (3, 9, '2026-10-10');

-- ---------------------------------------------------------------------
--  1. TURNUVA: fikstür, sonuçlar, tamamlama (ödüller tetikleyici ile dağıtılır)
-- ---------------------------------------------------------------------
SELECT fikstur_olustur(1, '2026-09-05 18:00', '1 day');

CALL mac_sonucu_gir(1, 2::SMALLINT, 0::SMALLINT);   -- AKR - BGK
CALL mac_sonucu_gir(2, 2::SMALLINT, 0::SMALLINT);   -- AKR - KPE
CALL mac_sonucu_gir(3, 1::SMALLINT, 2::SMALLINT);   -- AKR - NFR
CALL mac_sonucu_gir(4, 2::SMALLINT, 1::SMALLINT);   -- BGK - KPE
CALL mac_sonucu_gir(5, 0::SMALLINT, 2::SMALLINT);   -- BGK - NFR
CALL mac_sonucu_gir(6, 2::SMALLINT, 1::SMALLINT);   -- KPE - NFR

-- ---------------------------------------------------------------------
--  2. TURNUVA: fikstür ve oynanmış ilk maçlar (turnuva devam ediyor)
-- ---------------------------------------------------------------------
SELECT fikstur_olustur(2, '2026-10-15 13:00', '1 day');

CALL mac_sonucu_gir(7,  2::SMALLINT, 1::SMALLINT);  -- AKR - BGK
CALL mac_sonucu_gir(8,  0::SMALLINT, 2::SMALLINT);  -- AKR - KPE
CALL mac_sonucu_gir(9,  2::SMALLINT, 0::SMALLINT);  -- AKR - NFR
CALL mac_sonucu_gir(10, 2::SMALLINT, 1::SMALLINT);  -- AKR - BPH
CALL mac_sonucu_gir(11, 1::SMALLINT, 2::SMALLINT);  -- AKR - STG
CALL mac_sonucu_gir(12, 2::SMALLINT, 0::SMALLINT);  -- BGK - KPE

-- ---------------------------------------------------------------------
--  OYUNCU MAÇ İSTATİSTİKLERİ (oynanmış maçlarda sahadaki oyuncular için)
--  Değerler örnek olsun diye oyuncu ve maç numarasından türetilmiştir.
-- ---------------------------------------------------------------------
INSERT INTO oyuncu_mac_istatistik (mac_id, oyuncu_id, takim_id, oldurme, olum, asist)
SELECT m.mac_id,
       ko.oyuncu_id,
       ko.takim_id,
       10 + (ko.oyuncu_id * 7 + m.mac_id * 13) % 20 + CASE WHEN ko.takim_id = m.kazanan_id THEN 6 ELSE 0 END,
       12 + (ko.oyuncu_id * 11 + m.mac_id * 5) % 14 - CASE WHEN ko.takim_id = m.kazanan_id THEN 3 ELSE 0 END,
       3  + (ko.oyuncu_id * 3 + m.mac_id * 17) % 12
  FROM mac m
  JOIN takim_oyuncu ko
    ON ko.takim_id IN (m.takim1_id, m.takim2_id)
   AND ko.ayrilma_tarihi IS NULL
   AND ko.rol <> 'Yedek'
 WHERE m.durum = 'Oynandı';

-- Her maçın MVP'si: kazanan takımda en çok öldürme yapan oyuncu
UPDATE oyuncu_mac_istatistik s
   SET mvp = TRUE
 WHERE s.istatistik_id IN (
       SELECT DISTINCT ON (s2.mac_id) s2.istatistik_id
         FROM oyuncu_mac_istatistik s2
         JOIN mac m ON m.mac_id = s2.mac_id AND m.kazanan_id = s2.takim_id
        ORDER BY s2.mac_id, s2.oldurme DESC, s2.asist DESC);
```

```
-- 1. turnuvayı tamamla: sıralama ve ödül dağıtımı tetikleyici ile yapılır
UPDATE turnuva SET durum = 'Tamamlandı' WHERE turnuva_id = 1;
```

d) Saklı Yordamlar / Fonksiyonlar ve Tetikleyiciler

Fonksiyonlar ve Saklı Yordamlar (9 adet)

Tetikleyiciler için tanımlanan fonksiyonlar bu listeye dahil değildir. Listedeki tüm fonksiyon ve yordamlar web uygulamasında kullanılmaktadır.

1. **`oyuncu_ekle(ad, soyad, eposta, dogum, ulke_id, takma_ad, elo) → INT`**
   Fonksiyon

   Kalıtım hiyerarşisine uygun olarak önce üst tip *kisi* tablosuna (kisi_turu = 'O'), ardından alt tip *oyuncu* tablosuna kayıt ekler ve yeni kişinin numarasını döndürür. İki ekleme tek işlemde yapılır.

   Uygulamada: Oyuncular → Yeni Oyuncu formu

2. **`oyuncu_guncelle(kisi_id, ad, soyad, eposta, dogum, ulke_id, takma_ad, elo)`**
   Saklı yordam (PROCEDURE)

   Oyuncunun üst tip (kisi) ve alt tip (oyuncu) tablolarındaki bilgilerini birlikte günceller; oyuncu bulunamazsa hata verir.

   Uygulamada: Oyuncular → Düzenle formu

3. **`oyuncu_ara(aranan) → TABLE`**
   Fonksiyon

   Takma ad, ad, soyad veya takım adına göre büyük/küçük harf duyarsız arama yapar. Oyuncunun ülkesini, yaşını, aktif takımını ve oyununu ELO puanına göre sıralı döndürür. Boş metin tüm oyuncuları getirir.

   Uygulamada: Oyuncular → Arama; Ana sayfa → En yüksek ELO listesi

4. **`turnuva_olustur(adi, oyun_id, baslangic, bitis, odul, max_takim, tur, platform, bolge, mekan_id, bilet) → INT`**
   Fonksiyon

   Turnuvayı üst tip *turnuva* tablosuna ekler; türüne göre *online_turnuva* veya *salon_turnuva* alt tablosuna da kayıt açar. Türe göre zorunlu bilgiler eksikse hata verir.

   Uygulamada: Turnuvalar → Yeni Turnuva formu

5. **`fikstur_olustur(turnuva_id, ilk_mac, aralik) → INT`**
   Fonksiyon

   Turnuvaya kayıtlı takımlar için herkesin herkesle oynadığı lig usulü maçları oluşturur, hakemleri sırayla atar, maç zamanlarını verilen aralıkla planlar ve turnuvayı "Devam Ediyor" durumuna getirir. Oluşan maç sayısını döndürür.

   Uygulamada: Turnuva detayı → Fikstürü Oluştur

6. **`mac_sonucu_gir(mac_id, skor1, skor2)`**
   Saklı yordam (PROCEDURE)

   Maç skorunu kaydeder ve maçı "Oynandı" yapar; beraberlik skorunu reddeder. Kazananın belirlenmesi ve puan tablosunun güncellenmesi bu güncellemeyle tetiklenen tetikleyicilerce yapılır.

   Uygulamada: Turnuva detayı ve Maç detayı → Skor girişi

7. **`puan_tablosu(turnuva_id) → TABLE`**
   Fonksiyon

   Turnuvanın sıralamasını puan, skor farkı, galibiyet ve takım adına göre hesaplayarak sıra numarasıyla döndürür. Turnuva tamamlanırken ödül dağıtan tetikleyici de bu fonksiyonu kullanır.

   Uygulamada: Turnuva detayı → Puan Tablosu

8. **`oyuncu_istatistik(oyuncu_id) → TABLE`**
   Fonksiyon

   Oyuncunun oynadığı maç sayısını, toplam öldürme / ölüm / asist değerlerini, KDA oranını ((öldürme + asist) / ölüm) ve MVP sayısını hesaplar.

   Uygulamada: Oyuncu detayı → İstatistik kartları

9. **`oyuncu_transfer(oyuncu_id, yeni_takim_id, rol)`**
   Saklı yordam (PROCEDURE)

   Oyuncunun aktif kadro kaydını bugünün tarihiyle kapatır (geçmiş korunur) ve yeni takım verilmişse o takımın kadrosuna ekler; yeni takım boşsa oyuncu serbest kalır.

   Uygulamada: Oyuncu detayı → Transfer; Takım detayı → Kadroya ekle / Kadrodan çıkar; Yeni Oyuncu formu (takım seçimi)

Tetikleyiciler (10 grup, 14 tetikleyici)

1. **`trg_oyuncu_alt_tip, trg_koc_alt_tip, trg_hakem_alt_tip`**
   BEFORE INSERT — oyuncu, koc, hakem · Fonksiyon: trg_kisi_alt_tip_kontrol()

   Alt tip tablosuna eklenen kişinin *kisi_turu* değerinin tabloyla uyumlu olduğunu denetler (kalıtım tutarlılığı). Oyuncuların en az 16, koç ve hakemlerin en az 18 yaşında olmasını zorunlu kılar.

2. **`trg_kadro_kontrol`**
   BEFORE INSERT OR UPDATE — takim_oyuncu · Fonksiyon: trg_kadro_kontrol()

   Takımın aktif kadrosunun takım boyutu + 2 sınırını aşmasını engeller. Bir oyuncu kaptan yapıldığında takımdaki diğer oyuncuların kaptanlığını otomatik kaldırır; kadrodan ayrılan oyuncunun kaptanlığını siler.

3. **`trg_katilim_kontrol`**
   BEFORE INSERT — turnuva_katilim · Fonksiyon: trg_katilim_kontrol()

   Turnuva kaydında; kaydın açık olmasını, takımın turnuvanın oyununda yarışmasını, kontenjanın dolmamış olmasını ve takımın aktif kadrosunun yeterli olmasını denetler.

4. **`trg_mac_kontrol`**
   BEFORE INSERT OR UPDATE — mac · Fonksiyon: trg_mac_kontrol()

   Maçtaki iki takımın da turnuvaya kayıtlı olmasını, sonucun yalnızca devam eden turnuvada girilmesini denetler; sonucu girilmiş maçın değiştirilmesini engeller ve skora göre *kazanan_id* alanını otomatik doldurur.

5. **`trg_mac_puan_guncelle`**
   AFTER UPDATE OF durum — mac (durum 'Oynandı' olduğunda) · Fonksiyon: trg_mac_puan_guncelle()

   Maç oynandığında puan tablosunu (turnuva_katilim) günceller: kazanana galibiyet ve 3 puan, kaybedene mağlubiyet yazar; iki takımın skor farkını (averaj) günceller.

6. **`trg_istatistik_kontrol`**
   BEFORE INSERT OR UPDATE — oyuncu_mac_istatistik · Fonksiyon: trg_istatistik_kontrol()

   İstatistiğin yalnızca oynanmış maç için ve maç tarihinde maçtaki takımlardan birinin kadrosunda olan oyuncu için girilmesini sağlar; oyuncunun takımını otomatik bulur. MVP seçildiğinde aynı maçtaki diğer MVP'yi kaldırır.

7. **`trg_turnuva_durum`**
   AFTER UPDATE OF durum — turnuva · Fonksiyon: trg_turnuva_durum()

   Tamamlanmış turnuvanın yeniden açılmasını ve oynanmamış maç varken turnuvanın bitirilmesini engeller. Turnuva "Tamamlandı" olduğunda *puan_tablosu* fonksiyonuyla final sıralarını yazar ve ödülleri dereceye giren takımlara atar.

8. **`trg_turnuva_silme_kontrol`**
   BEFORE DELETE — turnuva · Fonksiyon: trg_turnuva_silme_kontrol()

   Devam eden bir turnuvanın silinmesini engeller.

9. **`trg_odul_kontrol`**
   BEFORE INSERT OR UPDATE OF tutar — odul · Fonksiyon: trg_odul_kontrol()

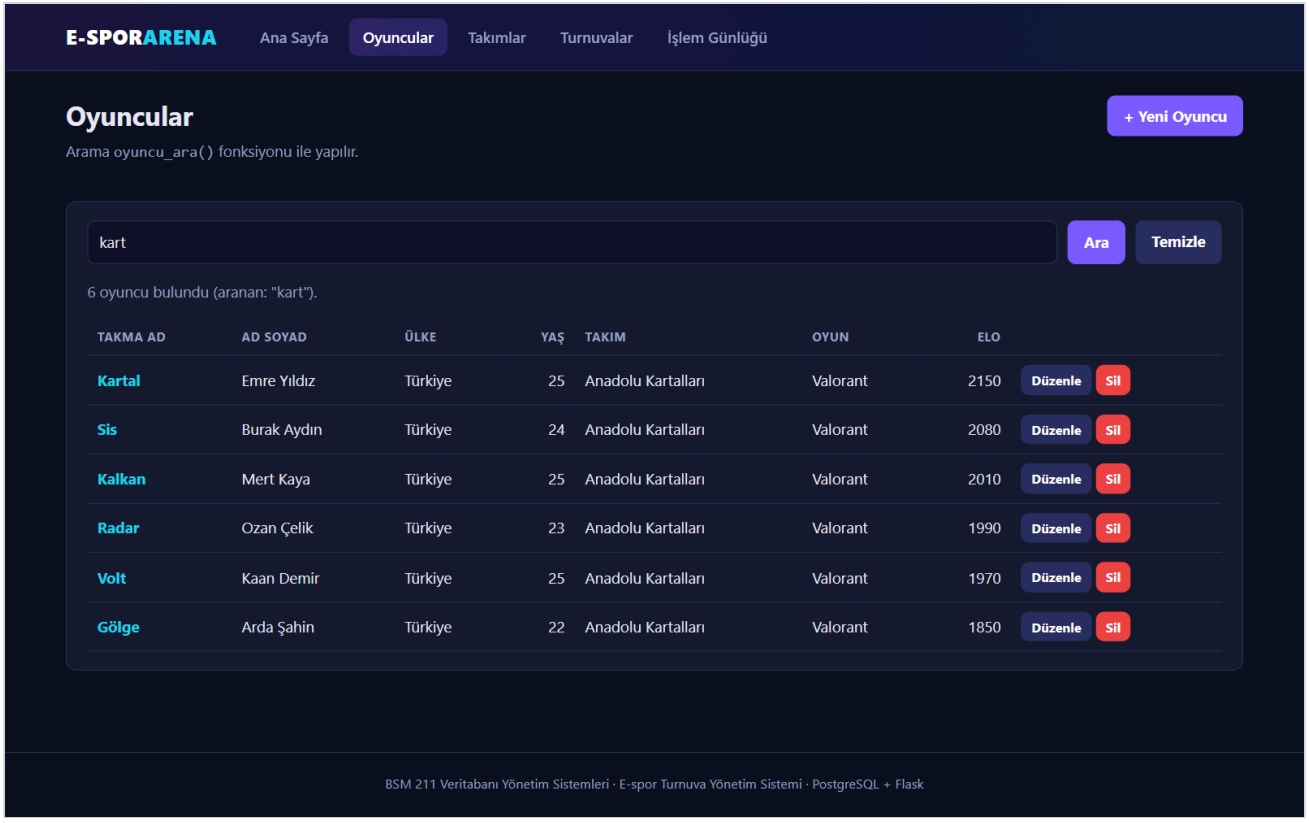
   Bir turnuvanın ödülleri toplamının turnuvanın ödül havuzunu aşmasını engeller.

10. **`trg_kisi_log, trg_takim_log, trg_turnuva_log`**
    AFTER INSERT OR UPDATE OR DELETE — kisi, takim, turnuva · Fonksiyon: trg_islem_log()

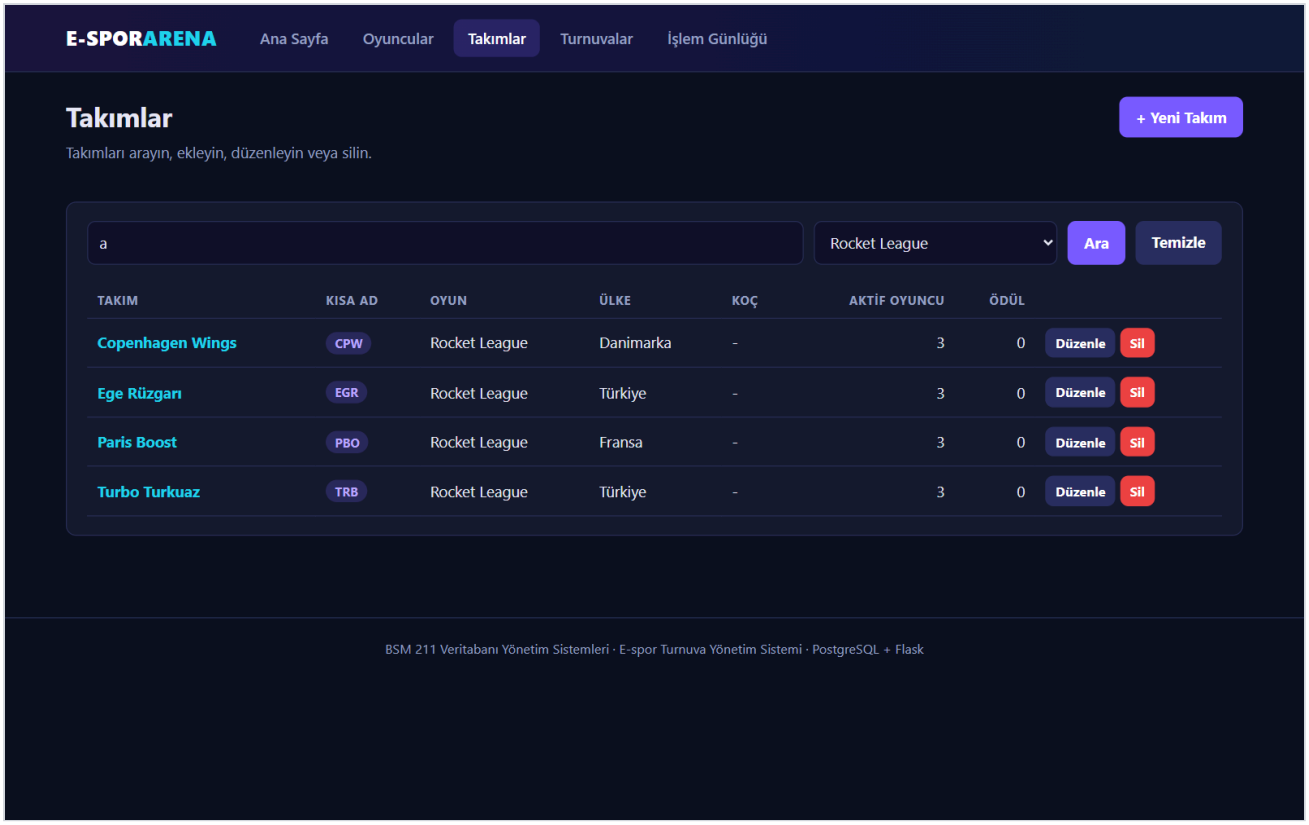
    Yapılan her değişikliği tablo adı, işlem türü, kaydın JSON içeriği, veritabanı kullanıcısı ve zaman bilgisiyle *islem_log* tablosuna yazar (denetim kaydı).

e) Ekran Görüntüleri

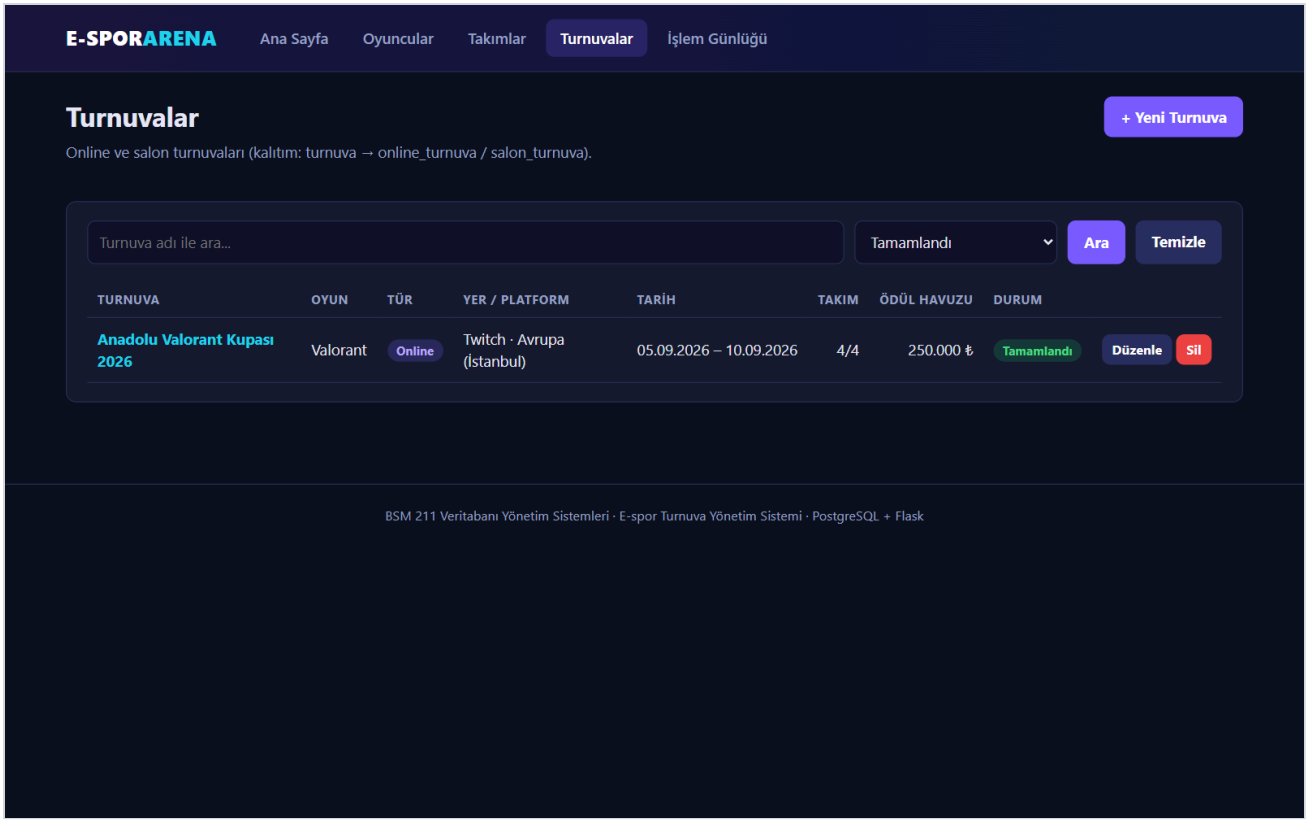
Arama İşlemleri



Şekil 1. Oyuncu arama: "kart" metniyle *oyuncu_ara()* fonksiyonu takma ad, ad, soyad ve takım adında arar.

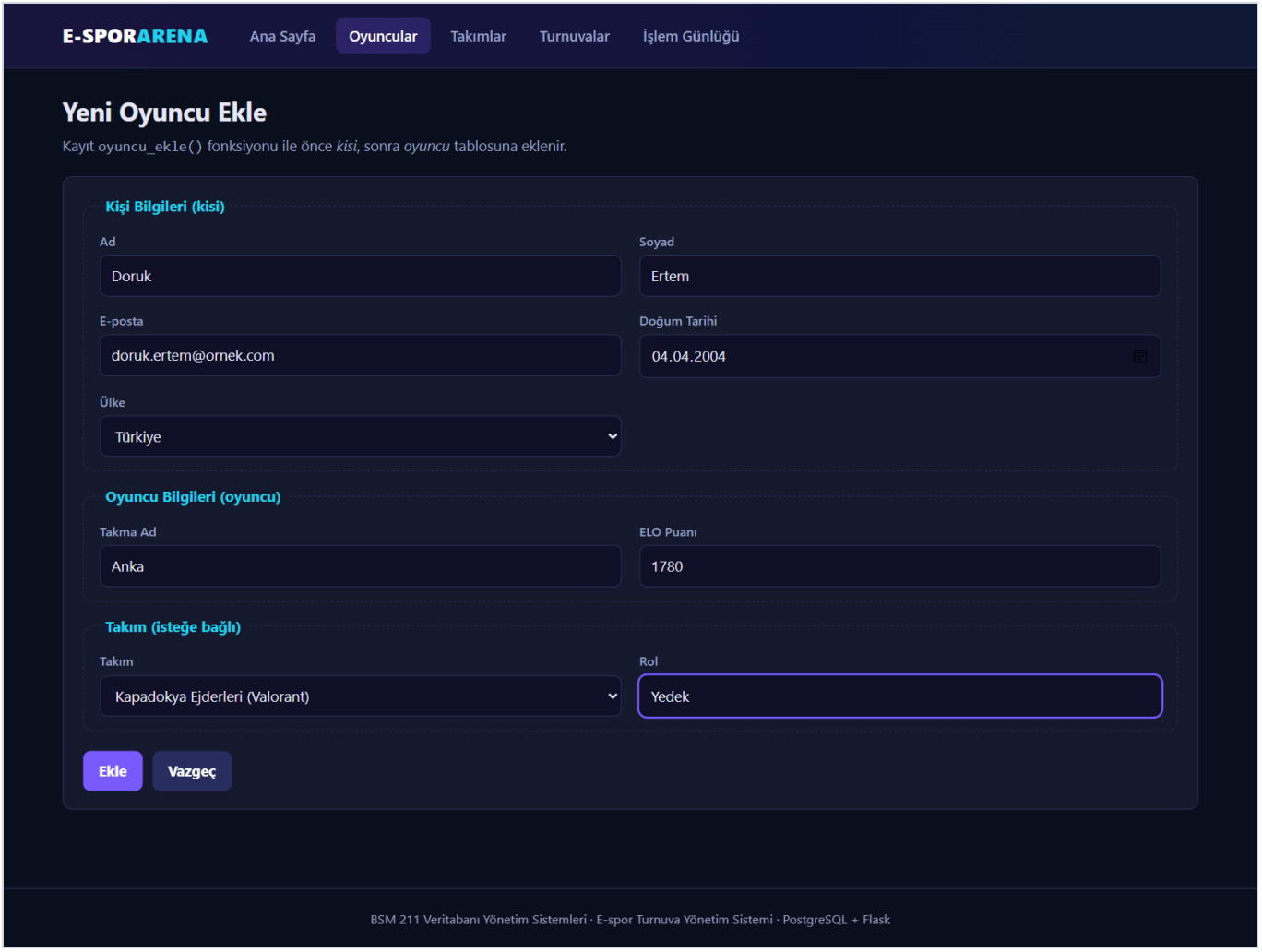


Şekil 2. Takım arama: ad filtresi ve oyun seçimi (Rocket League) ile arama.

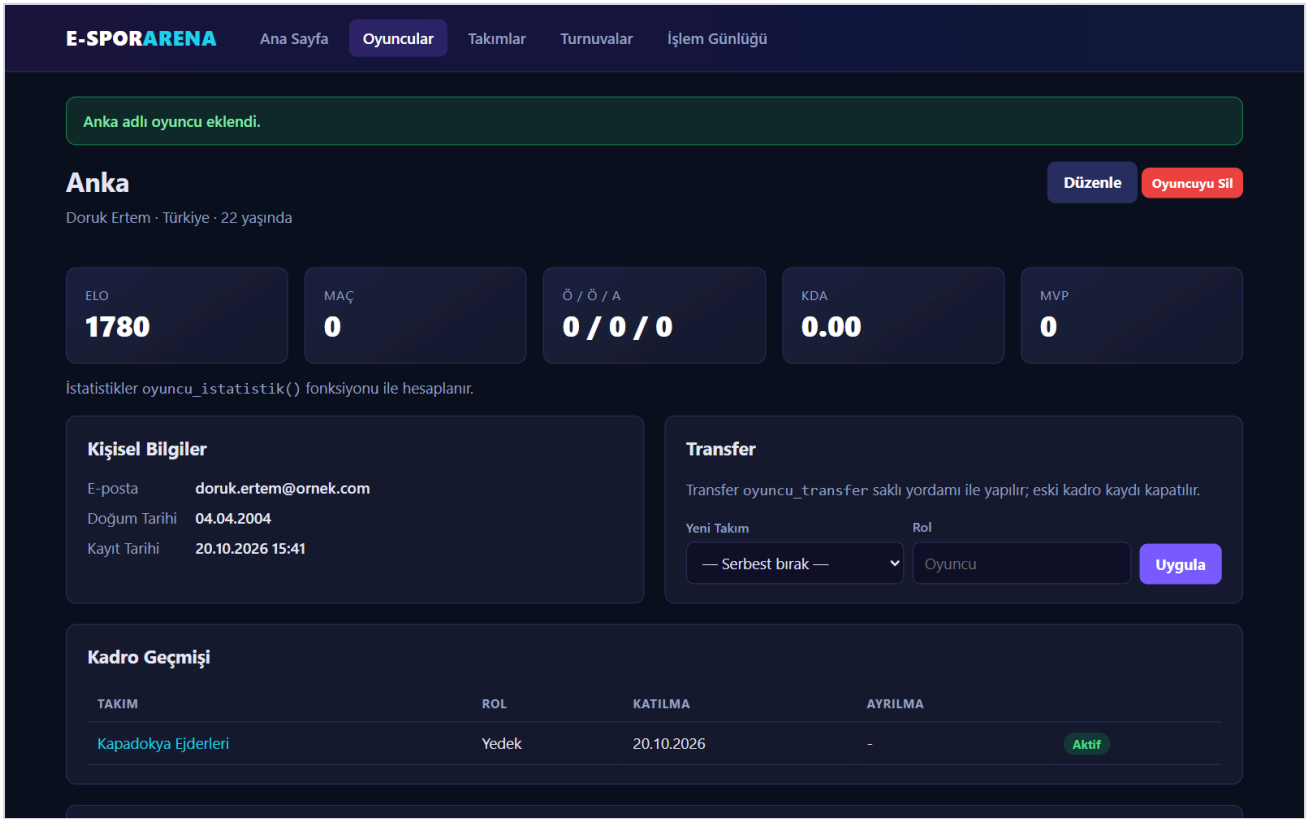


Şekil 3. Turnuva arama: durum filtresi (Tamamlandı) ile arama.

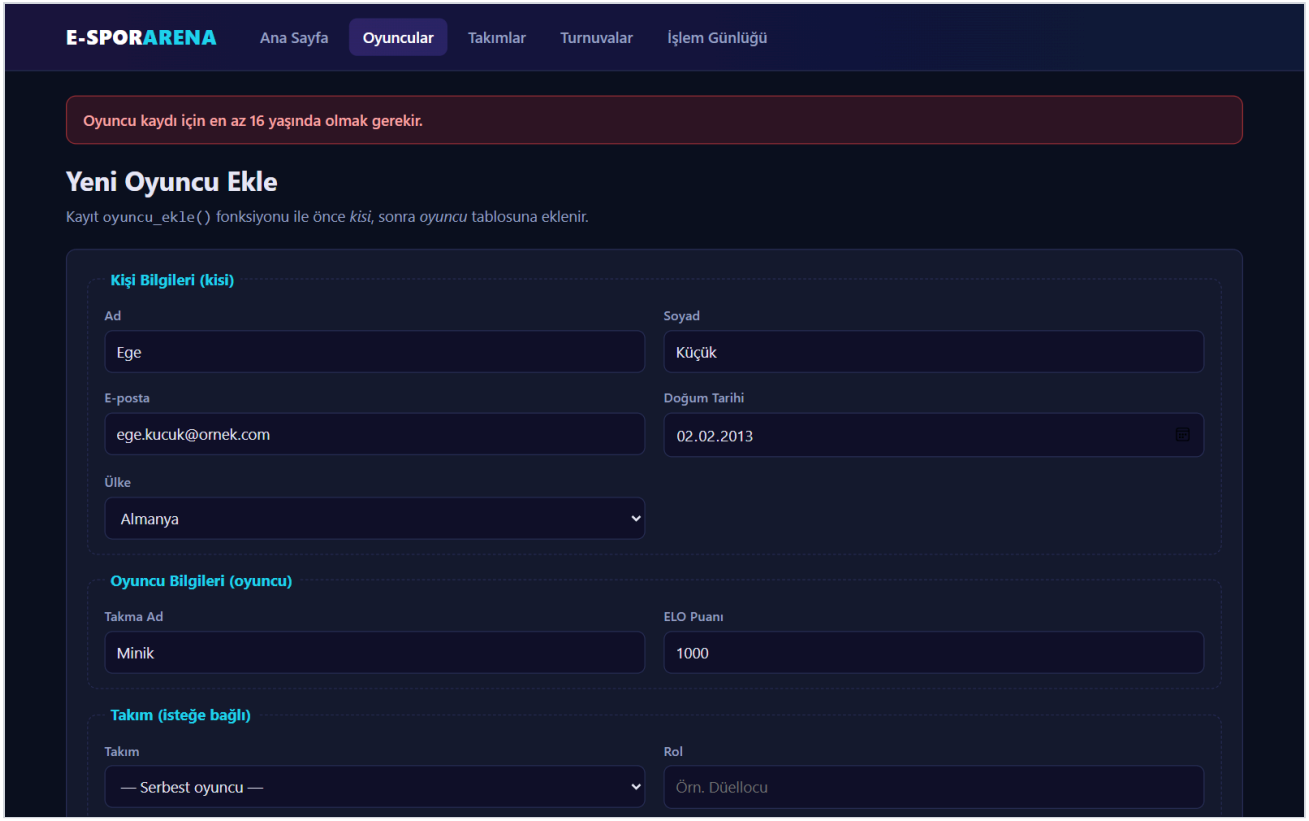
Ekleme İşlemleri



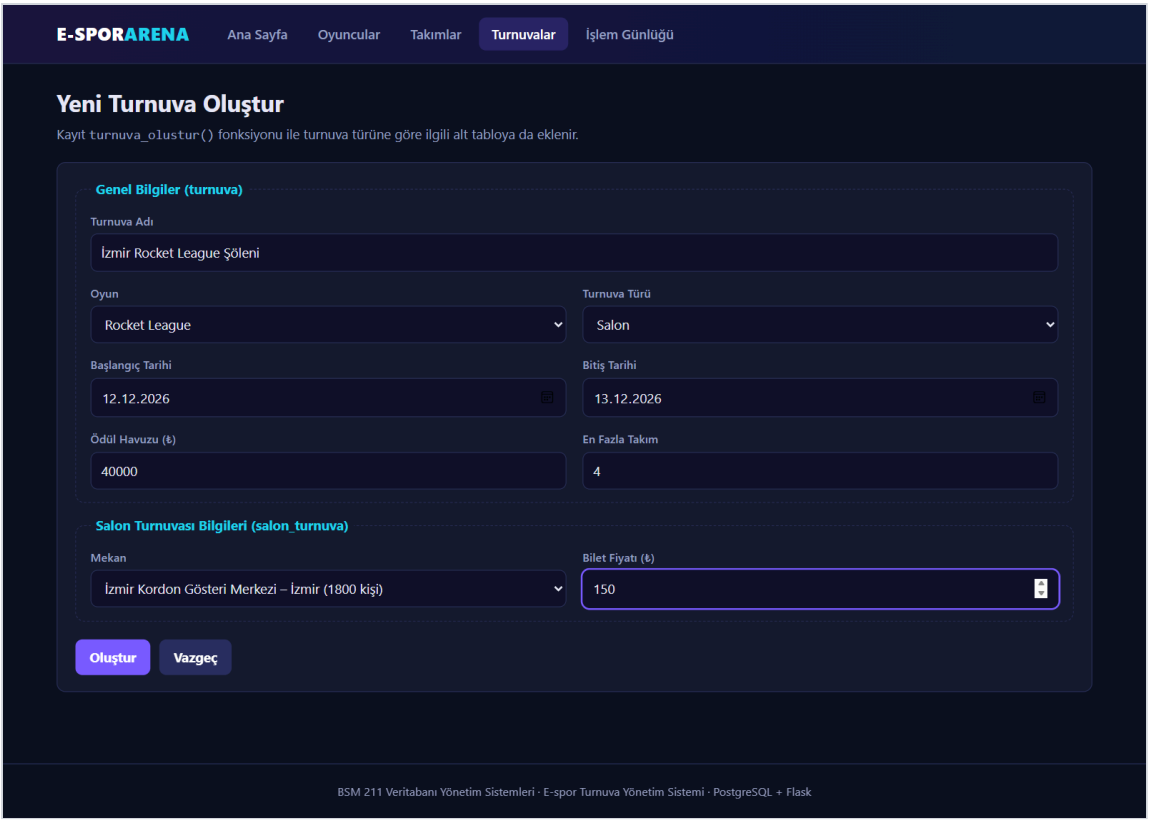
Şekil 4. Yeni oyuncu formu: kişi (üst tip) ve oyuncu (alt tip) bilgileri ile takım seçimi.



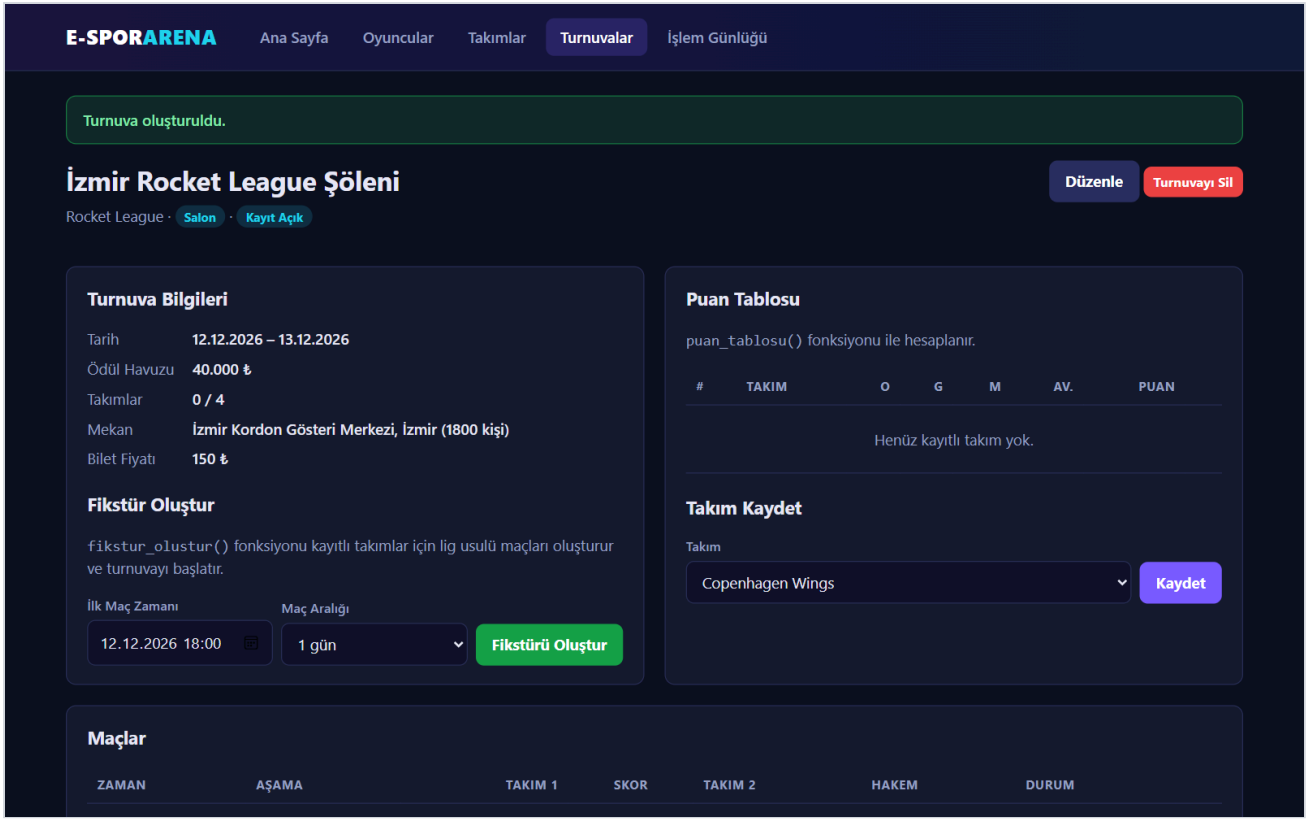
Şekil 5. Oyuncu *oyuncu_ekle()* fonksiyonuyla eklendi ve *oyuncu_transfer* yordamıyla takıma katıldı.



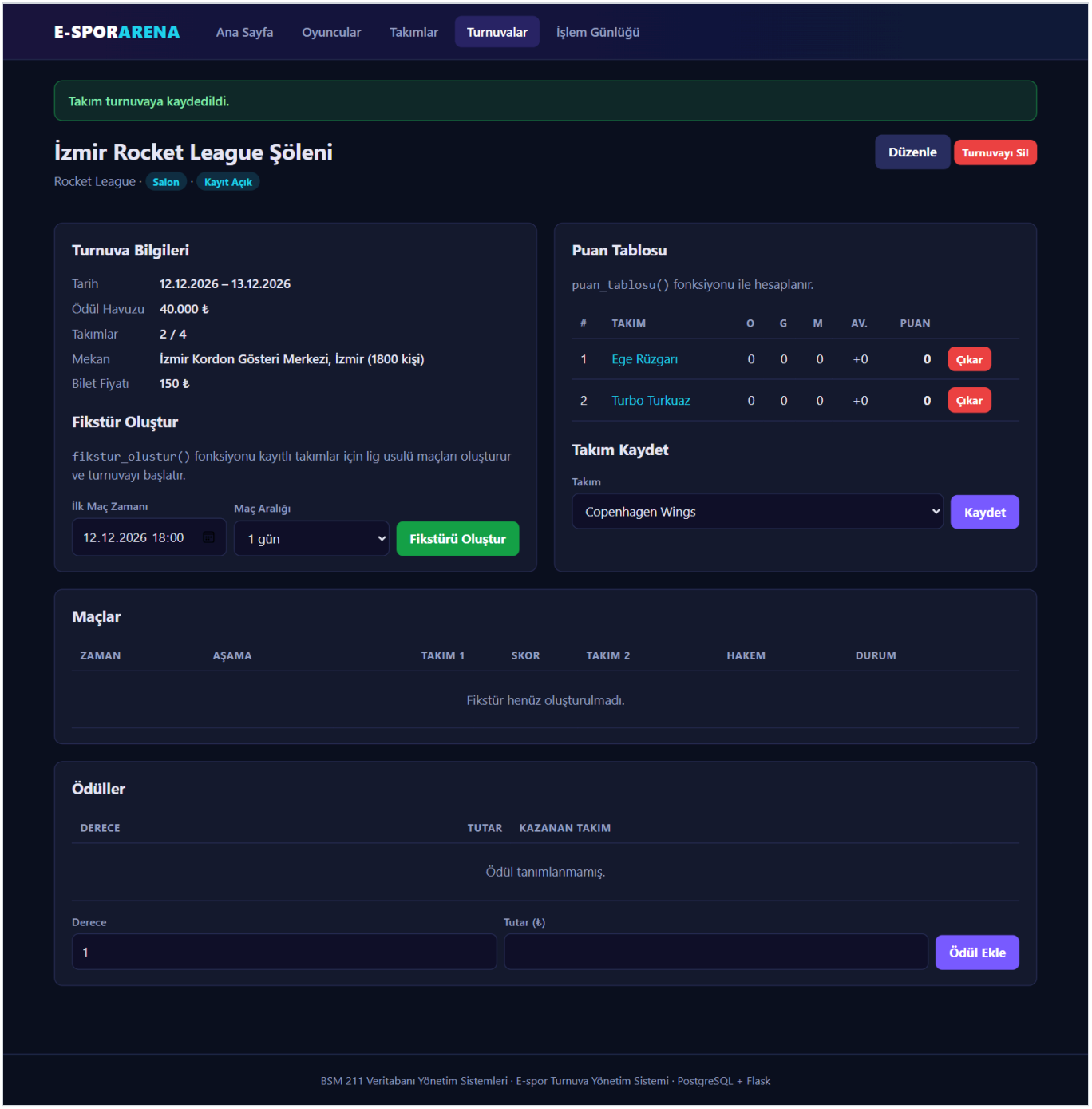
Şekil 6. 16 yaşından küçük oyuncu eklenmek istendiğinde *trg_oyuncu_alt_tip* tetikleyicisi işlemi engelledi.



Şekil 7. Yeni salon turnuvası formu (türe göre online / salon alanları değişir).

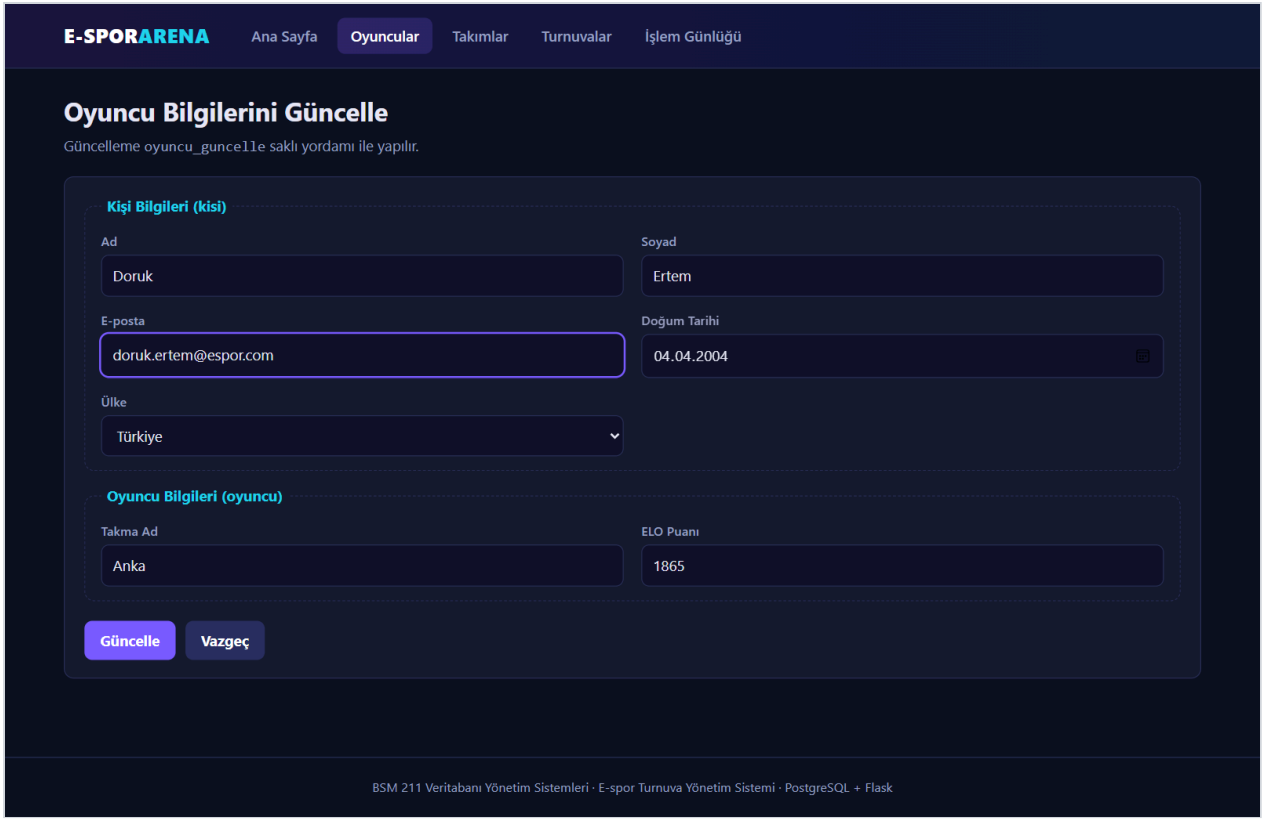


Şekil 8. Turnuva *turnuva_olustur()* fonksiyonuyla turnuva ve salon_turnuva tablolarına eklendi.

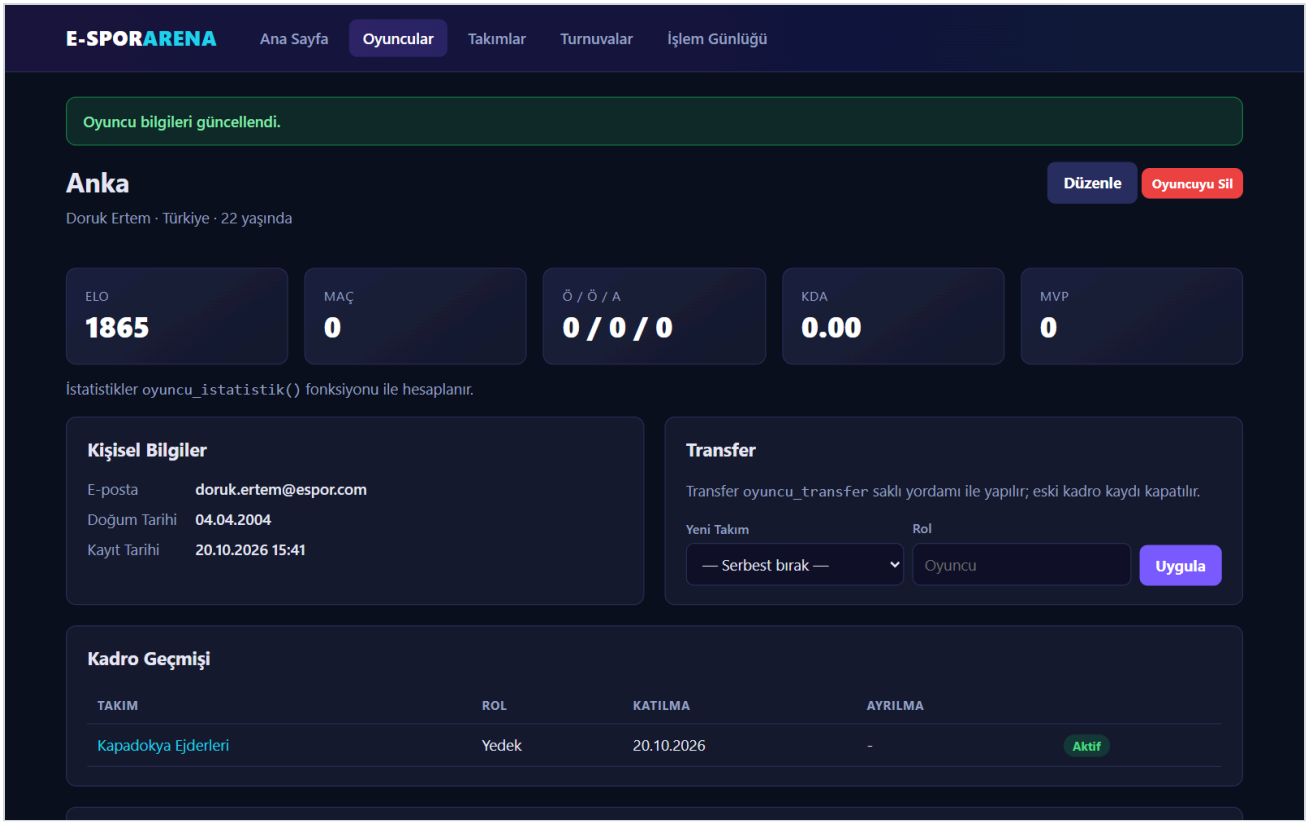


Şekil 9. Turnuvaya takım kaydı eklendi (*trg_katilim_kontrol* denetiminden geçerek).

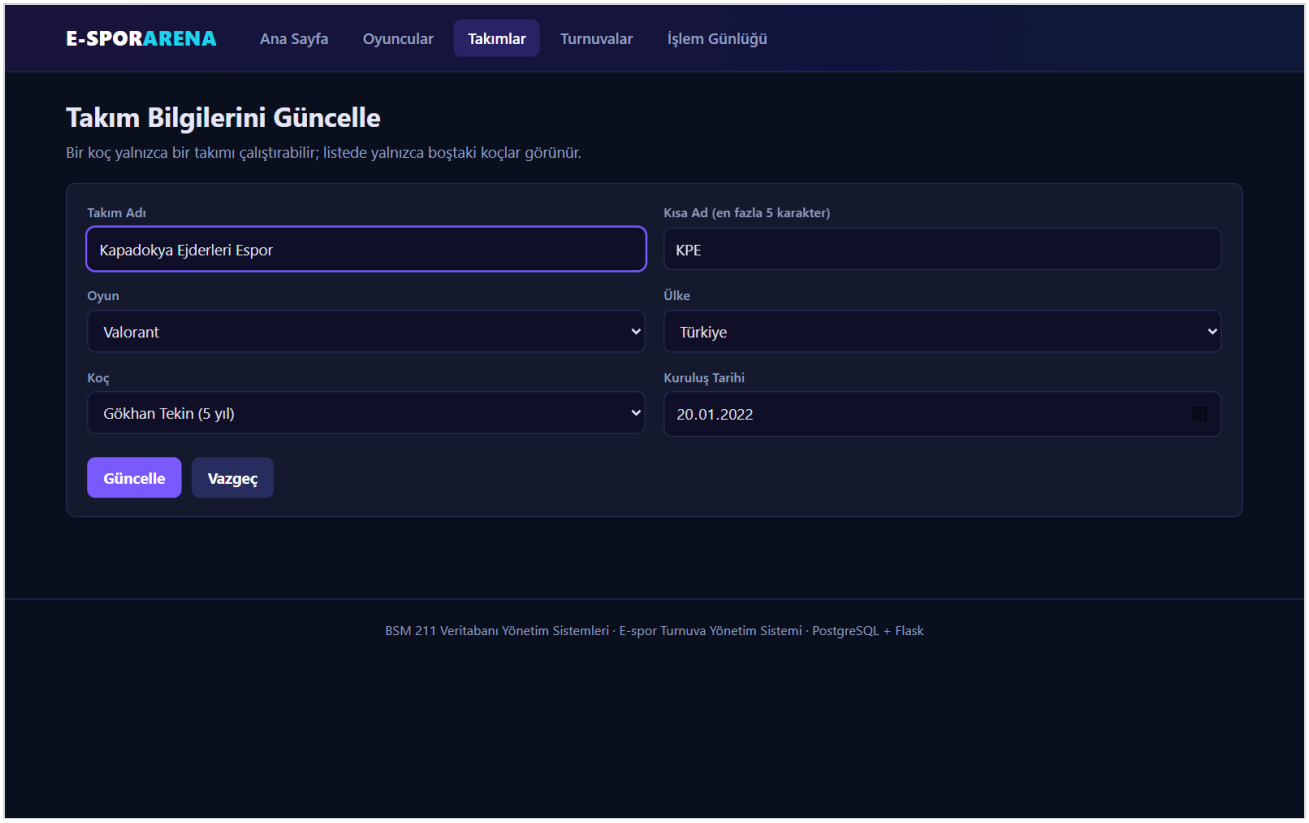
Güncelleme İşlemleri



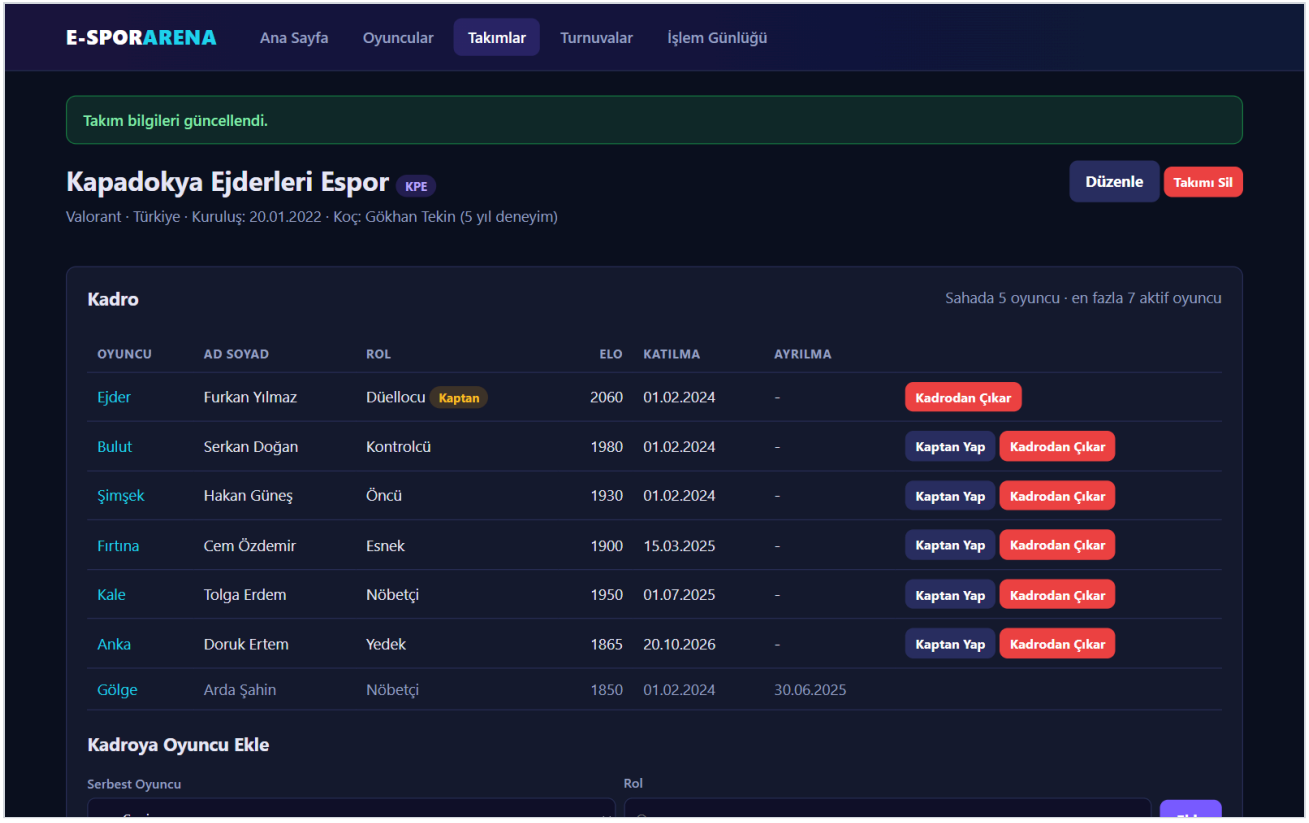
Şekil 10. Oyuncu düzenleme formu (ELO ve e-posta değiştiriliyor).



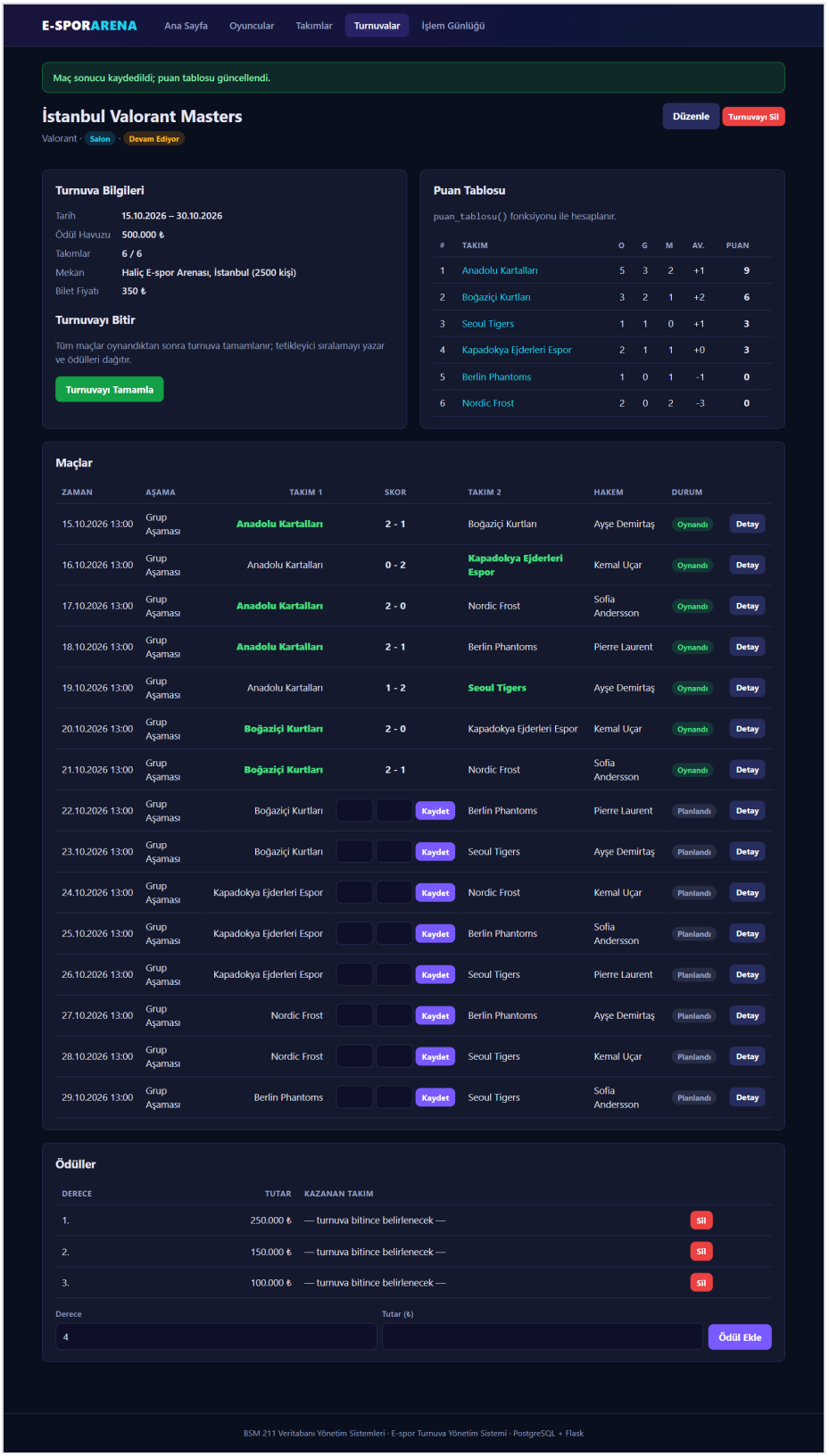
Şekil 11. Oyuncu *oyuncu_guncelle* saklı yordamıyla güncellendi.



Şekil 12. Takım düzenleme formu (takım adı değiştiriliyor).

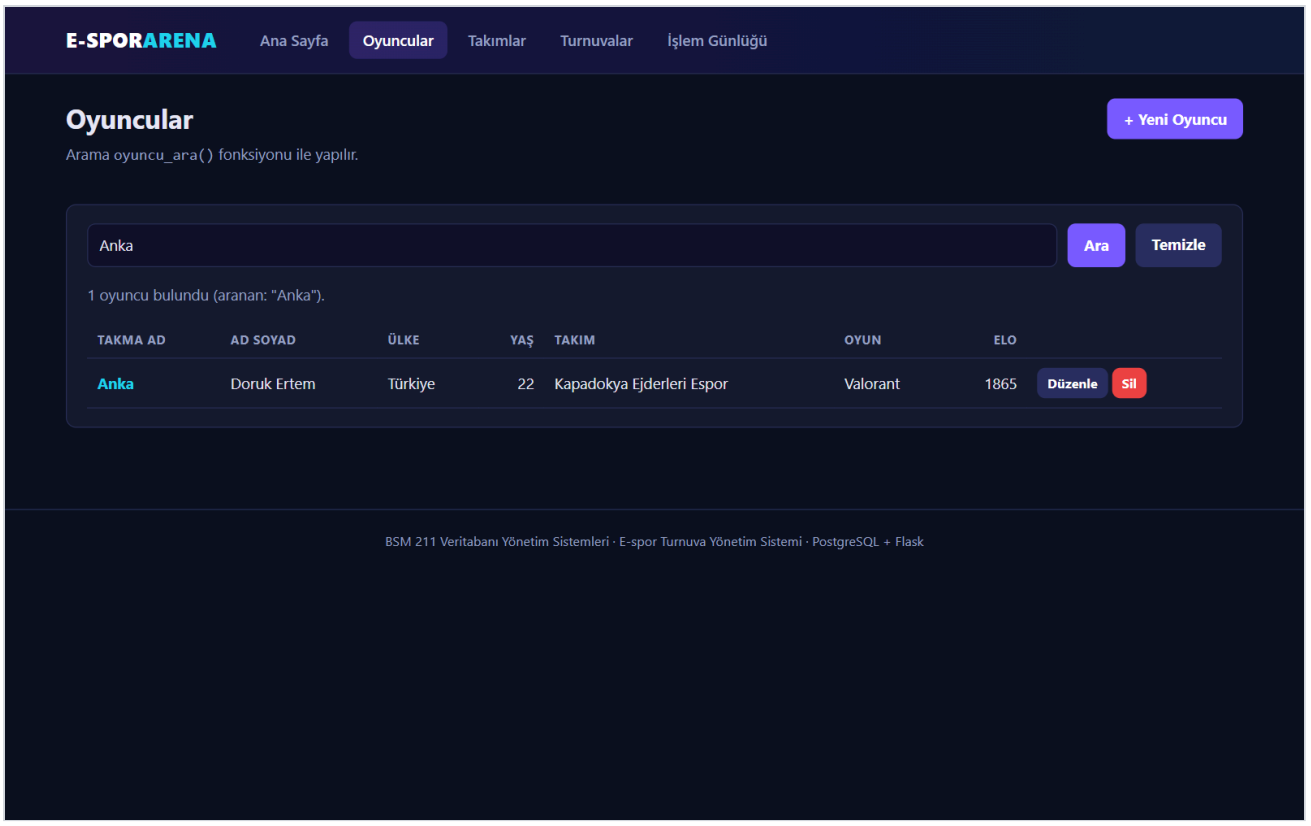


Şekil 13. Takım bilgileri güncellendi.

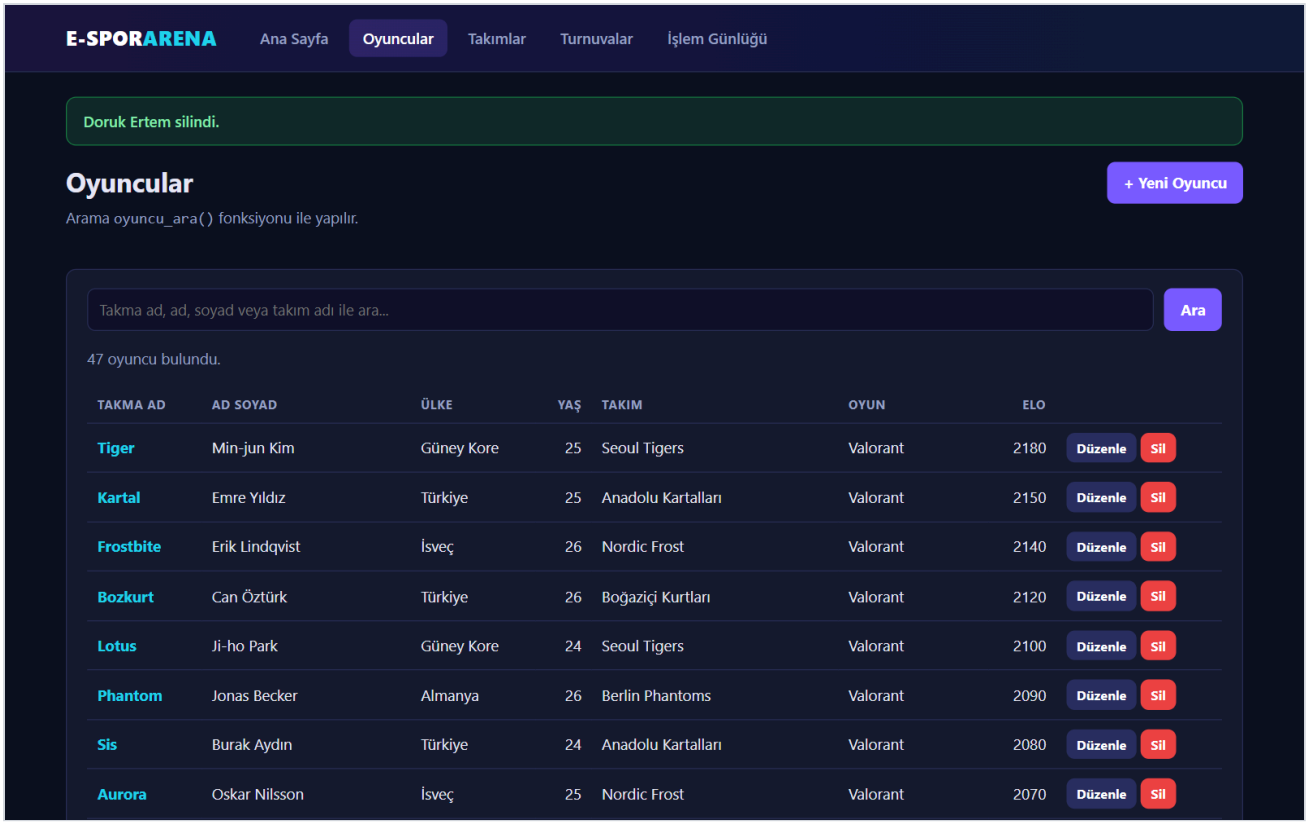


Şekil 14. Maç sonucu *mac_sonucu_gir* yordamıyla girildi; kazanan ve puan tablosu tetikleyicilerle güncellendi.

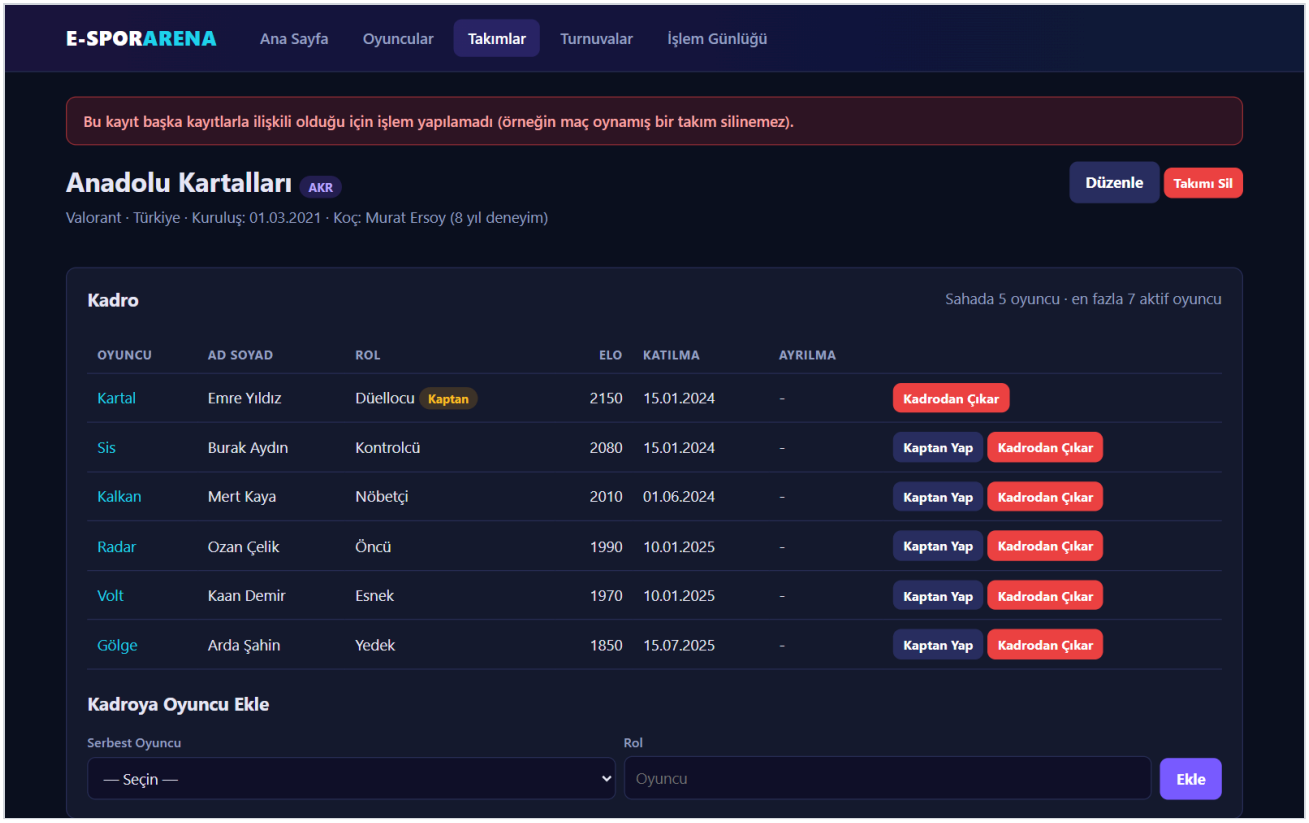
Silme İşlemleri



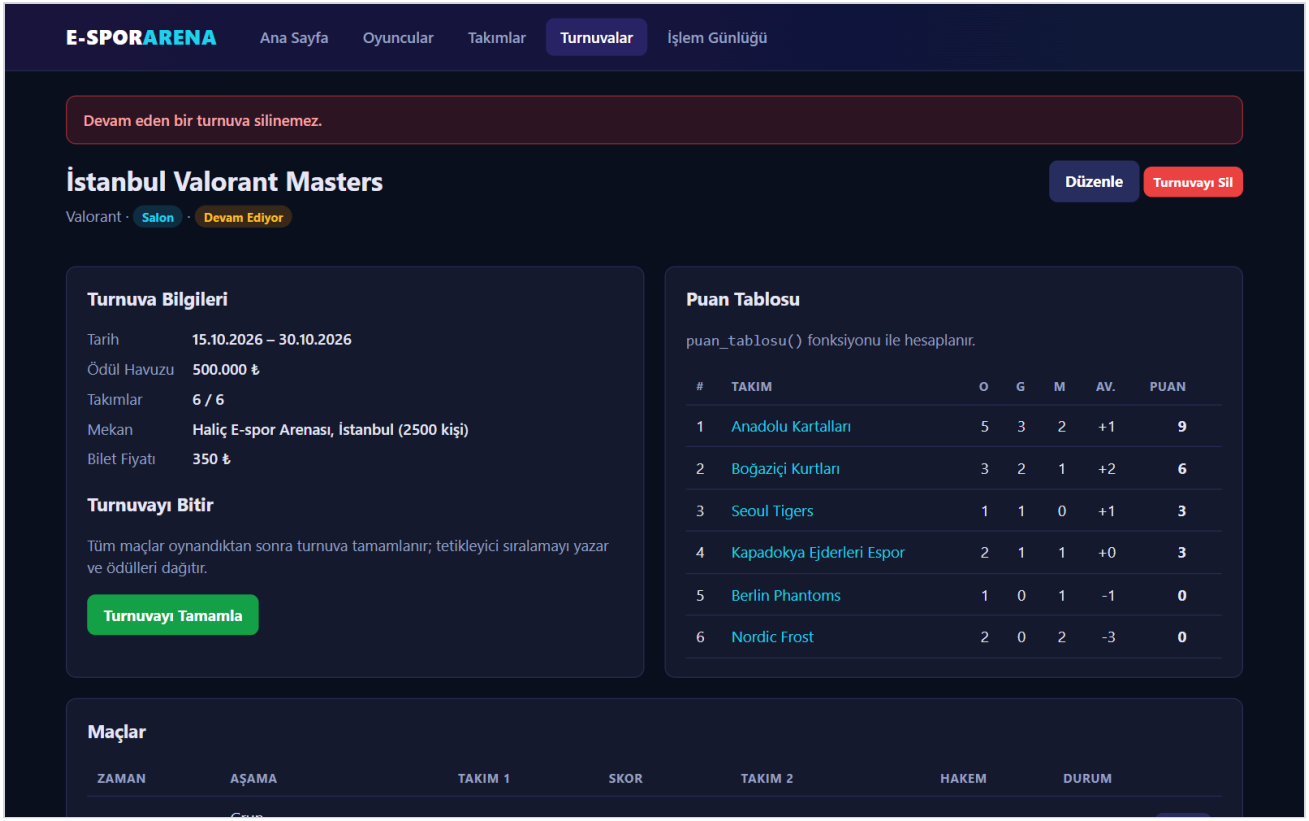
Şekil 15. Silinecek oyuncu (Anka) listede; Sil butonu onay sorduktan sonra kaydı siler.



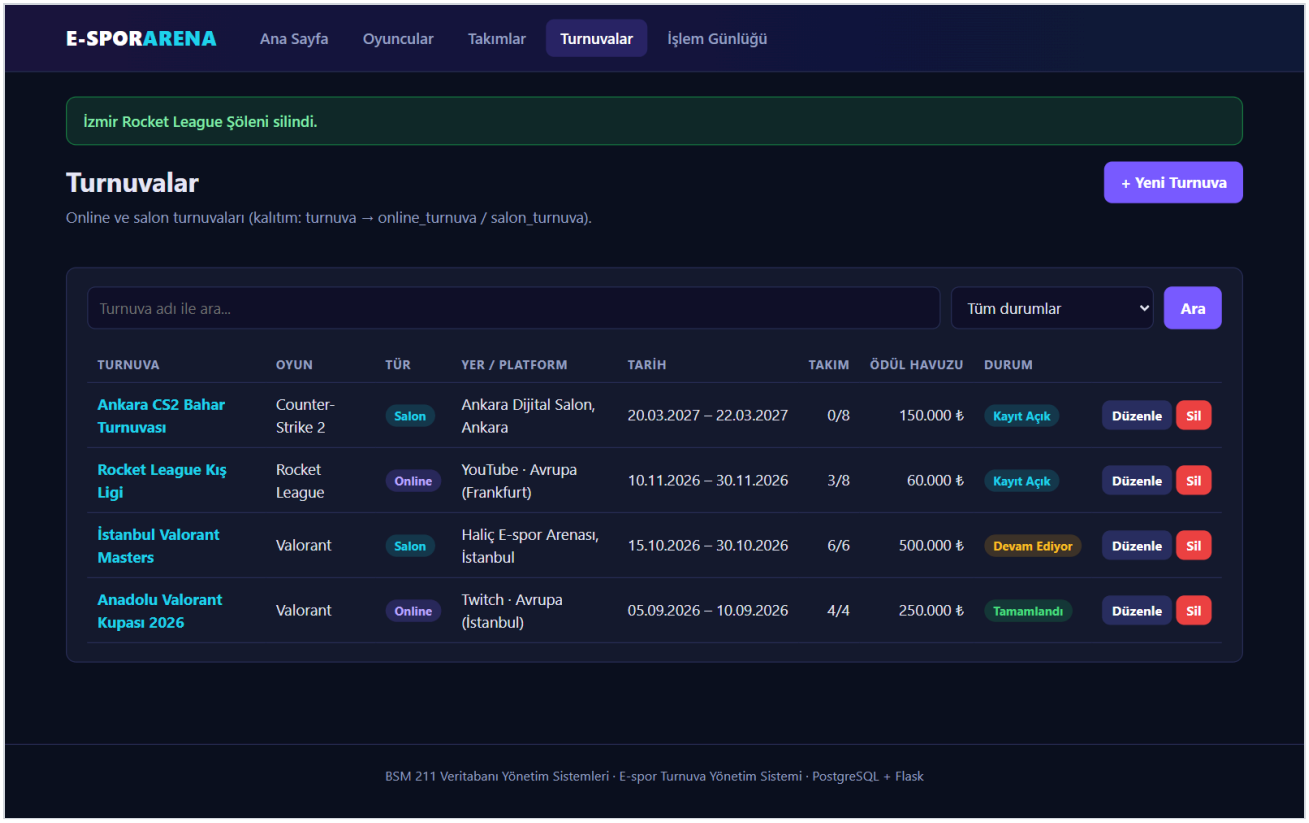
Şekil 16. Oyuncu silindi (kisi kaydı silinince oyuncu alt tip kaydı ON DELETE CASCADE ile silinir).



Şekil 17. Maç oynamış bir takım silinmek istendiğinde yabancı anahtar kısıtı işlemi engelledi.

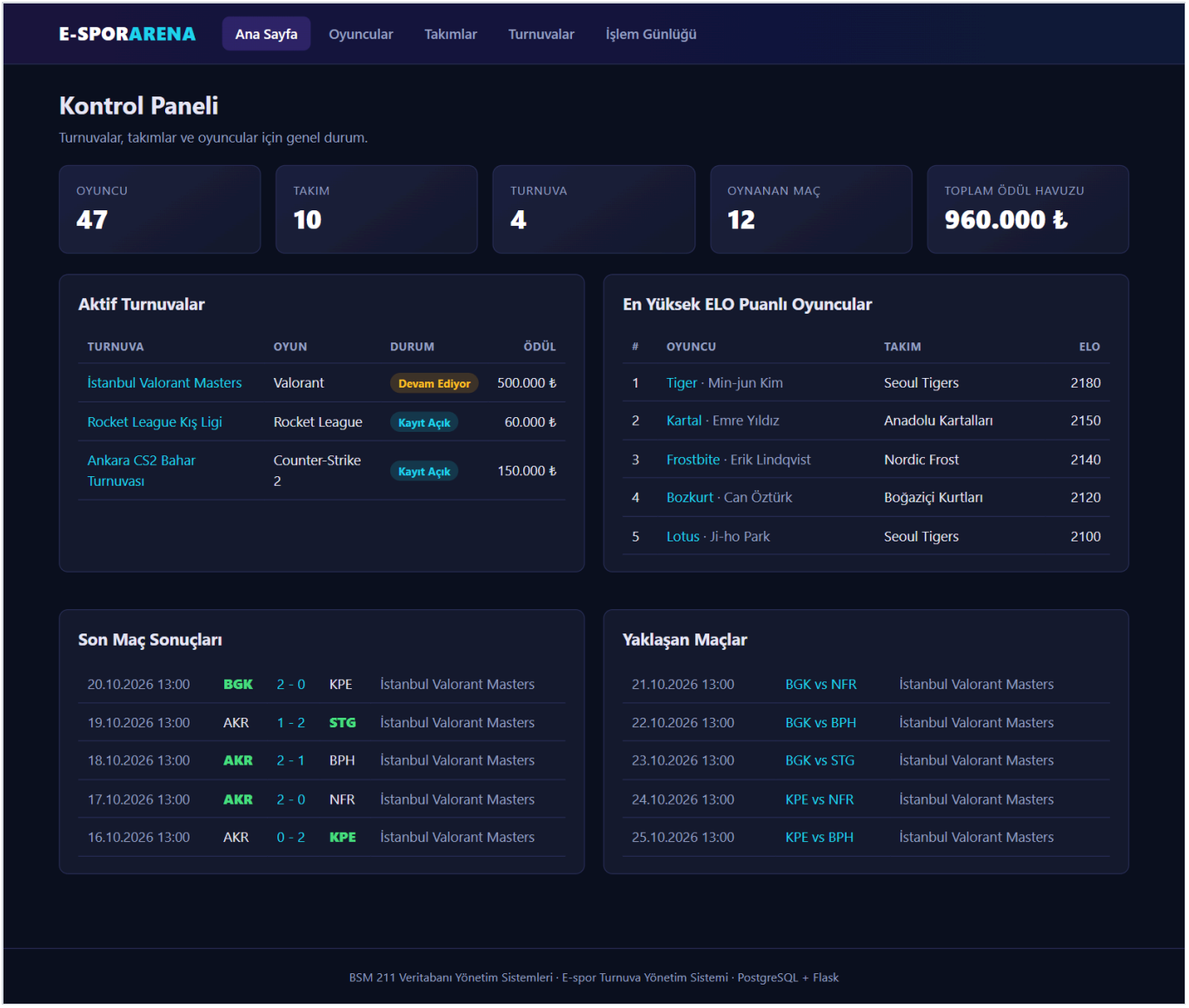


Şekil 18. Devam eden turnuva silinmek istendiğinde *trg_turnuva_silme_kontrol* tetikleyicisi işlemi engelledi.

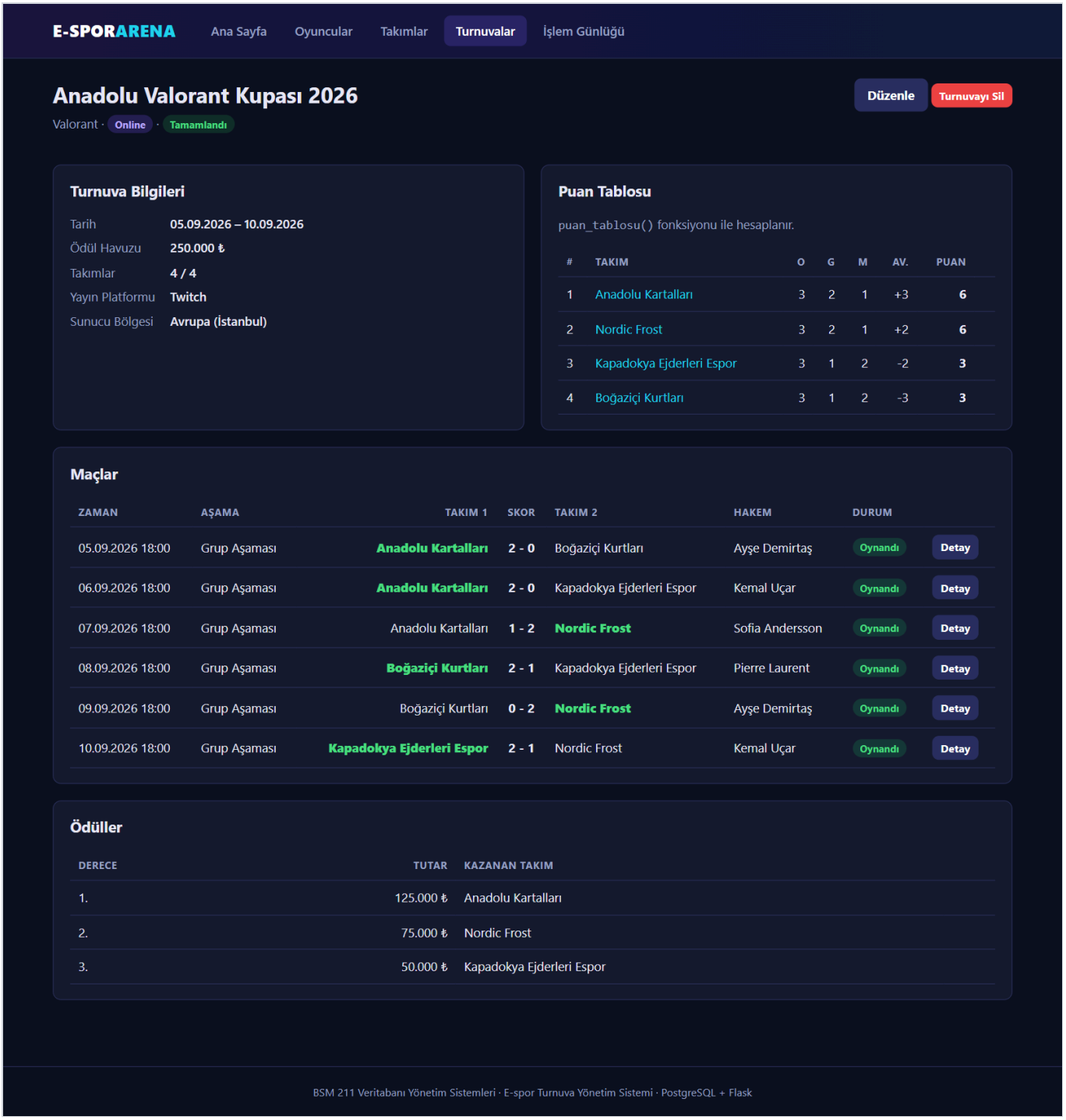


Şekil 19. Kayıt açık durumdaki turnuva silindi.

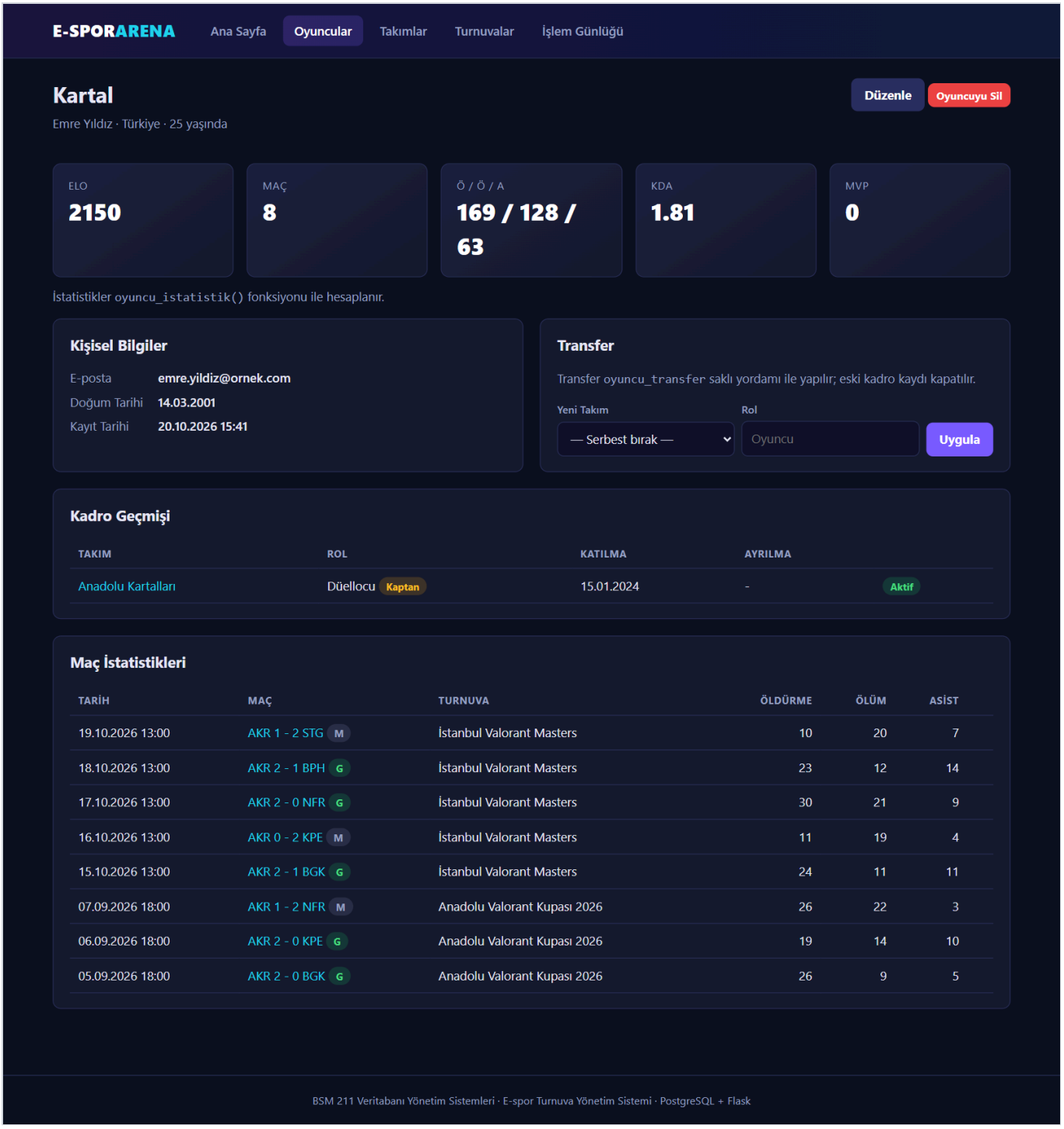
Diğer Ekranlar



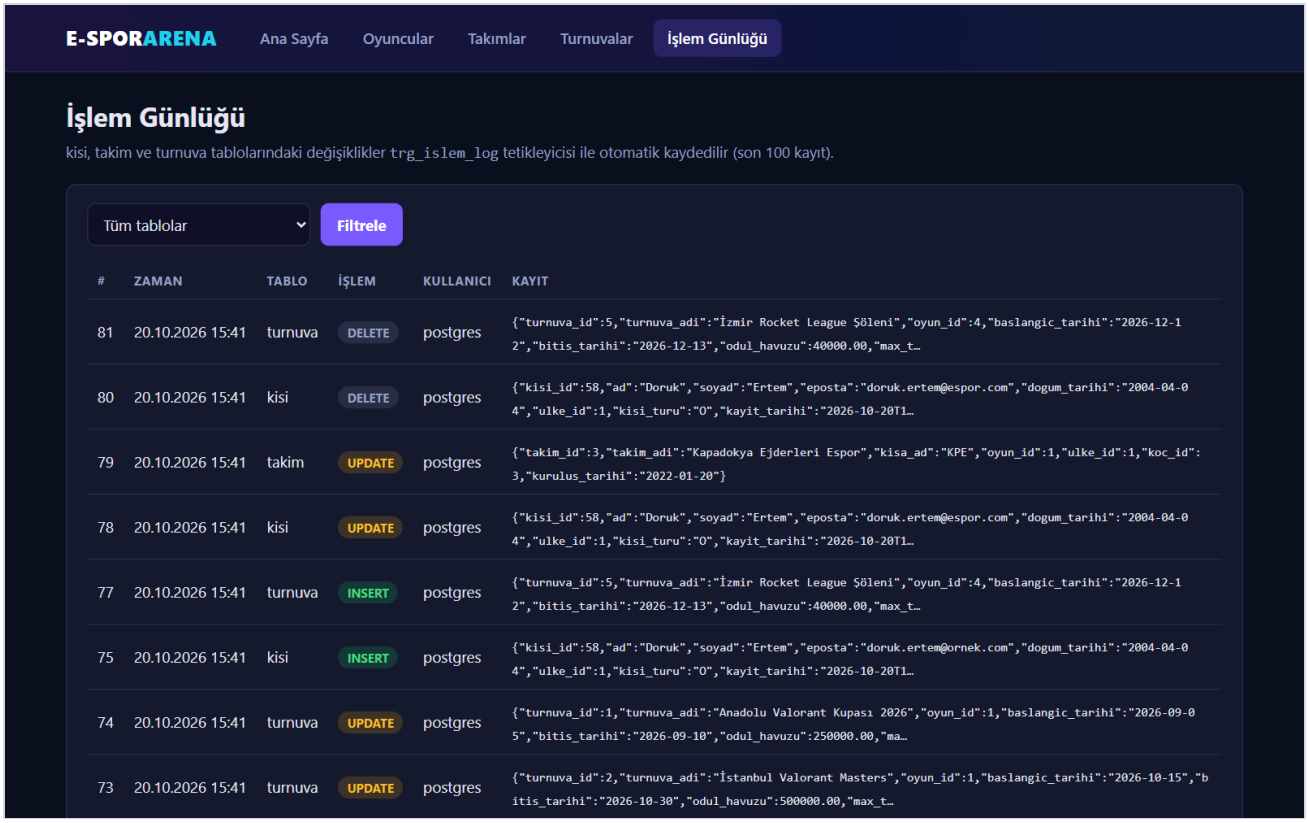
Şekil 20. Ana sayfa (kontrol paneli).



Şekil 21. Tamamlanmış turnuva: final sıralaması ve ödüller *trg_turnuva_durum* tetikleyicisi ile atandı.



Şekil 22. Oyuncu detayı: *oyuncu_istatistik()* ile hesaplanan istatistikler, kadro geçmişi ve maçlar.



Şekil 23. İşlem günlüğü: *trg_islem_log* tetikleyicisinin tuttuğu denetim kayıtları.

f) Uygulamanın Kaynak Kodları

Kaynak kodlarının tamamı şu depoda yer almaktadır: https://github.com/Turan16-k/vtys

Proje klasör yapısı:

```
espor-turnuva/
├── app.py                 Flask web uygulaması (rotalar, veritabanı erişimi)
├── config.py              PostgreSQL bağlantı ayarları
├── kurulum.py             Veritabanını oluşturan ve SQL betiklerini çalıştıran betik
├── requirements.txt
├── sql/                   01_tablolar, 02_fonksiyonlar, 03_tetikleyiciler, 04_ornek_veriler
├── templates/             Jinja2 HTML şablonları
├── static/style.css
├── diyagram/              er_diyagrami.drawio, er_diyagrami.svg
└── rapor/                 rapor_olustur.py, Rapor.pdf, goruntuler/
```

config.py

```
import os

# PostgreSQL bağlantı bilgileri. Kendi kurulumunuza göre düzenleyin
# ya da aynı isimli ortam değişkenlerini tanımlayın.
DB_AYARLARI = {
    "host": os.environ.get("PGHOST", "localhost"),
    "port": int(os.environ.get("PGPORT", "5432")),
    "user": os.environ.get("PGUSER", "postgres"),
    "password": os.environ.get("PGPASSWORD", "postgres"),
    "dbname": os.environ.get("ESPOR_DB", "espor_turnuva"),
}
```

kurulum.py

```
"""Veritabanını oluşturur ve sql/ klasöründeki betikleri sırayla çalıştırır.

Kullanım:  python kurulum.py
"""
from pathlib import Path

import psycopg
from psycopg import sql

from config import DB_AYARLARI

SQL_KLASORU = Path(__file__).parent / "sql"
BETIKLER = ["01_tablolar.sql", "02_fonksiyonlar.sql", "03_tetikleyiciler.sql", "04_ornek_veriler.sql"]


def veritabani_olustur():
    ayarlar = {**DB_AYARLARI, "dbname": "postgres"}
    with psycopg.connect(**ayarlar, autocommit=True) as conn:
        var_mi = conn.execute(
            "SELECT 1 FROM pg_database WHERE datname = %s", (DB_AYARLARI["dbname"],)
        ).fetchone()
        if not var_mi:
            conn.execute(
                sql.SQL("CREATE DATABASE {} ENCODING 'UTF8' TEMPLATE template0").format(
                    sql.Identifier(DB_AYARLARI["dbname"])
                )
            )
            print(f"'{DB_AYARLARI['dbname']}' veritabanı oluşturuldu.")


def betikleri_calistir():
    with psycopg.connect(**DB_AYARLARI) as conn:
        for ad in BETIKLER:
            conn.execute((SQL_KLASORU / ad).read_text(encoding="utf-8"))
            print(f"{ad} çalıştırıldı.")
        conn.commit()
```

```
    # Raporda tek dosya olarak verilebilmesi için tüm betikler birleştirilir.
    birlesik = "\n\n".join((SQL_KLASORU / ad).read_text(encoding="utf-8") for ad in BETIKLER)
    (SQL_KLASORU / "tum_kurulum.sql").write_text(birlesik, encoding="utf-8")


if __name__ == "__main__":
    veritabani_olustur()
    betikleri_calistir()
    print("Kurulum tamamlandı.")
```

app.py

```
"""E-spor Turnuva Yönetim Sistemi - Flask web uygulaması.

Database-First: Veritabanı sql/ klasöründeki betiklerle oluşturulur; uygulama
ORM kullanmaz, tablolara SQL ile, iş mantığına ise veritabanındaki
fonksiyon / saklı yordamlar aracılığıyla erişir.
"""
from datetime import date, datetime

import psycopg
from flask import Flask, flash, g, redirect, render_template, request, url_for
from psycopg import errors
from psycopg.rows import dict_row

from config import DB_AYARLARI

app = Flask(__name__)
app.secret_key = "espor-turnuva-vtys-2026"


# ---------------------------------------------------------------------
#  Veritabanı yardımcıları
# ---------------------------------------------------------------------
def baglanti():
    if "db" not in g:
        g.db = psycopg.connect(**DB_AYARLARI, row_factory=dict_row)
    return g.db


@app.teardown_appcontext
def baglanti_kapat(_hata):
    db = g.pop("db", None)
    if db is not None:
        db.close()


def sorgu(sql, parametreler=None):
    return baglanti().execute(sql, parametreler).fetchall()


def tek(sql, parametreler=None):
    return baglanti().execute(sql, parametreler).fetchone()


def calistir(sql, parametreler=None):
    """Değişiklik yapan bir komutu çalıştırır ve onaylar (commit)."""
    db = baglanti()
    try:
        sonuc = db.execute(sql, parametreler)
        satir = sonuc.fetchone() if sonuc.description else None
        db.commit()
        return satir
    except psycopg.Error:
        db.rollback()
        raise


def hata_mesaji(hata):
    """Veritabanından gelen hatayı kullanıcıya gösterilecek metne çevirir."""
    mesaj = hata.diag.message_primary or str(hata)
    if isinstance(hata, errors.RaiseException):
        return mesaj  # tetikleyici / fonksiyonlardan gelen iş kuralı mesajı
    if isinstance(hata, errors.ForeignKeyViolation):
        return "Bu kayıt başka kayıtlarla ilişkili olduğu için işlem yapılamadı (örneğin maç oynamış bir takım
```

```
silinemez)."
    if isinstance(hata, errors.UniqueViolation):
        return "Aynı bilgilerle kayıtlı başka bir kayıt var: " + (hata.diag.message_detail or mesaj)
    if isinstance(hata, errors.CheckViolation):
        return f"Girilen değerler kurallara uymuyor ({hata.diag.constraint_name})."
    if isinstance(hata, errors.NotNullViolation):
        return f"Zorunlu alan boş bırakılamaz ({hata.diag.column_name})."
    return mesaj


def donus_adresi(varsayilan):
    """Formdan gelen dönüş adresini yalnızca site içi bir yol ise kullanır."""
    adres = request.form.get("donus") or ""
    return adres if adres.startswith("/") and not adres.startswith("//") else varsayilan


def bos_ise_none(deger):
    deger = (deger or "").strip()
    return deger or None


# ---------------------------------------------------------------------
#  Şablon filtreleri
# ---------------------------------------------------------------------
@app.template_filter("tarih")
def tarih_filtresi(d):
    if isinstance(d, datetime):
        return d.strftime("%d.%m.%Y %H:%M")
    if isinstance(d, date):
        return d.strftime("%d.%m.%Y")
    return d or "-"


@app.template_filter("para")
def para_filtresi(tutar):
    if tutar is None:
        return "-"
    return f"{tutar:,.0f} ₺".replace(",", ".")


@app.template_filter("html_tarih")
def html_tarih_filtresi(d):
    if isinstance(d, datetime):
        return d.strftime("%Y-%m-%dT%H:%M")
    if isinstance(d, date):
        return d.isoformat()
    return d or ""


@app.context_processor
def ortak_degiskenler():
    return {"bugun": date.today()}


# ---------------------------------------------------------------------
#  ANA SAYFA
# ---------------------------------------------------------------------
@app.route("/")
def ana_sayfa():
    sayilar = tek("""
        SELECT (SELECT count(*) FROM oyuncu)                        AS oyuncu,
               (SELECT count(*) FROM takim)                         AS takim,
               (SELECT count(*) FROM turnuva)                       AS turnuva,
               (SELECT count(*) FROM mac WHERE durum = 'Oynandı')    AS mac,
               (SELECT COALESCE(sum(odul_havuzu), 0) FROM turnuva)   AS odul
    """)
    devam_eden = sorgu("""
        SELECT t.turnuva_id, t.turnuva_adi, o.oyun_adi, t.durum, t.bitis_tarihi, t.odul_havuzu
          FROM turnuva t JOIN oyun o ON o.oyun_id = t.oyun_id
         WHERE t.durum <> 'Tamamlandı'
         ORDER BY t.baslangic_tarihi
    """)
    son_maclar = sorgu("""
        SELECT m.mac_id, m.mac_zamani, tr.turnuva_adi, t1.kisa_ad AS t1, t2.kisa_ad AS t2,
               m.takim1_skor, m.takim2_skor, m.kazanan_id, m.takim1_id, m.takim2_id
          FROM mac m
          JOIN turnuva tr ON tr.turnuva_id = m.turnuva_id
```

```
          JOIN takim t1 ON t1.takim_id = m.takim1_id
          JOIN takim t2 ON t2.takim_id = m.takim2_id
         WHERE m.durum = 'Oynandı'
         ORDER BY m.mac_zamani DESC LIMIT 5
    """)
    yaklasan = sorgu("""
        SELECT m.mac_id, m.mac_zamani, tr.turnuva_adi, t1.kisa_ad AS t1, t2.kisa_ad AS t2
          FROM mac m
          JOIN turnuva tr ON tr.turnuva_id = m.turnuva_id
          JOIN takim t1 ON t1.takim_id = m.takim1_id
          JOIN takim t2 ON t2.takim_id = m.takim2_id
         WHERE m.durum = 'Planlandı'
         ORDER BY m.mac_zamani LIMIT 5
    """)
    en_iyiler = sorgu("SELECT * FROM oyuncu_ara('') LIMIT 5")
    return render_template("index.html", sayilar=sayilar, devam_eden=devam_eden,
                           son_maclar=son_maclar, yaklasan=yaklasan, en_iyiler=en_iyiler)


# ---------------------------------------------------------------------
#  OYUNCULAR
# ---------------------------------------------------------------------
def ulkeler():
    return sorgu("SELECT ulke_id, ulke_adi FROM ulke ORDER BY ulke_adi")


def oyuncu_form_degerleri(form):
    return (form["ad"].strip(), form["soyad"].strip(), form["eposta"].strip(),
            form["dogum_tarihi"], int(form["ulke_id"]), form["takma_ad"].strip(),
            int(form.get("elo_puani") or 1000))


@app.route("/oyuncular")
def oyuncular():
    aranan = request.args.get("q", "").strip()
    kayitlar = sorgu("SELECT * FROM oyuncu_ara(%s)", (aranan,))
    return render_template("oyuncular.html", kayitlar=kayitlar, aranan=aranan)


@app.route("/oyuncular/yeni", methods=["GET", "POST"])
def oyuncu_ekle():
    takimlar = sorgu("""SELECT t.takim_id, t.takim_adi, o.oyun_adi FROM takim t
                        JOIN oyun o ON o.oyun_id = t.oyun_id ORDER BY t.takim_adi""")
    if request.method == "POST":
        db = baglanti()
        try:
            yeni_id = db.execute(
                "SELECT oyuncu_ekle(%s, %s, %s, %s, %s, %s, %s) AS id",
                oyuncu_form_degerleri(request.form)).fetchone()["id"]
            takim_id = bos_ise_none(request.form.get("takim_id"))
            if takim_id:
                db.execute("CALL oyuncu_transfer(%s, %s, %s)",
                           (yeni_id, int(takim_id), request.form.get("rol") or "Oyuncu"))
            db.commit()
            flash(f"{request.form['takma_ad']} adlı oyuncu eklendi.", "basari")
            return redirect(url_for("oyuncu_detay", oyuncu_id=yeni_id))
        except psycopg.Error as e:
            db.rollback()
            flash(hata_mesaji(e), "hata")
    return render_template("oyuncu_form.html", deger=request.form, ulkeler=ulkeler(),
                           takimlar=takimlar, yeni=True)


@app.route("/oyuncular/<int:oyuncu_id>")
def oyuncu_detay(oyuncu_id):
    oyuncu = tek("""
        SELECT k.*, o.takma_ad, o.elo_puani, u.ulke_adi,
               date_part('year', age(k.dogum_tarihi))::INT AS yas
          FROM oyuncu o JOIN kisi k ON k.kisi_id = o.kisi_id
          JOIN ulke u ON u.ulke_id = k.ulke_id
         WHERE o.kisi_id = %s""", (oyuncu_id,))
    if not oyuncu:
        flash("Oyuncu bulunamadı.", "hata")
        return redirect(url_for("oyuncular"))
    istatistik = tek("SELECT * FROM oyuncu_istatistik(%s)", (oyuncu_id,))
    kadro = sorgu("""
        SELECT ko.*, t.takim_adi FROM takim_oyuncu ko JOIN takim t ON t.takim_id = ko.takim_id
```

```
         WHERE ko.oyuncu_id = %s ORDER BY ko.katilma_tarihi DESC""", (oyuncu_id,))
    maclar = sorgu("""
        SELECT s.*, m.mac_zamani, m.kazanan_id, tr.turnuva_adi,
               t1.kisa_ad AS t1, t2.kisa_ad AS t2, m.takim1_skor, m.takim2_skor
          FROM oyuncu_mac_istatistik s
          JOIN mac m ON m.mac_id = s.mac_id
          JOIN turnuva tr ON tr.turnuva_id = m.turnuva_id
          JOIN takim t1 ON t1.takim_id = m.takim1_id
          JOIN takim t2 ON t2.takim_id = m.takim2_id
         WHERE s.oyuncu_id = %s ORDER BY m.mac_zamani DESC""", (oyuncu_id,))
    takimlar = sorgu("""SELECT t.takim_id, t.takim_adi, o.oyun_adi FROM takim t
                        JOIN oyun o ON o.oyun_id = t.oyun_id ORDER BY t.takim_adi""")
    return render_template("oyuncu_detay.html", oyuncu=oyuncu, istatistik=istatistik,
                           kadro=kadro, maclar=maclar, takimlar=takimlar)


@app.route("/oyuncular/<int:oyuncu_id>/duzenle", methods=["GET", "POST"])
def oyuncu_duzenle(oyuncu_id):
    if request.method == "POST":
        try:
            calistir("CALL oyuncu_guncelle(%s, %s, %s, %s, %s, %s, %s, %s)",
                     (oyuncu_id, *oyuncu_form_degerleri(request.form)))
            flash("Oyuncu bilgileri güncellendi.", "basari")
            return redirect(url_for("oyuncu_detay", oyuncu_id=oyuncu_id))
        except psycopg.Error as e:
            flash(hata_mesaji(e), "hata")
        deger = request.form
    else:
        deger = tek("""SELECT k.*, o.takma_ad, o.elo_puani FROM oyuncu o
                       JOIN kisi k ON k.kisi_id = o.kisi_id WHERE o.kisi_id = %s""", (oyuncu_id,))
        if not deger:
            flash("Oyuncu bulunamadı.", "hata")
            return redirect(url_for("oyuncular"))
        deger = {**deger, "dogum_tarihi": deger["dogum_tarihi"].isoformat()}
    return render_template("oyuncu_form.html", deger=deger, ulkeler=ulkeler(),
                           oyuncu_id=oyuncu_id, yeni=False)


@app.route("/oyuncular/<int:oyuncu_id>/sil", methods=["POST"])
def oyuncu_sil(oyuncu_id):
    try:
        silinen = calistir("""DELETE FROM kisi WHERE kisi_id = %s AND kisi_turu = 'O'
                              RETURNING ad || ' ' || soyad AS ad""", (oyuncu_id,))
        if silinen:
            flash(f"{silinen['ad']} silindi.", "basari")
        else:
            flash("Silinecek oyuncu bulunamadı.", "hata")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("oyuncular"))


@app.route("/oyuncular/<int:oyuncu_id>/transfer", methods=["POST"])
def oyuncu_transfer(oyuncu_id):
    takim_id = bos_ise_none(request.form.get("takim_id"))
    try:
        calistir("CALL oyuncu_transfer(%s, %s, %s)",
                 (oyuncu_id, int(takim_id) if takim_id else None, request.form.get("rol") or "Oyuncu"))
        flash("Transfer işlemi tamamlandı." if takim_id else "Oyuncu serbest bırakıldı.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(donus_adresi(url_for("oyuncu_detay", oyuncu_id=oyuncu_id)))


# ---------------------------------------------------------------------
#  TAKIMLAR
# ---------------------------------------------------------------------
def takim_form_secenekleri(takim_id=None):
    return {
        "oyunlar": sorgu("SELECT oyun_id, oyun_adi FROM oyun ORDER BY oyun_adi"),
        "ulkeler": ulkeler(),
        # takımı olmayan koçlar (+ düzenlenen takımın mevcut koçu)
        "koclar": sorgu("""
            SELECT k.kisi_id, ki.ad || ' ' || ki.soyad AS ad, k.deneyim_yili
              FROM koc k JOIN kisi ki ON ki.kisi_id = k.kisi_id
             WHERE NOT EXISTS (SELECT 1 FROM takim t WHERE t.koc_id = k.kisi_id
                                                       AND t.takim_id IS DISTINCT FROM %s)
```

```
            ORDER BY ad""", (takim_id,)),
    }


def takim_form_degerleri(form):
    koc = bos_ise_none(form.get("koc_id"))
    return (form["takim_adi"].strip(), form["kisa_ad"].strip().upper(), int(form["oyun_id"]),
            int(form["ulke_id"]), int(koc) if koc else None, form["kurulus_tarihi"])


@app.route("/takimlar")
def takimlar():
    aranan = request.args.get("q", "").strip()
    oyun_id = bos_ise_none(request.args.get("oyun_id"))
    kayitlar = sorgu("""
        SELECT t.takim_id, t.takim_adi, t.kisa_ad, o.oyun_adi, u.ulke_adi,
               ki.ad || ' ' || ki.soyad AS koc,
               (SELECT count(*) FROM takim_oyuncu ko
                 WHERE ko.takim_id = t.takim_id AND ko.ayrilma_tarihi IS NULL) AS oyuncu_sayisi,
               (SELECT count(*) FROM odul od WHERE od.takim_id = t.takim_id) AS odul_sayisi
          FROM takim t
          JOIN oyun o ON o.oyun_id = t.oyun_id
          JOIN ulke u ON u.ulke_id = t.ulke_id
          LEFT JOIN kisi ki ON ki.kisi_id = t.koc_id
         WHERE (t.takim_adi ILIKE %(q)s OR t.kisa_ad ILIKE %(q)s)
           AND (%(oyun)s::INT IS NULL OR t.oyun_id = %(oyun)s::INT)
         ORDER BY t.takim_adi""", {"q": f"%{aranan}%", "oyun": oyun_id})
    oyunlar = sorgu("SELECT oyun_id, oyun_adi FROM oyun ORDER BY oyun_adi")
    return render_template("takimlar.html", kayitlar=kayitlar, aranan=aranan,
                           oyunlar=oyunlar, secili_oyun=oyun_id)


@app.route("/takimlar/yeni", methods=["GET", "POST"])
def takim_ekle():
    if request.method == "POST":
        try:
            yeni = calistir("""INSERT INTO takim (takim_adi, kisa_ad, oyun_id, ulke_id, koc_id, kurulus_tarihi)
                               VALUES (%s, %s, %s, %s, %s, %s) RETURNING takim_id""",
                            takim_form_degerleri(request.form))
            flash("Takım eklendi.", "basari")
            return redirect(url_for("takim_detay", takim_id=yeni["takim_id"]))
        except psycopg.Error as e:
            flash(hata_mesaji(e), "hata")
    return render_template("takim_form.html", deger=request.form, yeni=True, **takim_form_secenekleri())


@app.route("/takimlar/<int:takim_id>")
def takim_detay(takim_id):
    takim = tek("""
        SELECT t.*, o.oyun_adi, o.takim_boyutu, u.ulke_adi, ki.ad || ' ' || ki.soyad AS koc,
               k.deneyim_yili
          FROM takim t JOIN oyun o ON o.oyun_id = t.oyun_id
          JOIN ulke u ON u.ulke_id = t.ulke_id
          LEFT JOIN koc k ON k.kisi_id = t.koc_id
          LEFT JOIN kisi ki ON ki.kisi_id = t.koc_id
         WHERE t.takim_id = %s""", (takim_id,))
    if not takim:
        flash("Takım bulunamadı.", "hata")
        return redirect(url_for("takimlar"))
    kadro = sorgu("""
        SELECT ko.*, o.takma_ad, ki.ad || ' ' || ki.soyad AS ad_soyad, o.elo_puani
          FROM takim_oyuncu ko JOIN oyuncu o ON o.kisi_id = ko.oyuncu_id
          JOIN kisi ki ON ki.kisi_id = ko.oyuncu_id
         WHERE ko.takim_id = %s
         ORDER BY ko.ayrilma_tarihi NULLS FIRST, ko.kaptan DESC, ko.katilma_tarihi""", (takim_id,))
    sponsorlar = sorgu("""
        SELECT s.sponsor_adi, s.sektor, ts.baslangic, ts.bitis, ts.yillik_tutar
          FROM takim_sponsor ts JOIN sponsor s ON s.sponsor_id = ts.sponsor_id
         WHERE ts.takim_id = %s ORDER BY ts.yillik_tutar DESC""", (takim_id,))
    turnuvalar = sorgu("""
        SELECT tr.turnuva_id, tr.turnuva_adi, tr.durum, tk.galibiyet, tk.maglubiyet, tk.puan, tk.sira,
               od.tutar AS odul
          FROM turnuva_katilim tk JOIN turnuva tr ON tr.turnuva_id = tk.turnuva_id
          LEFT JOIN odul od ON od.turnuva_id = tk.turnuva_id AND od.takim_id = tk.takim_id
         WHERE tk.takim_id = %s ORDER BY tr.baslangic_tarihi DESC""", (takim_id,))
    serbest_oyuncular = sorgu("""
        SELECT o.kisi_id, o.takma_ad FROM oyuncu o
```

```
         WHERE NOT EXISTS (SELECT 1 FROM takim_oyuncu ko
                           WHERE ko.oyuncu_id = o.kisi_id AND ko.ayrilma_tarihi IS NULL)
         ORDER BY o.takma_ad""")
    return render_template("takim_detay.html", takim=takim, kadro=kadro, sponsorlar=sponsorlar,
                           turnuvalar=turnuvalar, serbest_oyuncular=serbest_oyuncular)


@app.route("/takimlar/<int:takim_id>/duzenle", methods=["GET", "POST"])
def takim_duzenle(takim_id):
    if request.method == "POST":
        try:
            calistir("""UPDATE takim SET takim_adi = %s, kisa_ad = %s, oyun_id = %s, ulke_id = %s,
                                koc_id = %s, kurulus_tarihi = %s
                        WHERE takim_id = %s""", (*takim_form_degerleri(request.form), takim_id))
            flash("Takım bilgileri güncellendi.", "basari")
            return redirect(url_for("takim_detay", takim_id=takim_id))
        except psycopg.Error as e:
            flash(hata_mesaji(e), "hata")
        deger = request.form
    else:
        deger = tek("SELECT * FROM takim WHERE takim_id = %s", (takim_id,))
        if not deger:
            return redirect(url_for("takimlar"))
        deger = {**deger, "kurulus_tarihi": deger["kurulus_tarihi"].isoformat()}
    return render_template("takim_form.html", deger=deger, yeni=False, takim_id=takim_id,
                           **takim_form_secenekleri(takim_id))


@app.route("/takimlar/<int:takim_id>/sil", methods=["POST"])
def takim_sil(takim_id):
    try:
        silinen = calistir("DELETE FROM takim WHERE takim_id = %s RETURNING takim_adi", (takim_id,))
        if silinen:
            flash(f"{silinen['takim_adi']} silindi.", "basari")
            return redirect(url_for("takimlar"))
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("takim_detay", takim_id=takim_id))


@app.route("/takimlar/<int:takim_id>/kadro", methods=["POST"])
def kadro_ekle(takim_id):
    try:
        calistir("CALL oyuncu_transfer(%s, %s, %s)",
                 (int(request.form["oyuncu_id"]), takim_id, request.form.get("rol") or "Oyuncu"))
        flash("Oyuncu kadroya eklendi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("takim_detay", takim_id=takim_id))


@app.route("/takimlar/<int:takim_id>/kaptan/<int:kadro_id>", methods=["POST"])
def kaptan_yap(takim_id, kadro_id):
    try:
        calistir("UPDATE takim_oyuncu SET kaptan = TRUE WHERE kadro_id = %s AND takim_id = %s",
                 (kadro_id, takim_id))
        flash("Kaptan değiştirildi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("takim_detay", takim_id=takim_id))


# ---------------------------------------------------------------------
#  TURNUVALAR
# ---------------------------------------------------------------------
TURNUVA_DETAY_SQL = """
    SELECT t.*, o.oyun_adi, o.takim_boyutu,
           ot.yayin_platformu, ot.sunucu_bolgesi,
           st.mekan_id, st.bilet_fiyati, m.mekan_adi, m.sehir, m.kapasite,
           (SELECT count(*) FROM turnuva_katilim tk WHERE tk.turnuva_id = t.turnuva_id) AS takim_sayisi
      FROM turnuva t
      JOIN oyun o ON o.oyun_id = t.oyun_id
      LEFT JOIN online_turnuva ot ON ot.turnuva_id = t.turnuva_id
      LEFT JOIN salon_turnuva  st ON st.turnuva_id = t.turnuva_id
      LEFT JOIN mekan m           ON m.mekan_id   = st.mekan_id
"""
```

```
def turnuva_form_secenekleri():
    return {
        "oyunlar": sorgu("SELECT oyun_id, oyun_adi FROM oyun ORDER BY oyun_adi"),
        "mekanlar": sorgu("SELECT mekan_id, mekan_adi, sehir, kapasite FROM mekan ORDER BY mekan_adi"),
    }


@app.route("/turnuvalar")
def turnuvalar():
    aranan = request.args.get("q", "").strip()
    durum = bos_ise_none(request.args.get("durum"))
    kayitlar = sorgu(TURNUVA_DETAY_SQL + """
         WHERE t.turnuva_adi ILIKE %(q)s
           AND (%(durum)s::VARCHAR IS NULL OR t.durum = %(durum)s::VARCHAR)
         ORDER BY t.baslangic_tarihi DESC""", {"q": f"%{aranan}%", "durum": durum})
    return render_template("turnuvalar.html", kayitlar=kayitlar, aranan=aranan, secili_durum=durum)


@app.route("/turnuvalar/yeni", methods=["GET", "POST"])
def turnuva_ekle():
    if request.method == "POST":
        f = request.form
        try:
            mekan = bos_ise_none(f.get("mekan_id"))
            yeni = calistir(
                "SELECT turnuva_olustur(%s, %s, %s, %s, %s, %s::SMALLINT, %s, %s, %s, %s, %s) AS id",
                (f["turnuva_adi"].strip(), int(f["oyun_id"]), f["baslangic_tarihi"], f["bitis_tarihi"],
                 f["odul_havuzu"] or 0, int(f["max_takim"]), f["turnuva_turu"],
                 bos_ise_none(f.get("yayin_platformu")), bos_ise_none(f.get("sunucu_bolgesi")),
                 int(mekan) if mekan else None, f.get("bilet_fiyati") or 0))
            flash("Turnuva oluşturuldu.", "basari")
            return redirect(url_for("turnuva_detay", turnuva_id=yeni["id"]))
        except psycopg.Error as e:
            flash(hata_mesaji(e), "hata")
    return render_template("turnuva_form.html", deger=request.form, yeni=True, **turnuva_form_secenekleri())


@app.route("/turnuvalar/<int:turnuva_id>")
def turnuva_detay(turnuva_id):
    turnuva = tek(TURNUVA_DETAY_SQL + " WHERE t.turnuva_id = %s", (turnuva_id,))
    if not turnuva:
        flash("Turnuva bulunamadı.", "hata")
        return redirect(url_for("turnuvalar"))
    puan_tablosu = sorgu("SELECT * FROM puan_tablosu(%s)", (turnuva_id,))
    maclar = sorgu("""
        SELECT m.*, a.asama_adi, t1.takim_adi AS t1_adi, t2.takim_adi AS t2_adi,
               ki.ad || ' ' || ki.soyad AS hakem
          FROM mac m JOIN asama a ON a.asama_id = m.asama_id
          JOIN takim t1 ON t1.takim_id = m.takim1_id
          JOIN takim t2 ON t2.takim_id = m.takim2_id
          LEFT JOIN kisi ki ON ki.kisi_id = m.hakem_id
         WHERE m.turnuva_id = %s ORDER BY m.mac_zamani""", (turnuva_id,))
    oduller = sorgu("""
        SELECT od.*, t.takim_adi FROM odul od LEFT JOIN takim t ON t.takim_id = od.takim_id
         WHERE od.turnuva_id = %s ORDER BY od.derece""", (turnuva_id,))
    kayit_olabilir = sorgu("""
        SELECT t.takim_id, t.takim_adi FROM takim t
         WHERE t.oyun_id = %s
           AND NOT EXISTS (SELECT 1 FROM turnuva_katilim tk
                           WHERE tk.turnuva_id = %s AND tk.takim_id = t.takim_id)
         ORDER BY t.takim_adi""", (turnuva["oyun_id"], turnuva_id))
    return render_template("turnuva_detay.html", turnuva=turnuva, puan_tablosu=puan_tablosu,
                           maclar=maclar, oduller=oduller, kayit_olabilir=kayit_olabilir)


@app.route("/turnuvalar/<int:turnuva_id>/duzenle", methods=["GET", "POST"])
def turnuva_duzenle(turnuva_id):
    turnuva = tek(TURNUVA_DETAY_SQL + " WHERE t.turnuva_id = %s", (turnuva_id,))
    if not turnuva:
        return redirect(url_for("turnuvalar"))
    if request.method == "POST":
        f = request.form
        db = baglanti()
        try:
            db.execute("""UPDATE turnuva SET turnuva_adi = %s, baslangic_tarihi = %s, bitis_tarihi = %s,
                                  odul_havuzu = %s, max_takim = %s
```

```
                            WHERE turnuva_id = %s""",
                        (f["turnuva_adi"].strip(), f["baslangic_tarihi"], f["bitis_tarihi"],
                         f["odul_havuzu"] or 0, int(f["max_takim"]), turnuva_id))
            if turnuva["turnuva_turu"] == "O":
                db.execute("""UPDATE online_turnuva SET yayin_platformu = %s, sunucu_bolgesi = %s
                              WHERE turnuva_id = %s""",
                           (f["yayin_platformu"].strip(), f["sunucu_bolgesi"].strip(), turnuva_id))
            else:
                db.execute("""UPDATE salon_turnuva SET mekan_id = %s, bilet_fiyati = %s
                              WHERE turnuva_id = %s""",
                           (int(f["mekan_id"]), f.get("bilet_fiyati") or 0, turnuva_id))
            db.commit()
            flash("Turnuva güncellendi.", "basari")
            return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))
        except psycopg.Error as e:
            db.rollback()
            flash(hata_mesaji(e), "hata")
        deger = {**turnuva, **f.to_dict()}
    else:
        deger = {**turnuva,
                 "baslangic_tarihi": turnuva["baslangic_tarihi"].isoformat(),
                 "bitis_tarihi": turnuva["bitis_tarihi"].isoformat()}
    return render_template("turnuva_form.html", deger=deger, yeni=False, turnuva_id=turnuva_id,
                           **turnuva_form_secenekleri())


@app.route("/turnuvalar/<int:turnuva_id>/sil", methods=["POST"])
def turnuva_sil(turnuva_id):
    try:
        silinen = calistir("DELETE FROM turnuva WHERE turnuva_id = %s RETURNING turnuva_adi", (turnuva_id,))
        if silinen:
            flash(f"{silinen['turnuva_adi']} silindi.", "basari")
            return redirect(url_for("turnuvalar"))
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/katilim", methods=["POST"])
def katilim_ekle(turnuva_id):
    try:
        calistir("INSERT INTO turnuva_katilim (turnuva_id, takim_id) VALUES (%s, %s)",
                 (turnuva_id, int(request.form["takim_id"])))
        flash("Takım turnuvaya kaydedildi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/katilim/<int:takim_id>/sil", methods=["POST"])
def katilim_sil(turnuva_id, takim_id):
    try:
        durum = tek("SELECT durum FROM turnuva WHERE turnuva_id = %s", (turnuva_id,))
        if durum and durum["durum"] != "Kayıt Açık":
            flash("Kayıt dönemi bittiği için takım turnuvadan çıkarılamaz.", "hata")
        else:
            calistir("DELETE FROM turnuva_katilim WHERE turnuva_id = %s AND takim_id = %s",
                     (turnuva_id, takim_id))
            flash("Takımın turnuva kaydı silindi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/fikstur", methods=["POST"])
def fikstur(turnuva_id):
    try:
        sonuc = calistir("SELECT fikstur_olustur(%s, %s, %s::INTERVAL) AS adet",
                         (turnuva_id, request.form["ilk_mac"], request.form.get("aralik") or "1 day"))
        flash(f"Fikstür oluşturuldu: {sonuc['adet']} maç planlandı. Turnuva başladı.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/tamamla", methods=["POST"])
def turnuva_tamamla(turnuva_id):
```

```
    try:
        calistir("UPDATE turnuva SET durum = 'Tamamlandı' WHERE turnuva_id = %s", (turnuva_id,))
        flash("Turnuva tamamlandı; sıralama ve ödüller belirlendi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/odul", methods=["POST"])
def odul_ekle(turnuva_id):
    try:
        calistir("INSERT INTO odul (turnuva_id, derece, tutar) VALUES (%s, %s, %s)",
                 (turnuva_id, int(request.form["derece"]), request.form["tutar"]))
        flash("Ödül eklendi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


@app.route("/turnuvalar/<int:turnuva_id>/odul/<int:odul_id>/sil", methods=["POST"])
def odul_sil(turnuva_id, odul_id):
    try:
        calistir("DELETE FROM odul WHERE odul_id = %s AND turnuva_id = %s", (odul_id, turnuva_id))
        flash("Ödül silindi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("turnuva_detay", turnuva_id=turnuva_id))


# ---------------------------------------------------------------------
#  MAÇLAR
# ---------------------------------------------------------------------
@app.route("/maclar/<int:mac_id>")
def mac_detay(mac_id):
    mac = tek("""
        SELECT m.*, a.asama_adi, tr.turnuva_adi, t1.takim_adi AS t1_adi, t2.takim_adi AS t2_adi,
               ki.ad || ' ' || ki.soyad AS hakem
          FROM mac m JOIN asama a ON a.asama_id = m.asama_id
          JOIN turnuva tr ON tr.turnuva_id = m.turnuva_id
          JOIN takim t1 ON t1.takim_id = m.takim1_id
          JOIN takim t2 ON t2.takim_id = m.takim2_id
          LEFT JOIN kisi ki ON ki.kisi_id = m.hakem_id
         WHERE m.mac_id = %s""", (mac_id,))
    if not mac:
        flash("Maç bulunamadı.", "hata")
        return redirect(url_for("turnuvalar"))
    istatistikler = sorgu("""
        SELECT s.*, o.takma_ad, t.kisa_ad,
               round((s.oldurme + s.asist)::NUMERIC / GREATEST(s.olum, 1), 2) AS kda
          FROM oyuncu_mac_istatistik s
          JOIN oyuncu o ON o.kisi_id = s.oyuncu_id
          JOIN takim t ON t.takim_id = s.takim_id
         WHERE s.mac_id = %s
         ORDER BY s.takim_id = %s DESC, s.oldurme DESC""", (mac_id, mac["takim1_id"]))
    oyuncular = sorgu("""
        SELECT o.kisi_id, o.takma_ad, t.kisa_ad FROM takim_oyuncu ko
          JOIN oyuncu o ON o.kisi_id = ko.oyuncu_id
          JOIN takim t ON t.takim_id = ko.takim_id
         WHERE ko.takim_id IN (%s, %s) AND ko.ayrilma_tarihi IS NULL
           AND NOT EXISTS (SELECT 1 FROM oyuncu_mac_istatistik s
                           WHERE s.mac_id = %s AND s.oyuncu_id = o.kisi_id)
         ORDER BY t.kisa_ad, o.takma_ad""", (mac["takim1_id"], mac["takim2_id"], mac_id))
    return render_template("mac_detay.html", mac=mac, istatistikler=istatistikler, oyuncular=oyuncular)


@app.route("/maclar/<int:mac_id>/sonuc", methods=["POST"])
def mac_sonuc(mac_id):
    try:
        calistir("CALL mac_sonucu_gir(%s, %s::SMALLINT, %s::SMALLINT)",
                 (mac_id, int(request.form["skor1"]), int(request.form["skor2"])))
        flash("Maç sonucu kaydedildi; puan tablosu güncellendi.", "basari")
    except (psycopg.Error, ValueError) as e:
        flash(hata_mesaji(e) if isinstance(e, psycopg.Error) else "Skorlar sayı olmalıdır.", "hata")
    return redirect(donus_adresi(url_for("mac_detay", mac_id=mac_id)))


@app.route("/maclar/<int:mac_id>/istatistik", methods=["POST"])
```

```
def istatistik_ekle(mac_id):
    f = request.form
    try:
        calistir("""INSERT INTO oyuncu_mac_istatistik (mac_id, oyuncu_id, oldurme, olum, asist, mvp)
                    VALUES (%s, %s, %s, %s, %s, %s)""",
                 (mac_id, int(f["oyuncu_id"]), int(f["oldurme"]), int(f["olum"]), int(f["asist"]),
                  "mvp" in f))
        flash("Oyuncu istatistiği eklendi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("mac_detay", mac_id=mac_id))


@app.route("/maclar/<int:mac_id>/istatistik/<int:istatistik_id>/sil", methods=["POST"])
def istatistik_sil(mac_id, istatistik_id):
    try:
        calistir("DELETE FROM oyuncu_mac_istatistik WHERE istatistik_id = %s AND mac_id = %s",
                 (istatistik_id, mac_id))
        flash("İstatistik silindi.", "basari")
    except psycopg.Error as e:
        flash(hata_mesaji(e), "hata")
    return redirect(url_for("mac_detay", mac_id=mac_id))


# ---------------------------------------------------------------------
#  İŞLEM GÜNLÜĞÜ
# ---------------------------------------------------------------------
@app.route("/islem-gunlugu")
def islem_gunlugu():
    tablo = bos_ise_none(request.args.get("tablo"))
    kayitlar = sorgu("""
        SELECT * FROM islem_log
         WHERE %(tablo)s::VARCHAR IS NULL OR tablo_adi = %(tablo)s::VARCHAR
         ORDER BY log_id DESC LIMIT 100""", {"tablo": tablo})
    return render_template("islem_gunlugu.html", kayitlar=kayitlar, secili_tablo=tablo)


if __name__ == "__main__":
    app.run(debug=True)
```

templates/base.html

```
<!doctype html>
<html lang="tr">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block baslik %}E-spor Turnuva{% endblock %} · E-spor Turnuva Yönetimi</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<header class="ust-menu">
    <div class="kap">
        <a class="logo" href="{{ url_for('ana_sayfa') }}">E-SPOR<span>ARENA</span></a>
        <nav>
            {% set uc = request.endpoint or '' %}
            <a href="{{ url_for('ana_sayfa') }}" class="{{ 'aktif' if uc == 'ana_sayfa' }}">Ana Sayfa</a>
            <a href="{{ url_for('oyuncular') }}" class="{{ 'aktif' if uc.startswith('oyuncu') }}">Oyuncular</a>
            <a href="{{ url_for('takimlar') }}" class="{{ 'aktif' if uc.startswith('takim') or uc == 'kaptan_yap'
}}">Takımlar</a>
            <a href="{{ url_for('turnuvalar') }}" class="{{ 'aktif' if uc.startswith('turnuva') or uc.startswith('mac')
}}">Turnuvalar</a>
            <a href="{{ url_for('islem_gunlugu') }}" class="{{ 'aktif' if uc == 'islem_gunlugu' }}">İşlem Günlüğü</a>
        </nav>
    </div>
</header>

<main>
    {% with mesajlar = get_flashed_messages(with_categories=true) %}
        {% for tur, mesaj in mesajlar %}
            <div class="bildirim {{ tur }}">{{ mesaj }}</div>
        {% endfor %}
    {% endwith %}
    {% block icerik %}{% endblock %}
</main>
```

```
<footer>BSM 211 Veritabanı Yönetim Sistemleri · E-spor Turnuva Yönetim Sistemi · PostgreSQL + Flask</footer>
</body>
</html>
```

templates/_makrolar.html

```
{% macro durum_rozeti(durum) -%}
    {%- set renk = {'Kayıt Açık': 'mavi', 'Devam Ediyor': 'sari', 'Tamamlandı': 'yesil',
                    'Planlandı': 'gri', 'Oynandı': 'yesil', 'İptal': 'gri'} -%}
    <span class="rozet {{ renk.get(durum, '') }}">{{ durum }}</span>
{%- endmacro %}

{% macro tur_rozeti(tur) -%}
    {%- if tur == 'O' -%}<span class="rozet mor">Online</span>
    {%- else -%}<span class="rozet mavi">Salon</span>{%- endif -%}
{%- endmacro %}

{% macro sil_butonu(adres, soru, yazi='Sil') -%}
    <form class="satir-ici" method="post" action="{{ adres }}" onsubmit='return confirm({{ soru|tojson }});'>
        <button class="btn btn-tehlike btn-kucuk" type="submit">{{ yazi }}</button>
    </form>
{%- endmacro %}
```

templates/index.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import durum_rozeti %}
{% block baslik %}Ana Sayfa{% endblock %}
{% block icerik %}
<h1>Kontrol Paneli</h1>
<p class="alt-baslik">Turnuvalar, takımlar ve oyuncular için genel durum.</p>

<div class="kartlar">
    <div class="kart"><div class="etiket">Oyuncu</div><div class="deger">{{ sayilar.oyuncu }}</div></div>
    <div class="kart"><div class="etiket">Takım</div><div class="deger">{{ sayilar.takim }}</div></div>
    <div class="kart"><div class="etiket">Turnuva</div><div class="deger">{{ sayilar.turnuva }}</div></div>
    <div class="kart"><div class="etiket">Oynanan Maç</div><div class="deger">{{ sayilar.mac }}</div></div>
    <div class="kart"><div class="etiket">Toplam Ödül Havuzu</div><div class="deger">{{ sayilar.odul|para }}</div></div>
</div>

<div class="izgara izgara-2">
    <section class="panel">
        <h2>Aktif Turnuvalar</h2>
        <table>
            <thead><tr><th>Turnuva</th><th>Oyun</th><th>Durum</th><th class="sayi">Ödül</th></tr></thead>
            <tbody>
            {% for t in devam_eden %}
                <tr>
                    <td><a href="{{ url_for('turnuva_detay', turnuva_id=t.turnuva_id) }}">{{ t.turnuva_adi }}</a></td>
                    <td>{{ t.oyun_adi }}</td>
                    <td>{{ durum_rozeti(t.durum) }}</td>
                    <td class="sayi">{{ t.odul_havuzu|para }}</td>
                </tr>
            {% else %}
                <tr><td colspan="4" class="bos">Aktif turnuva yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>
    </section>

    <section class="panel">
        <h2>En Yüksek ELO Puanlı Oyuncular</h2>
        <table>
            <thead><tr><th>#</th><th>Oyuncu</th><th>Takım</th><th class="sayi">ELO</th></tr></thead>
            <tbody>
            {% for o in en_iyiler %}
                <tr>
                    <td>{{ loop.index }}</td>
                    <td><a href="{{ url_for('oyuncu_detay', oyuncu_id=o.oyuncu_id) }}">{{ o.takma_ad }}</a>
                        <span class="soluk">· {{ o.ad_soyad }}</span></td>
                    <td>{{ o.takim_adi or '-' }}</td>
                    <td class="sayi">{{ o.elo_puani }}</td>
```

```
                </tr>
            {% endfor %}
            </tbody>
        </table>
    </section>

    <section class="panel">
        <h2>Son Maç Sonuçları</h2>
        <table>
            <tbody>
            {% for m in son_maclar %}
                <tr>
                    <td class="soluk">{{ m.mac_zamani|tarih }}</td>
                    <td class="{{ 'kazanan' if m.kazanan_id == m.takim1_id }}">{{ m.t1 }}</td>
                    <td><a href="{{ url_for('mac_detay', mac_id=m.mac_id) }}">{{ m.takim1_skor }} - {{ m.takim2_skor }}
</a></td>
                    <td class="{{ 'kazanan' if m.kazanan_id == m.takim2_id }}">{{ m.t2 }}</td>
                    <td class="soluk">{{ m.turnuva_adi }}</td>
                </tr>
            {% else %}
                <tr><td class="bos">Henüz oynanmış maç yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>
    </section>

    <section class="panel">
        <h2>Yaklaşan Maçlar</h2>
        <table>
            <tbody>
            {% for m in yaklasan %}
                <tr>
                    <td class="soluk">{{ m.mac_zamani|tarih }}</td>
                    <td><a href="{{ url_for('mac_detay', mac_id=m.mac_id) }}">{{ m.t1 }} vs {{ m.t2 }}</a></td>
                    <td class="soluk">{{ m.turnuva_adi }}</td>
                </tr>
            {% else %}
                <tr><td class="bos">Planlanmış maç yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>
    </section>
</div>
{% endblock %}
```

templates/oyuncular.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu %}
{% block baslik %}Oyuncular{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
    <div>
        <h1>Oyuncular</h1>
        <p class="alt-baslik">Arama <code>oyuncu_ara()</code> fonksiyonu ile yapılır.</p>
    </div>
    <a class="btn" href="{{ url_for('oyuncu_ekle') }}">+ Yeni Oyuncu</a>
</div>

<section class="panel">
    <form class="arama" method="get">
        <input type="text" name="q" value="{{ aranan }}" placeholder="Takma ad, ad, soyad veya takım adı ile ara...">
        <button class="btn" type="submit">Ara</button>
        {% if aranan %}<a class="btn btn-ikincil" href="{{ url_for('oyuncular') }}">Temizle</a>{% endif %}
    </form>

    <p class="soluk">{{ kayitlar|length }} oyuncu bulundu{% if aranan %} (aranan: "{{ aranan }}"){% endif %}.</p>

    <div class="tablo-kap">
    <table>
        <thead>
        <tr><th>Takma Ad</th><th>Ad Soyad</th><th>Ülke</th><th class="sayi">Yaş</th><th>Takım</th>
            <th>Oyun</th><th class="sayi">ELO</th><th></th></tr>
        </thead>
        <tbody>
```

```
        {% for o in kayitlar %}
            <tr>
                <td><a href="{{ url_for('oyuncu_detay', oyuncu_id=o.oyuncu_id) }}"><strong>{{ o.takma_ad }}</strong></a>
</td>
                <td>{{ o.ad_soyad }}</td>
                <td>{{ o.ulke_adi }}</td>
                <td class="sayi">{{ o.yas }}</td>
                <td>{{ o.takim_adi or '' }}{% if not o.takim_adi %}<span class="rozet gri">Serbest</span>{% endif %}</td>
                <td>{{ o.oyun_adi or '-' }}</td>
                <td class="sayi">{{ o.elo_puani }}</td>
                <td style="white-space:nowrap">
                    <a class="btn btn-ikincil btn-kucuk" href="{{ url_for('oyuncu_duzenle', oyuncu_id=o.oyuncu_id)
}}">Düzenle</a>
                    {{ sil_butonu(url_for('oyuncu_sil', oyuncu_id=o.oyuncu_id), o.takma_ad ~ ' silinsin mi?') }}
                </td>
            </tr>
        {% else %}
            <tr><td colspan="8" class="bos">Aramaya uygun oyuncu bulunamadı.</td></tr>
        {% endfor %}
        </tbody>
    </table>
    </div>
</section>
{% endblock %}
```

templates/oyuncu_form.html

```
{% extends "base.html" %}
{% block baslik %}{{ 'Yeni Oyuncu' if yeni else 'Oyuncu Düzenle' }}{% endblock %}
{% block icerik %}
<h1>{{ 'Yeni Oyuncu Ekle' if yeni else 'Oyuncu Bilgilerini Güncelle' }}</h1>
<p class="alt-baslik">
    {% if yeni %}Kayıt <code>oyuncu_ekle()</code> fonksiyonu ile önce <em>kisi</em>, sonra <em>oyuncu</em> tablosuna
eklenir.
    {% else %}Güncelleme <code>oyuncu_guncelle</code> saklı yordamı ile yapılır.{% endif %}
</p>

<form class="panel" method="post">
    <div class="form-izgara">
        <fieldset class="tam">
            <legend>Kişi Bilgileri (kisi)</legend>
            <div class="form-izgara">
                <div><label>Ad</label><input name="ad" required maxlength="40" value="{{ deger.get('ad', '') }}"></div>
                <div><label>Soyad</label><input name="soyad" required maxlength="40" value="{{ deger.get('soyad', '') }}">
</div>
                <div><label>E-posta</label><input name="eposta" type="email" required maxlength="100" value="{{
deger.get('eposta', '') }}"></div>
                <div><label>Doğum Tarihi</label><input name="dogum_tarihi" type="date" required value="{{
deger.get('dogum_tarihi', '') }}"></div>
                <div>
                    <label>Ülke</label>
                    <select name="ulke_id" required>
                        {% for u in ulkeler %}
                            <option value="{{ u.ulke_id }}" {{ 'selected' if deger.get('ulke_id')|string ==
u.ulke_id|string }}>{{ u.ulke_adi }}</option>
                        {% endfor %}
                    </select>
                </div>
            </div>
        </fieldset>

        <fieldset class="tam">
            <legend>Oyuncu Bilgileri (oyuncu)</legend>
            <div class="form-izgara">
                <div><label>Takma Ad</label><input name="takma_ad" required maxlength="30" value="{{ deger.get('takma_ad',
'') }}"></div>
                <div><label>ELO Puanı</label><input name="elo_puani" type="number" min="0" value="{{
deger.get('elo_puani', 1000) }}"></div>
            </div>
        </fieldset>

        {% if yeni %}
        <fieldset class="tam">
            <legend>Takım (isteğe bağlı)</legend>
            <div class="form-izgara">
```

```
                <div>
                    <label>Takım</label>
                    <select name="takim_id">
                        <option value="">— Serbest oyuncu —</option>
                        {% for t in takimlar %}
                            <option value="{{ t.takim_id }}" {{ 'selected' if deger.get('takim_id') == t.takim_id|string
}}>{{ t.takim_adi }} ({{ t.oyun_adi }})</option>
                        {% endfor %}
                    </select>
                </div>
                <div><label>Rol</label><input name="rol" maxlength="20" placeholder="Örn. Düellocu" value="{{
deger.get('rol', '') }}"></div>
            </div>
        </fieldset>
        {% endif %}
    </div>

    <div class="form-alt">
        <button class="btn" type="submit">{{ 'Ekle' if yeni else 'Güncelle' }}</button>
        <a class="btn btn-ikincil" href="{{ url_for('oyuncular') if yeni else url_for('oyuncu_detay', oyuncu_id=oyuncu_id)
}}">Vazgeç</a>
    </div>
</form>
{% endblock %}
```

templates/oyuncu_detay.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu %}
{% block baslik %}{{ oyuncu.takma_ad }}{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
    <div>
        <h1>{{ oyuncu.takma_ad }}</h1>
        <p class="alt-baslik">{{ oyuncu.ad }} {{ oyuncu.soyad }} · {{ oyuncu.ulke_adi }} · {{ oyuncu.yas }} yaşında</p>
    </div>
    <div>
        <a class="btn btn-ikincil" href="{{ url_for('oyuncu_duzenle', oyuncu_id=oyuncu.kisi_id) }}">Düzenle</a>
        {{ sil_butonu(url_for('oyuncu_sil', oyuncu_id=oyuncu.kisi_id), oyuncu.takma_ad ~ ' silinsin mi? İstatistikleri ve
kadro geçmişi de silinir.', 'Oyuncuyu Sil') }}
    </div>
</div>

<div class="kartlar">
    <div class="kart"><div class="etiket">ELO</div><div class="deger">{{ oyuncu.elo_puani }}</div></div>
    <div class="kart"><div class="etiket">Maç</div><div class="deger">{{ istatistik.mac_sayisi }}</div></div>
    <div class="kart"><div class="etiket">Ö / Ö / A</div><div class="deger">{{ istatistik.toplam_oldurme }} / {{
istatistik.toplam_olum }} / {{ istatistik.toplam_asist }}</div></div>
    <div class="kart"><div class="etiket">KDA</div><div class="deger">{{ istatistik.kda }}</div></div>
    <div class="kart"><div class="etiket">MVP</div><div class="deger">{{ istatistik.mvp_sayisi }}</div></div>
</div>
<p class="soluk" style="margin-top:-10px">İstatistikler <code>oyuncu_istatistik()</code> fonksiyonu ile hesaplanır.</p>

<div class="izgara izgara-2">
    <section class="panel">
        <h2>Kişisel Bilgiler</h2>
        <dl class="bilgi-listesi">
            <dt>E-posta</dt><dd>{{ oyuncu.eposta }}</dd>
            <dt>Doğum Tarihi</dt><dd>{{ oyuncu.dogum_tarihi|tarih }}</dd>
            <dt>Kayıt Tarihi</dt><dd>{{ oyuncu.kayit_tarihi|tarih }}</dd>
        </dl>
    </section>

    <section class="panel">
        <h2>Transfer</h2>
        <p class="soluk">Transfer <code>oyuncu_transfer</code> saklı yordamı ile yapılır; eski kadro kaydı kapatılır.</p>
        <form class="kucuk-form" method="post" action="{{ url_for('oyuncu_transfer', oyuncu_id=oyuncu.kisi_id) }}">
            <div>
                <label>Yeni Takım</label>
                <select name="takim_id">
                    <option value="">— Serbest bırak —</option>
                    {% for t in takimlar %}<option value="{{ t.takim_id }}">{{ t.takim_adi }} ({{ t.oyun_adi }})</option>
{% endfor %}
                </select>
            </div>
```

```
            <div><label>Rol</label><input name="rol" placeholder="Oyuncu"></div>
            <button class="btn" type="submit">Uygula</button>
        </form>
    </section>
</div>

<section class="panel">
    <h2>Kadro Geçmişi</h2>
    <table>
        <thead><tr><th>Takım</th><th>Rol</th><th>Katılma</th><th>Ayrılma</th><th></th></tr></thead>
        <tbody>
        {% for k in kadro %}
            <tr>
                <td><a href="{{ url_for('takim_detay', takim_id=k.takim_id) }}">{{ k.takim_adi }}</a></td>
                <td>{{ k.rol }} {% if k.kaptan %}<span class="rozet sari">Kaptan</span>{% endif %}</td>
                <td>{{ k.katilma_tarihi|tarih }}</td>
                <td>{{ k.ayrilma_tarihi|tarih }}</td>
                <td>{% if not k.ayrilma_tarihi %}<span class="rozet yesil">Aktif</span>{% endif %}</td>
            </tr>
        {% else %}
            <tr><td colspan="5" class="bos">Oyuncu hiçbir takımda oynamadı.</td></tr>
        {% endfor %}
        </tbody>
    </table>
</section>

<section class="panel">
    <h2>Maç İstatistikleri</h2>
    <table>
        <thead><tr><th>Tarih</th><th>Maç</th><th>Turnuva</th><th class="sayi">Öldürme</th><th class="sayi">Ölüm</th><th
class="sayi">Asist</th><th></th></tr></thead>
        <tbody>
        {% for m in maclar %}
            <tr>
                <td>{{ m.mac_zamani|tarih }}</td>
                <td><a href="{{ url_for('mac_detay', mac_id=m.mac_id) }}">{{ m.t1 }} {{ m.takim1_skor }} - {{
m.takim2_skor }} {{ m.t2 }}</a>
                    {% if m.kazanan_id == m.takim_id %}<span class="rozet yesil">G</span>{% else %}<span class="rozet
gri">M</span>{% endif %}</td>
                <td>{{ m.turnuva_adi }}</td>
                <td class="sayi">{{ m.oldurme }}</td>
                <td class="sayi">{{ m.olum }}</td>
                <td class="sayi">{{ m.asist }}</td>
                <td>{% if m.mvp %}<span class="rozet sari">MVP</span>{% endif %}</td>
            </tr>
        {% else %}
            <tr><td colspan="7" class="bos">İstatistik kaydı yok.</td></tr>
        {% endfor %}
        </tbody>
    </table>
</section>
{% endblock %}
```

templates/takimlar.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu %}
{% block baslik %}Takımlar{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
    <div>
        <h1>Takımlar</h1>
        <p class="alt-baslik">Takımları arayın, ekleyin, düzenleyin veya silin.</p>
    </div>
    <a class="btn" href="{{ url_for('takim_ekle') }}">+ Yeni Takım</a>
</div>

<section class="panel">
    <form class="arama" method="get">
        <input type="text" name="q" value="{{ aranan }}" placeholder="Takım adı veya kısa ad ile ara...">
        <select name="oyun_id" style="max-width:240px">
            <option value="">Tüm oyunlar</option>
            {% for o in oyunlar %}
                <option value="{{ o.oyun_id }}" {{ 'selected' if secili_oyun == o.oyun_id|string }}>{{ o.oyun_adi }}
</option>
```

```
            {% endfor %}
        </select>
        <button class="btn" type="submit">Ara</button>
        {% if aranan or secili_oyun %}<a class="btn btn-ikincil" href="{{ url_for('takimlar') }}">Temizle</a>{% endif %}
    </form>

    <div class="tablo-kap">
    <table>
        <thead><tr><th>Takım</th><th>Kısa Ad</th><th>Oyun</th><th>Ülke</th><th>Koç</th>
            <th class="sayi">Aktif Oyuncu</th><th class="sayi">Ödül</th><th></th></tr></thead>
        <tbody>
        {% for t in kayitlar %}
            <tr>
                <td><a href="{{ url_for('takim_detay', takim_id=t.takim_id) }}"><strong>{{ t.takim_adi }}</strong></a>
</td>
                <td><span class="rozet mor">{{ t.kisa_ad }}</span></td>
                <td>{{ t.oyun_adi }}</td>
                <td>{{ t.ulke_adi }}</td>
                <td>{{ t.koc or '-' }}</td>
                <td class="sayi">{{ t.oyuncu_sayisi }}</td>
                <td class="sayi">{{ t.odul_sayisi }}</td>
                <td style="white-space:nowrap">
                    <a class="btn btn-ikincil btn-kucuk" href="{{ url_for('takim_duzenle', takim_id=t.takim_id)
}}">Düzenle</a>
                    {{ sil_butonu(url_for('takim_sil', takim_id=t.takim_id), t.takim_adi ~ ' silinsin mi?') }}
                </td>
            </tr>
        {% else %}
            <tr><td colspan="8" class="bos">Takım bulunamadı.</td></tr>
        {% endfor %}
        </tbody>
    </table>
    </div>
</section>
{% endblock %}
```

templates/takim_form.html

```
{% extends "base.html" %}
{% block baslik %}{{ 'Yeni Takım' if yeni else 'Takım Düzenle' }}{% endblock %}
{% block icerik %}
<h1>{{ 'Yeni Takım Ekle' if yeni else 'Takım Bilgilerini Güncelle' }}</h1>
<p class="alt-baslik">Bir koç yalnızca bir takımı çalıştırabilir; listede yalnızca boştaki koçlar görünür.</p>

<form class="panel" method="post">
    <div class="form-izgara">
        <div><label>Takım Adı</label><input name="takim_adi" required maxlength="60" value="{{ deger.get('takim_adi', '')
}}"></div>
        <div><label>Kısa Ad (en fazla 5 karakter)</label><input name="kisa_ad" required maxlength="5" value="{{
deger.get('kisa_ad', '') }}"></div>
        <div>
            <label>Oyun</label>
            <select name="oyun_id" required>
                {% for o in oyunlar %}
                    <option value="{{ o.oyun_id }}" {{ 'selected' if deger.get('oyun_id')|string == o.oyun_id|string }}>{{
o.oyun_adi }}</option>
                {% endfor %}
            </select>
        </div>
        <div>
            <label>Ülke</label>
            <select name="ulke_id" required>
                {% for u in ulkeler %}
                    <option value="{{ u.ulke_id }}" {{ 'selected' if deger.get('ulke_id')|string == u.ulke_id|string }}>{{
u.ulke_adi }}</option>
                {% endfor %}
            </select>
        </div>
        <div>
            <label>Koç</label>
            <select name="koc_id">
                <option value="">— Koç yok —</option>
                {% for k in koclar %}
                    <option value="{{ k.kisi_id }}" {{ 'selected' if deger.get('koc_id')|string == k.kisi_id|string }}>{{
k.ad }} ({{ k.deneyim_yili }} yıl)</option>
```

```
                {% endfor %}
            </select>
        </div>
        <div><label>Kuruluş Tarihi</label><input name="kurulus_tarihi" type="date" required value="{{
deger.get('kurulus_tarihi', bugun.isoformat()) }}"></div>
    </div>
    <div class="form-alt">
        <button class="btn" type="submit">{{ 'Ekle' if yeni else 'Güncelle' }}</button>
        <a class="btn btn-ikincil" href="{{ url_for('takimlar') if yeni else url_for('takim_detay', takim_id=takim_id)
}}">Vazgeç</a>
    </div>
</form>
{% endblock %}
```

templates/takim_detay.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu, durum_rozeti %}
{% block baslik %}{{ takim.takim_adi }}{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
    <div>
        <h1>{{ takim.takim_adi }} <span class="rozet mor">{{ takim.kisa_ad }}</span></h1>
        <p class="alt-baslik">{{ takim.oyun_adi }} · {{ takim.ulke_adi }} · Kuruluş: {{ takim.kurulus_tarihi|tarih }}
            · Koç: {{ takim.koc or 'yok' }}{% if takim.koc %} ({{ takim.deneyim_yili }} yıl deneyim){% endif %}</p>
    </div>
    <div>
        <a class="btn btn-ikincil" href="{{ url_for('takim_duzenle', takim_id=takim.takim_id) }}">Düzenle</a>
        {{ sil_butonu(url_for('takim_sil', takim_id=takim.takim_id), takim.takim_adi ~ ' silinsin mi?', 'Takımı Sil') }}
    </div>
</div>

<section class="panel">
    <div class="baslik-satiri" style="margin-bottom:8px">
        <h2>Kadro</h2>
        <span class="soluk">Sahada {{ takim.takim_boyutu }} oyuncu · en fazla {{ takim.takim_boyutu + 2 }} aktif
oyuncu</span>
    </div>
    <table>
        <thead><tr><th>Oyuncu</th><th>Ad Soyad</th><th>Rol</th><th class="sayi">ELO</th><th>Katılma</th><th>Ayrılma</th>
<th></th></tr></thead>
        <tbody>
        {% for k in kadro %}
            <tr class="{{ 'soluk' if k.ayrilma_tarihi }}">
                <td><a href="{{ url_for('oyuncu_detay', oyuncu_id=k.oyuncu_id) }}">{{ k.takma_ad }}</a></td>
                <td>{{ k.ad_soyad }}</td>
                <td>{{ k.rol }} {% if k.kaptan %}<span class="rozet sari">Kaptan</span>{% endif %}</td>
                <td class="sayi">{{ k.elo_puani }}</td>
                <td>{{ k.katilma_tarihi|tarih }}</td>
                <td>{{ k.ayrilma_tarihi|tarih }}</td>
                <td style="white-space:nowrap">
                    {% if not k.ayrilma_tarihi %}
                        {% if not k.kaptan %}
                        <form class="satir-ici" method="post" action="{{ url_for('kaptan_yap', takim_id=takim.takim_id,
kadro_id=k.kadro_id) }}">
                            <button class="btn btn-ikincil btn-kucuk">Kaptan Yap</button>
                        </form>
                        {% endif %}
                        <form class="satir-ici" method="post" action="{{ url_for('oyuncu_transfer', oyuncu_id=k.oyuncu_id)
}}"
                              onsubmit='return confirm({{ (k.takma_ad ~ " kadrodan çıkarılsın mı?")|tojson }});'>
                            <input type="hidden" name="takim_id" value="">
                            <input type="hidden" name="donus" value="{{ url_for('takim_detay', takim_id=takim.takim_id)
}}">
                            <button class="btn btn-tehlike btn-kucuk">Kadrodan Çıkar</button>
                        </form>
                    {% endif %}
                </td>
            </tr>
        {% else %}
            <tr><td colspan="7" class="bos">Kadroda oyuncu yok.</td></tr>
        {% endfor %}
        </tbody>
    </table>
```

```
    <h2 style="margin-top:20px">Kadroya Oyuncu Ekle</h2>
    <form class="kucuk-form" method="post" action="{{ url_for('kadro_ekle', takim_id=takim.takim_id) }}">
        <div>
            <label>Serbest Oyuncu</label>
            <select name="oyuncu_id" required>
                <option value="">— Seçin —</option>
                {% for o in serbest_oyuncular %}
                    <option value="{{ o.kisi_id }}">{{ o.takma_ad }}</option>
                {% endfor %}
            </select>
        </div>
        <div><label>Rol</label><input name="rol" placeholder="Oyuncu"></div>
        <button class="btn" type="submit">Ekle</button>
    </form>
</section>

<div class="izgara izgara-2">
    <section class="panel">
        <h2>Turnuvalar</h2>
        <table>
            <thead><tr><th>Turnuva</th><th>Durum</th><th class="sayi">G-M</th><th class="sayi">Puan</th><th
class="sayi">Sıra</th><th class="sayi">Ödül</th></tr></thead>
            <tbody>
            {% for t in turnuvalar %}
                <tr>
                    <td><a href="{{ url_for('turnuva_detay', turnuva_id=t.turnuva_id) }}">{{ t.turnuva_adi }}</a></td>
                    <td>{{ durum_rozeti(t.durum) }}</td>
                    <td class="sayi">{{ t.galibiyet }}-{{ t.maglubiyet }}</td>
                    <td class="sayi">{{ t.puan }}</td>
                    <td class="sayi">{{ t.sira or '-' }}</td>
                    <td class="sayi">{{ t.odul|para }}</td>
                </tr>
            {% else %}
                <tr><td colspan="6" class="bos">Turnuva kaydı yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>
    </section>

    <section class="panel">
        <h2>Sponsorlar</h2>
        <table>
            <thead><tr><th>Sponsor</th><th>Sektör</th><th>Süre</th><th class="sayi">Yıllık</th></tr></thead>
            <tbody>
            {% for s in sponsorlar %}
                <tr>
                    <td>{{ s.sponsor_adi }}</td>
                    <td>{{ s.sektor }}</td>
                    <td>{{ s.baslangic|tarih }} – {{ s.bitis|tarih }}</td>
                    <td class="sayi">{{ s.yillik_tutar|para }}</td>
                </tr>
            {% else %}
                <tr><td colspan="4" class="bos">Sponsor yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>
    </section>
</div>
{% endblock %}
```

templates/turnuvalar.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu, durum_rozeti, tur_rozeti %}
{% block baslik %}Turnuvalar{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
    <div>
        <h1>Turnuvalar</h1>
        <p class="alt-baslik">Online ve salon turnuvaları (kalıtım: turnuva → online_turnuva / salon_turnuva).</p>
    </div>
    <a class="btn" href="{{ url_for('turnuva_ekle') }}">+ Yeni Turnuva</a>
</div>

<section class="panel">
```

```
    <form class="arama" method="get">
        <input type="text" name="q" value="{{ aranan }}" placeholder="Turnuva adı ile ara...">
        <select name="durum" style="max-width:200px">
            <option value="">Tüm durumlar</option>
            {% for d in ['Kayıt Açık', 'Devam Ediyor', 'Tamamlandı'] %}
                <option {{ 'selected' if secili_durum == d }}>{{ d }}</option>
            {% endfor %}
        </select>
        <button class="btn" type="submit">Ara</button>
        {% if aranan or secili_durum %}<a class="btn btn-ikincil" href="{{ url_for('turnuvalar') }}">Temizle</a>{% endif
%}
    </form>

    <div class="tablo-kap">
    <table>
        <thead><tr><th>Turnuva</th><th>Oyun</th><th>Tür</th><th>Yer / Platform</th><th>Tarih</th>
            <th class="sayi">Takım</th><th class="sayi">Ödül Havuzu</th><th>Durum</th><th></th></tr></thead>
        <tbody>
        {% for t in kayitlar %}
            <tr>
                <td><a href="{{ url_for('turnuva_detay', turnuva_id=t.turnuva_id) }}"><strong>{{ t.turnuva_adi }}</strong>
</a></td>
                <td>{{ t.oyun_adi }}</td>
                <td>{{ tur_rozeti(t.turnuva_turu) }}</td>
                <td>{% if t.turnuva_turu == 'O' %}{{ t.yayin_platformu }} · {{ t.sunucu_bolgesi }}{% else %}{{ t.mekan_adi
}}, {{ t.sehir }}{% endif %}</td>
                <td style="white-space:nowrap">{{ t.baslangic_tarihi|tarih }} – {{ t.bitis_tarihi|tarih }}</td>
                <td class="sayi">{{ t.takim_sayisi }}/{{ t.max_takim }}</td>
                <td class="sayi">{{ t.odul_havuzu|para }}</td>
                <td>{{ durum_rozeti(t.durum) }}</td>
                <td style="white-space:nowrap">
                    <a class="btn btn-ikincil btn-kucuk" href="{{ url_for('turnuva_duzenle', turnuva_id=t.turnuva_id)
}}">Düzenle</a>
                    {{ sil_butonu(url_for('turnuva_sil', turnuva_id=t.turnuva_id), t.turnuva_adi ~ ' silinsin mi? Maçları
ve kayıtları da silinir.') }}
                </td>
            </tr>
        {% else %}
            <tr><td colspan="9" class="bos">Turnuva bulunamadı.</td></tr>
        {% endfor %}
        </tbody>
    </table>
    </div>
</section>
{% endblock %}
```

templates/turnuva_form.html

```
{% extends "base.html" %}
{% block baslik %}{{ 'Yeni Turnuva' if yeni else 'Turnuva Düzenle' }}{% endblock %}
{% block icerik %}
<h1>{{ 'Yeni Turnuva Oluştur' if yeni else 'Turnuva Bilgilerini Güncelle' }}</h1>
<p class="alt-baslik">
    {% if yeni %}Kayıt <code>turnuva_olustur()</code> fonksiyonu ile turnuva türüne göre ilgili alt tabloya da eklenir.
    {% else %}Turnuva türü sonradan değiştirilemez.{% endif %}
</p>

{% set tur = deger.get('turnuva_turu', 'O') %}
<form class="panel" method="post">
    <div class="form-izgara">
        <fieldset class="tam">
            <legend>Genel Bilgiler (turnuva)</legend>
            <div class="form-izgara">
                <div class="tam"><label>Turnuva Adı</label><input name="turnuva_adi" required maxlength="80" value="{{
deger.get('turnuva_adi', '') }}"></div>
                {% if yeni %}
                <div>
                    <label>Oyun</label>
                    <select name="oyun_id" required>
                        {% for o in oyunlar %}
                            <option value="{{ o.oyun_id }}" {{ 'selected' if deger.get('oyun_id')|string ==
o.oyun_id|string }}>{{ o.oyun_adi }}</option>
                        {% endfor %}
                    </select>
                </div>
```

```
                <div>
                    <label>Turnuva Türü</label>
                    <select name="turnuva_turu" id="tur" onchange="turDegisti()">
                        <option value="O" {{ 'selected' if tur == 'O' }}>Online</option>
                        <option value="S" {{ 'selected' if tur == 'S' }}>Salon</option>
                    </select>
                </div>
                {% else %}
                <div><label>Oyun</label><input value="{{ deger.oyun_adi }}" disabled></div>
                <div><label>Turnuva Türü</label><input value="{{ 'Online' if tur == 'O' else 'Salon' }}" disabled></div>
                {% endif %}
                <div><label>Başlangıç Tarihi</label><input name="baslangic_tarihi" type="date" required value="{{
deger.get('baslangic_tarihi', '') }}"></div>
                <div><label>Bitiş Tarihi</label><input name="bitis_tarihi" type="date" required value="{{
deger.get('bitis_tarihi', '') }}"></div>
                <div><label>Ödül Havuzu (₺)</label><input name="odul_havuzu" type="number" min="0" step="0.01" value="{{
deger.get('odul_havuzu', 0) }}"></div>
                <div><label>En Fazla Takım</label><input name="max_takim" type="number" min="2" max="64" required value="
{{ deger.get('max_takim', 8) }}"></div>
            </div>
        </fieldset>

        <fieldset class="tam" id="online-alanlar">
            <legend>Online Turnuva Bilgileri (online_turnuva)</legend>
            <div class="form-izgara">
                <div><label>Yayın Platformu</label><input name="yayin_platformu" maxlength="30" placeholder="Twitch,
YouTube, Kick..." value="{{ deger.get('yayin_platformu') or '' }}"></div>
                <div><label>Sunucu Bölgesi</label><input name="sunucu_bolgesi" maxlength="30" placeholder="Avrupa
(İstanbul)" value="{{ deger.get('sunucu_bolgesi') or '' }}"></div>
            </div>
        </fieldset>

        <fieldset class="tam" id="salon-alanlar">
            <legend>Salon Turnuvası Bilgileri (salon_turnuva)</legend>
            <div class="form-izgara">
                <div>
                    <label>Mekan</label>
                    <select name="mekan_id">
                        {% for m in mekanlar %}
                            <option value="{{ m.mekan_id }}" {{ 'selected' if deger.get('mekan_id')|string ==
m.mekan_id|string }}>{{ m.mekan_adi }} – {{ m.sehir }} ({{ m.kapasite }} kişi)</option>
                        {% endfor %}
                    </select>
                </div>
                <div><label>Bilet Fiyatı (₺)</label><input name="bilet_fiyati" type="number" min="0" step="0.01" value="{{
deger.get('bilet_fiyati') or 0 }}"></div>
            </div>
        </fieldset>
    </div>

    <div class="form-alt">
        <button class="btn" type="submit">{{ 'Oluştur' if yeni else 'Güncelle' }}</button>
        <a class="btn btn-ikincil" href="{{ url_for('turnuvalar') if yeni else url_for('turnuva_detay',
turnuva_id=turnuva_id) }}">Vazgeç</a>
    </div>
</form>

<script>
    function turDegisti() {
        var secim = document.getElementById('tur');
        var tur = secim ? secim.value : {{ tur|tojson }};
        document.getElementById('online-alanlar').style.display = tur === 'O' ? '' : 'none';
        document.getElementById('salon-alanlar').style.display = tur === 'S' ? '' : 'none';
    }
    turDegisti();
</script>
{% endblock %}
```

templates/turnuva_detay.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu, durum_rozeti, tur_rozeti %}
{% block baslik %}{{ turnuva.turnuva_adi }}{% endblock %}
{% block icerik %}
<div class="baslik-satiri">
```

```
    <div>
        <h1>{{ turnuva.turnuva_adi }}</h1>
        <p class="alt-baslik">{{ turnuva.oyun_adi }} · {{ tur_rozeti(turnuva.turnuva_turu) }} · {{
durum_rozeti(turnuva.durum) }}</p>
    </div>
    <div>
        <a class="btn btn-ikincil" href="{{ url_for('turnuva_duzenle', turnuva_id=turnuva.turnuva_id) }}">Düzenle</a>
        {{ sil_butonu(url_for('turnuva_sil', turnuva_id=turnuva.turnuva_id), turnuva.turnuva_adi ~ ' silinsin mi?',
'Turnuvayı Sil') }}
    </div>
</div>

<div class="izgara izgara-2">
    <section class="panel">
        <h2>Turnuva Bilgileri</h2>
        <dl class="bilgi-listesi">
            <dt>Tarih</dt><dd>{{ turnuva.baslangic_tarihi|tarih }} – {{ turnuva.bitis_tarihi|tarih }}</dd>
            <dt>Ödül Havuzu</dt><dd>{{ turnuva.odul_havuzu|para }}</dd>
            <dt>Takımlar</dt><dd>{{ turnuva.takim_sayisi }} / {{ turnuva.max_takim }}</dd>
            {% if turnuva.turnuva_turu == 'O' %}
                <dt>Yayın Platformu</dt><dd>{{ turnuva.yayin_platformu }}</dd>
                <dt>Sunucu Bölgesi</dt><dd>{{ turnuva.sunucu_bolgesi }}</dd>
            {% else %}
                <dt>Mekan</dt><dd>{{ turnuva.mekan_adi }}, {{ turnuva.sehir }} ({{ turnuva.kapasite }} kişi)</dd>
                <dt>Bilet Fiyatı</dt><dd>{{ turnuva.bilet_fiyati|para }}</dd>
            {% endif %}
        </dl>

        {% if turnuva.durum == 'Kayıt Açık' %}
            <h2 style="margin-top:20px">Fikstür Oluştur</h2>
            <p class="soluk"><code>fikstur_olustur()</code> fonksiyonu kayıtlı takımlar için lig usulü maçları oluşturur
ve turnuvayı başlatır.</p>
            <form class="kucuk-form" method="post" action="{{ url_for('fikstur', turnuva_id=turnuva.turnuva_id) }}">
                <div><label>İlk Maç Zamanı</label><input type="datetime-local" name="ilk_mac" required value="{{
turnuva.baslangic_tarihi|html_tarih }}T18:00"></div>
                <div>
                    <label>Maç Aralığı</label>
                    <select name="aralik">
                        <option value="2 hours">2 saat</option>
                        <option value="6 hours">6 saat</option>
                        <option value="1 day" selected>1 gün</option>
                    </select>
                </div>
                <button class="btn btn-basari" type="submit">Fikstürü Oluştur</button>
            </form>
        {% elif turnuva.durum == 'Devam Ediyor' %}
            <h2 style="margin-top:20px">Turnuvayı Bitir</h2>
            <p class="soluk">Tüm maçlar oynandıktan sonra turnuva tamamlanır; tetikleyici sıralamayı yazar ve ödülleri
dağıtır.</p>
            <form method="post" action="{{ url_for('turnuva_tamamla', turnuva_id=turnuva.turnuva_id) }}">
                <button class="btn btn-basari" type="submit">Turnuvayı Tamamla</button>
            </form>
        {% endif %}
    </section>

    <section class="panel">
        <h2>Puan Tablosu</h2>
        <p class="soluk"><code>puan_tablosu()</code> fonksiyonu ile hesaplanır.</p>
        <table>
            <thead><tr><th>#</th><th>Takım</th><th class="sayi">O</th><th class="sayi">G</th><th class="sayi">M</th><th
class="sayi">Av.</th><th class="sayi">Puan</th><th></th></tr></thead>
            <tbody>
            {% for p in puan_tablosu %}
                <tr>
                    <td>{{ p.sira }}</td>
                    <td><a href="{{ url_for('takim_detay', takim_id=p.takim_id) }}">{{ p.takim_adi }}</a></td>
                    <td class="sayi">{{ p.oynanan }}</td>
                    <td class="sayi">{{ p.galibiyet }}</td>
                    <td class="sayi">{{ p.maglubiyet }}</td>
                    <td class="sayi">{{ '%+d'|format(p.skor_farki) }}</td>
                    <td class="sayi"><strong>{{ p.puan }}</strong></td>
                    <td>
                        {% if turnuva.durum == 'Kayıt Açık' %}
                            {{ sil_butonu(url_for('katilim_sil', turnuva_id=turnuva.turnuva_id, takim_id=p.takim_id),
p.takim_adi ~ ' turnuvadan çıkarılsın mı?', 'Çıkar') }}
                        {% endif %}
                    </td>
```

```
                </tr>
            {% else %}
                <tr><td colspan="8" class="bos">Henüz kayıtlı takım yok.</td></tr>
            {% endfor %}
            </tbody>
        </table>

        {% if turnuva.durum == 'Kayıt Açık' %}
            <h2 style="margin-top:20px">Takım Kaydet</h2>
            <form class="kucuk-form" method="post" action="{{ url_for('katilim_ekle', turnuva_id=turnuva.turnuva_id) }}">
                <div>
                    <label>Takım</label>
                    <select name="takim_id" required>
                        {% for t in kayit_olabilir %}<option value="{{ t.takim_id }}">{{ t.takim_adi }}</option>{% endfor
%}
                    </select>
                </div>
                <button class="btn" type="submit">Kaydet</button>
            </form>
        {% endif %}
    </section>
</div>

<section class="panel">
    <h2>Maçlar</h2>
    <div class="tablo-kap">
    <table>
        <thead><tr><th>Zaman</th><th>Aşama</th><th class="sayi">Takım 1</th><th style="text-align:center">Skor</th>
<th>Takım 2</th><th>Hakem</th><th>Durum</th><th></th></tr></thead>
        <tbody>
        {% for m in maclar %}
            <tr>
                <td style="white-space:nowrap">{{ m.mac_zamani|tarih }}</td>
                <td>{{ m.asama_adi }}</td>
                <td class="sayi {{ 'kazanan' if m.kazanan_id == m.takim1_id }}">{{ m.t1_adi }}</td>
                <td style="text-align:center; white-space:nowrap">
                    {% if m.durum == 'Oynandı' %}
                        <strong>{{ m.takim1_skor }} - {{ m.takim2_skor }}</strong>
                    {% elif m.durum == 'Planlandı' and turnuva.durum == 'Devam Ediyor' %}
                        <form class="satir-ici" method="post" action="{{ url_for('mac_sonuc', mac_id=m.mac_id) }}">
                            <input type="hidden" name="donus" value="{{ url_for('turnuva_detay',
turnuva_id=turnuva.turnuva_id) }}">
                            <input class="skor-input" type="number" name="skor1" min="0" max="99" required>
                            <input class="skor-input" type="number" name="skor2" min="0" max="99" required>
                            <button class="btn btn-kucuk" type="submit">Kaydet</button>
                        </form>
                    {% else %}-{% endif %}
                </td>
                <td class="{{ 'kazanan' if m.kazanan_id == m.takim2_id }}">{{ m.t2_adi }}</td>
                <td>{{ m.hakem or '-' }}</td>
                <td>{{ durum_rozeti(m.durum) }}</td>
                <td><a class="btn btn-ikincil btn-kucuk" href="{{ url_for('mac_detay', mac_id=m.mac_id) }}">Detay</a></td>
            </tr>
        {% else %}
            <tr><td colspan="8" class="bos">Fikstür henüz oluşturulmadı.</td></tr>
        {% endfor %}
        </tbody>
    </table>
    </div>
</section>

<section class="panel">
    <h2>Ödüller</h2>
    <table>
        <thead><tr><th>Derece</th><th class="sayi">Tutar</th><th>Kazanan Takım</th><th></th></tr></thead>
        <tbody>
        {% for o in oduller %}
            <tr>
                <td>{{ o.derece }}.</td>
                <td class="sayi">{{ o.tutar|para }}</td>
                <td>{{ o.takim_adi or '— turnuva bitince belirlenecek —' }}</td>
                <td>{% if turnuva.durum != 'Tamamlandı' %}{{ sil_butonu(url_for('odul_sil', turnuva_id=turnuva.turnuva_id,
odul_id=o.odul_id), 'Ödül silinsin mi?') }}{% endif %}</td>
            </tr>
        {% else %}
            <tr><td colspan="4" class="bos">Ödül tanımlanmamış.</td></tr>
        {% endfor %}
```

```
        </tbody>
    </table>
    {% if turnuva.durum != 'Tamamlandı' %}
        <form class="kucuk-form" style="margin-top:14px" method="post" action="{{ url_for('odul_ekle',
turnuva_id=turnuva.turnuva_id) }}">
            <div><label>Derece</label><input type="number" name="derece" min="1" required value="{{ oduller|length + 1
}}"></div>
            <div><label>Tutar (₺)</label><input type="number" name="tutar" min="1" step="0.01" required></div>
            <button class="btn" type="submit">Ödül Ekle</button>
        </form>
    {% endif %}
</section>
{% endblock %}
```

templates/mac_detay.html

```
{% extends "base.html" %}
{% from "_makrolar.html" import sil_butonu, durum_rozeti %}
{% block baslik %}Maç #{{ mac.mac_id }}{% endblock %}
{% block icerik %}
<p><a href="{{ url_for('turnuva_detay', turnuva_id=mac.turnuva_id) }}">← {{ mac.turnuva_adi }}</a></p>

<section class="panel" style="text-align:center">
    <p class="soluk">{{ mac.asama_adi }} · {{ mac.mac_zamani|tarih }} · Hakem: {{ mac.hakem or '-' }} · {{
durum_rozeti(mac.durum) }}</p>
    <div style="display:flex; justify-content:center; align-items:center; gap:30px; font-size:26px; font-weight:800; flex-
wrap:wrap">
        <a class="{{ 'kazanan' if mac.kazanan_id == mac.takim1_id }}" href="{{ url_for('takim_detay',
takim_id=mac.takim1_id) }}">{{ mac.t1_adi }}</a>
        <span style="font-size:40px">{{ mac.takim1_skor if mac.takim1_skor is not none else '-' }} : {{ mac.takim2_skor if
mac.takim2_skor is not none else '-' }}</span>
        <a class="{{ 'kazanan' if mac.kazanan_id == mac.takim2_id }}" href="{{ url_for('takim_detay',
takim_id=mac.takim2_id) }}">{{ mac.t2_adi }}</a>
    </div>
    {% if mac.durum == 'Planlandı' %}
        <form method="post" action="{{ url_for('mac_sonuc', mac_id=mac.mac_id) }}" style="margin-top:16px">
            <input class="skor-input" type="number" name="skor1" min="0" max="99" required>
            <input class="skor-input" type="number" name="skor2" min="0" max="99" required>
            <button class="btn" type="submit">Sonucu Kaydet</button>
        </form>
        <p class="soluk">Sonuç <code>mac_sonucu_gir</code> saklı yordamı ile girilir.</p>
    {% endif %}
</section>

<section class="panel">
    <h2>Oyuncu İstatistikleri</h2>
    <table>
        <thead><tr><th>Takım</th><th>Oyuncu</th><th class="sayi">Öldürme</th><th class="sayi">Ölüm</th><th
class="sayi">Asist</th><th class="sayi">KDA</th><th></th><th></th></tr></thead>
        <tbody>
        {% for s in istatistikler %}
            <tr>
                <td><span class="rozet mor">{{ s.kisa_ad }}</span></td>
                <td><a href="{{ url_for('oyuncu_detay', oyuncu_id=s.oyuncu_id) }}">{{ s.takma_ad }}</a></td>
                <td class="sayi">{{ s.oldurme }}</td>
                <td class="sayi">{{ s.olum }}</td>
                <td class="sayi">{{ s.asist }}</td>
                <td class="sayi">{{ s.kda }}</td>
                <td>{% if s.mvp %}<span class="rozet sari">MVP</span>{% endif %}</td>
                <td>{{ sil_butonu(url_for('istatistik_sil', mac_id=mac.mac_id, istatistik_id=s.istatistik_id), 'İstatistik
silinsin mi?') }}</td>
            </tr>
        {% else %}
            <tr><td colspan="8" class="bos">İstatistik girilmemiş.</td></tr>
        {% endfor %}
        </tbody>
    </table>

    {% if mac.durum == 'Oynandı' and oyuncular %}
        <h2 style="margin-top:20px">İstatistik Ekle</h2>
        <form class="kucuk-form" method="post" action="{{ url_for('istatistik_ekle', mac_id=mac.mac_id) }}">
            <div>
                <label>Oyuncu</label>
                <select name="oyuncu_id" required>
                    {% for o in oyuncular %}<option value="{{ o.kisi_id }}">{{ o.kisa_ad }} · {{ o.takma_ad }}</option>{%
```

```
endfor %}
                </select>
            </div>
            <div><label>Öldürme</label><input type="number" name="oldurme" min="0" value="0" required></div>
            <div><label>Ölüm</label><input type="number" name="olum" min="0" value="0" required></div>
            <div><label>Asist</label><input type="number" name="asist" min="0" value="0" required></div>
            <div style="flex:0"><label>MVP</label><input type="checkbox" name="mvp"></div>
            <button class="btn" type="submit">Ekle</button>
        </form>
    {% endif %}
</section>
{% endblock %}
```

templates/islem_gunlugu.html

```
{% extends "base.html" %}
{% block baslik %}İşlem Günlüğü{% endblock %}
{% block icerik %}
<h1>İşlem Günlüğü</h1>
<p class="alt-baslik">kisi, takim ve turnuva tablolarındaki değişiklikler <code>trg_islem_log</code> tetikleyicisi ile
otomatik kaydedilir (son 100 kayıt).</p>

<section class="panel">
    <form class="arama" method="get">
        <select name="tablo" style="max-width:220px">
            <option value="">Tüm tablolar</option>
            {% for t in ['kisi', 'takim', 'turnuva'] %}
                <option {{ 'selected' if secili_tablo == t }}>{{ t }}</option>
            {% endfor %}
        </select>
        <button class="btn" type="submit">Filtrele</button>
    </form>
    <div class="tablo-kap">
    <table>
        <thead><tr><th>#</th><th>Zaman</th><th>Tablo</th><th>İşlem</th><th>Kullanıcı</th><th>Kayıt</th></tr></thead>
        <tbody>
        {% for k in kayitlar %}
            <tr>
                <td>{{ k.log_id }}</td>
                <td style="white-space:nowrap">{{ k.islem_zamani|tarih }}</td>
                <td>{{ k.tablo_adi }}</td>
                <td><span class="rozet {{ {'INSERT': 'yesil', 'UPDATE': 'sari', 'DELETE': 'gri'}[k.islem_turu] }}">{{
k.islem_turu }}</span></td>
                <td>{{ k.kullanici }}</td>
                <td><code style="font-size:12px; word-break:break-all">{{ k.kayit_bilgisi[:160] }}{% if
k.kayit_bilgisi|length > 160 %}…{% endif %}</code></td>
            </tr>
        {% else %}
            <tr><td colspan="6" class="bos">Kayıt yok.</td></tr>
        {% endfor %}
        </tbody>
    </table>
    </div>
</section>
{% endblock %}
```

static/style.css

```
:root {
    --bg: #0d1020;
    --panel: #161a2e;
    --panel-2: #1d2240;
    --border: #2a3058;
    --text: #e7e9f5;
    --muted: #9aa0c3;
    --accent: #7c5cff;
    --accent-2: #22d3ee;
    --success: #22c55e;
    --danger: #ef4444;
    --warning: #f59e0b;
    --radius: 10px;
}
```

```
* { box-sizing: border-box; }

body {
    margin: 0;
    background: var(--bg);
    color: var(--text);
    font-family: "Segoe UI", system-ui, -apple-system, Roboto, sans-serif;
    font-size: 15px;
    line-height: 1.5;
}

a { color: var(--accent-2); text-decoration: none; }
a:hover { text-decoration: underline; }

/* ---------- Üst menü ---------- */
.ust-menu {
    background: linear-gradient(90deg, #1a1440, #0f1a3a);
    border-bottom: 1px solid var(--border);
}
.ust-menu .kap {
    display: flex; align-items: center; gap: 28px;
    max-width: 1200px; margin: 0 auto; padding: 14px 20px;
}
.logo { font-weight: 800; font-size: 20px; color: #fff; letter-spacing: .5px; }
.logo span { color: var(--accent-2); }
.ust-menu nav { display: flex; gap: 6px; flex-wrap: wrap; }
.ust-menu nav a {
    color: var(--muted); padding: 7px 14px; border-radius: 8px; font-weight: 600;
}
.ust-menu nav a:hover, .ust-menu nav a.aktif {
    color: #fff; background: rgba(124, 92, 255, .22); text-decoration: none;
}

main { max-width: 1200px; margin: 0 auto; padding: 24px 20px 60px; }

h1 { font-size: 26px; margin: 0 0 4px; }
h2 { font-size: 18px; margin: 0 0 14px; }
.alt-baslik { color: var(--muted); margin: 0 0 20px; }

.baslik-satiri {
    display: flex; justify-content: space-between; align-items: flex-start;
    gap: 16px; flex-wrap: wrap; margin-bottom: 18px;
}

/* ---------- Paneller ---------- */
.panel {
    background: var(--panel); border: 1px solid var(--border);
    border-radius: var(--radius); padding: 18px 20px; margin-bottom: 20px;
}
.izgara { display: grid; gap: 20px; }
.izgara-2 { grid-template-columns: repeat(2, minmax(0, 1fr)); }
.izgara-3 { grid-template-columns: repeat(3, minmax(0, 1fr)); }
@media (max-width: 860px) { .izgara-2, .izgara-3 { grid-template-columns: 1fr; } }

.kartlar { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px, 1fr)); gap: 16px; margin-bottom: 22px; }
.kart {
    background: linear-gradient(145deg, var(--panel-2), var(--panel));
    border: 1px solid var(--border); border-radius: var(--radius); padding: 16px 18px;
}
.kart .deger { font-size: 28px; font-weight: 800; color: #fff; }
.kart .etiket { color: var(--muted); font-size: 13px; text-transform: uppercase; letter-spacing: .6px; }

/* ---------- Tablolar ---------- */
table { width: 100%; border-collapse: collapse; }
th, td { padding: 9px 10px; text-align: left; border-bottom: 1px solid var(--border); vertical-align: middle; }
th { color: var(--muted); font-size: 12px; text-transform: uppercase; letter-spacing: .6px; font-weight: 700; }
tbody tr:hover { background: rgba(255, 255, 255, .03); }
td.sayi, th.sayi { text-align: right; font-variant-numeric: tabular-nums; white-space: nowrap; }
.tablo-kap { overflow-x: auto; }
.bos { color: var(--muted); text-align: center; padding: 22px; }

/* ---------- Formlar ---------- */
form.satir-ici { display: inline; }
.arama { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 16px; }
.arama input[type=text] { flex: 1; min-width: 220px; }

label { display: block; font-size: 13px; color: var(--muted); margin-bottom: 5px; font-weight: 600; }
```

```
input, select, textarea {
    width: 100%; background: #0f1328; color: var(--text);
    border: 1px solid var(--border); border-radius: 8px; padding: 9px 11px; font: inherit;
}
input:focus, select:focus { outline: 2px solid var(--accent); border-color: transparent; }
input[type=checkbox] { width: auto; }
.form-izgara { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 14px 18px; }
.form-izgara .tam { grid-column: 1 / -1; }
@media (max-width: 700px) { .form-izgara { grid-template-columns: 1fr; } }
.form-alt { display: flex; gap: 10px; margin-top: 20px; }
fieldset { border: 1px dashed var(--border); border-radius: var(--radius); padding: 14px 16px; margin: 0; }
legend { color: var(--accent-2); font-weight: 700; padding: 0 6px; }
.kucuk-form { display: flex; gap: 8px; align-items: flex-end; flex-wrap: wrap; }
.kucuk-form > div { flex: 1; min-width: 120px; }
.skor-input { width: 58px; padding: 6px; text-align: center; }

/* ---------- Butonlar ---------- */
.btn {
    display: inline-block; border: 0; border-radius: 8px; padding: 9px 16px;
    font: inherit; font-weight: 700; cursor: pointer; color: #fff;
    background: var(--accent); white-space: nowrap;
}
.btn:hover { filter: brightness(1.12); text-decoration: none; }
.btn-ikincil { background: #2b3160; }
.btn-tehlike { background: var(--danger); }
.btn-basari { background: #16a34a; }
.btn-kucuk { padding: 5px 10px; font-size: 13px; }

/* ---------- Rozetler ---------- */
.rozet {
    display: inline-block; padding: 2px 9px; border-radius: 999px;
    font-size: 12px; font-weight: 700; background: #2b3160; color: var(--text);
    white-space: nowrap;
}
.rozet.yesil { background: rgba(34, 197, 94, .18); color: #4ade80; }
.rozet.mavi { background: rgba(34, 211, 238, .16); color: var(--accent-2); }
.rozet.mor { background: rgba(124, 92, 255, .22); color: #b9a8ff; }
.rozet.gri { background: rgba(154, 160, 195, .16); color: var(--muted); }
.rozet.sari { background: rgba(245, 158, 11, .18); color: #fbbf24; }
.kazanan { color: #4ade80; font-weight: 800; }
.soluk { color: var(--muted); }

/* ---------- Bildirimler ---------- */
.bildirim { padding: 12px 16px; border-radius: var(--radius); margin-bottom: 16px; font-weight: 600; }
.bildirim.basari { background: rgba(34, 197, 94, .14); border: 1px solid rgba(34, 197, 94, .5); color: #86efac; }
.bildirim.hata { background: rgba(239, 68, 68, .14); border: 1px solid rgba(239, 68, 68, .55); color: #fca5a5; }

.bilgi-listesi { display: grid; grid-template-columns: max-content 1fr; gap: 7px 18px; margin: 0; }
.bilgi-listesi dt { color: var(--muted); }
.bilgi-listesi dd { margin: 0; font-weight: 600; }

footer { text-align: center; color: var(--muted); font-size: 13px; padding: 20px; border-top: 1px solid var(--border); }
```